



Convierte un documento Word o ODT en un sitio estático HTML compatible MkDocs con las IA Gemini 3 y ChatGPT 5.2

Serge Tahénero de 2026

[| English version](#) | [| Spanish version](#) | [| German version](#) |

Este sitio ha sido creado con el conversor [Word o ODT - > HTML] creado por eles IA Gemini 3 y ChatGPT 5.2 en enero de 2026 y se describen en este documento.

Con el conversor Gemini/ChatGPT se han creado varios sitios web a partir de documentos ODT de LibreOffice y de documentos de Word:

Java

- [\[Introducción al lenguaje Java \(1998\)\]](#);
- [\[Introducción a la programación web en Java con servlets y páginas JSP \(2002\)\]](#);
- [\[Introducción a STRUTS 1.x con ejemplos \(2003\)\]](#);
- [\[Fundamentos del desarrollo web MVC en Java con ejemplos \(2006\)\]](#);
- [\[Persistencia en Java 5 a través de la práctica \(2007\)\]](#);
- [\[Creación de un servicio web Java EE con el IDE NetBeans 6.5 y el servidor Java EE GlassFish \(2009\)\]](#);
- [\[Introducción a STRUTS 2 con ejemplos \(2012\)\]](#);
- [\[Introducción con ejemplos a Java Server Faces, PrimeFaces y PrimeFaces Mobile \(2012\)\]](#);
- [\[Introducción a Java EE con el IDE NetBeans y el servidor de aplicaciones GlassFish \(2012\)\]](#);
- [\[Introducción al lenguaje Java y al ecosistema Spring a través de un caso práctico \(2015\)\]](#);
- [\[Introducción a Spring MVC y Thymeleaf con ejemplos \(2015\)\]](#)
- [\[Aprovechamiento de una base de datos relacional con el ecosistema Spring \(2015\)\]](#)

.NET

- [\[Introducción al lenguaje VB.NET a través de ejemplos \(2004\)\]](#);
- [\[Desarrollo web con ASP.NET 1.1 \(2004\)\]](#);
- [\[Spring IoC para .NET \(2005\)\]](#);
- [\[Creación de una aplicación web de tres capas con Spring.NET y VB.NET \(2005\)\]](#);
- [\[Aprendizaje del lenguaje C# versión 3.0 con el Framework .NET 3.5 \(2008\)\]](#);
- [\[Introducción al framework NHibernate para la plataforma .NET \(2011\)\]](#);
- [\[Introducción con ejemplos a Entity Framework 5 Code First \(2012\)\]](#);
- [\[Introducción al framework ASP.NET MVC a través de ejemplos \(2013\)\]](#);

JavaScript

- [\[Introducción al lenguaje ECMASCRIPT6 con ejemplos \(2019\)\]](#);
- [\[Introducción al framework VUE.JS con ejemplos \(2019\)\]](#);
- [\[Introducción al framework NUXT.JS con ejemplos \(2019\)\]](#);

PHP

- [\[Metodología de desarrollo MVC de una aplicación web PHP4 \(2004\)\]](#);
- [\[Introducción al lenguaje PHP5 con ejemplos \(2011\)\]](#);
- [\[Introducción al lenguaje PHP7 con ejemplos \(2019\)\]](#);

Python

- [\[Introducción al lenguaje Python 2.7 con ejemplos \(2012\)\]](#);
- [\[Introducción al lenguaje Python y al framework web Flask con ejemplos \(2020\)\]](#);
- [\[Generar un script de Python con herramientas de IA \(2025\)\]](#);

VBScript

- [\[Introducción al lenguaje VBScript \(2002\)\]](#);

SQL

- [\[Introducción al lenguaje SQL con el SGBD Firebird \(2006\)\]](#);

Todos ellos son cursos antiguos de varios cientos de páginas. El tamaño del documento ODT o DOCX no importa al conversor Gemini / ChatGPT. Lo que le importa son las estructuras que se encuentran en él (véase el capítulo: [Los ejemplos de este documento](#)).

1. Introducción

El PDF de este documento está disponible en nuestro sitio web | [AQUÍ](#) |.

Los convertidores Gemini 3 / ChatGPT 5.2 [Palabra o ODT → HTML] son disponibles en | [AQUÍ](#) |.

Este documento le ofrece dos conversores:

- un convertidor de LibreOffice ODT a HTML;
 - un conversor de Word DOCX a HTML;
-

El propósito de este artículo es proporcionar al lector un conversor en Python de documentos Word o ODT a un sitio estático HTML. Este conversor ha sido construido inicialmente por el IA [Gemini 3](#) entonces por [ChatGPT 5.2](#). Se necesitaron 356 iteraciones para estos dos IA para producir el convertidor de este documento. Para ello fue necesario depurar varias semanas. Gemini 3 hizo todo el trabajo al principio. Al principio necesité varias docenas de iteraciones para conseguir que la primera versión fuera más o menos correcta. Luego añadí regularmente un nuevo problema planteado por nuevos documentos ODT. Y Gemini retrocedía a menudo. En otras palabras, lo que funcionaba en la etapa N ya no funcionaba en las etapas siguientes. Entonces procedí del siguiente modo: en cuanto Gemini producía un convertidor que resolvía uno de mis problemas, hacía una versión de referencia del mismo y se lo comunicaba a Gemini. He guardado esta referencia localmente. Entonces, cuando vi Gemini regresar durante demasiado tiempo le pedí que volviera a la última versión de referencia conocida dándoselo. Así fue como, poco a poco, construimos juntos este convertidor: yo le expresé lo que quería, esencialmente señalándole las anomalías que encontraba en el sitio HTML producido, y él produjo el código solicitado.

Utilicé [Gemini 3](#) con una licencia pro a 22 euros/mes y [ChatGPT 5.2](#) de la misma manera.

Gemini / ChatGPT generará dos scripts Python:

- [convertir] para convertir el documento ODT de [LibreOffice](#) o el documento DOCX en Word in situ [MkDocs](#) ;
- [build] para convertir el sitio MkDocs en el sitio estático HTML ;

Nunca miré el código generado. Quería pensar en ellos como cajas negras. Usted no necesita ser un desarrollador de Python para seguir este tutorial, o incluso un desarrollador en absoluto.

Proponiendo mejoras para el convertidor en IA Gemini 3 tiene a veces bloqueado. No fue posible realizar las mejoras solicitadas. En ese momento, estábamos en la versión estable V316. Para poder avanzar, entregué esta versión operativa a [ChatGPT 5.2](#) solicitando las mejoras deseadas. ChatGPT modificó correctamente el código de Gemini para satisfacer mis nuevas peticiones. Por eso considero que estos dos IA son los que generaron el convertidor.

Luego seguí utilizando esta técnica. Cuando un IA bloqueaba una función, le daba al otro IA la última versión estable conocida.

Los dos IA tienen métodos diferentes para entregar el código Python solicitado:

- Gemini proporciona el código para el script generado en la página de consulta. En debe luego copia y pega este código;
- ChatGPT proporciona un enlace para descargar el script generado ;

Más de 1000 líneas de código generado Gemini mostró graves deficiencias. Debido a limitaciones técnicas específicas de este IA, no pudo desplegarse, en la página de consulta, todo el código generado. Muy a menudo faltaban líneas de código. Debido a esta limitación, en algún momento ya no fue posible utilizar Gemini 3. Así que ChatGPT 5.2 terminó de escribir script.

También fue ChatGPT quien generó el conversor de Word a HTML. Le di el convertidor de ODT a HTML que funcionaba y le pedí que lo adaptara para un documento de Word. Lo hizo en 18 iteraciones. Es una constante con estos dos IA: entienden muy bien los scripts de Python que se les dan y pueden hacer cambios en ellos mejoras. Para

mí, es la mejor manera de trabajar con ellos.

2. Ejemplos de este documento

Me gustaría hacer de esto un artículo corto. Las interacciones entre un IA y un usuario se presentan en el artículo [[Generar un script Python con herramientas de IA](#) que ahora llamaré [[ref1](#)]. Interacciones con Gemini y ChatGPT sólo se presentarán al margen. En cualquier caso, era imposible presentar todas las iteraciones.

A continuación daré algunos ejemplos de las particularidades de mis ODT / documentos DOCX que el Gemini / ChatGPT gestiona correctamente. Es este documento el que propondremos para la conversión HTML a script de Gemini ChatGPT. Veremos qué hace con él.

2.1. Las listas

El Gemini / ChatGPT sabe gestionar listas numeradas y con viñetas aunque estén entrelazados :

2.1.1. Listas

- Tema 1;
- Tema 2;
- Tema 3;
 - Elemento 3.1;
 - Elemento 3.1.1
 - Elemento 3.1.2
 - Elemento 3.1.2.1
 - Elemento 3.1.2.2
 - Elemento 3.2;
- Tema 4;

2.1.2. Listas numeradas

1. Tema 1;
2. Tema 2;
 - a. Elemento 2.1
 - i. Elemento 2.1.1
 1. Elemento 2.1.1.1
 2. Elemento 2.1.1.2
 - ii. Elemento 2.1.2
 - b. Elemento 2.2
3. Tema 3;

2.1.3. Listas mixtas 1

- Tema 1;
- Tema 2;
- Tema 3;
 - Elemento 3.1;
 1. Elemento 3.1.1
 2. Elemento 3.1.2
 - Elemento 3.1.2.1

- Elemento 3.1.2.2
 - Elemento 3.2;
- Tema 4;

2.1.4. Listas mixtas 2

1. Tema 1;
2. Tema 2;
 - a. Elemento 2.1
 - i. Elemento 2.1.1
 - Elemento 2.1.1.1
 - Elemento 2.1.1.2
 - ii. Elemento 2.1.2
 - b. Elemento 2.2
3. Tema 3;

2.1.5. Listas numeradas manualmente

Numeración manual significa ici que el usuario establece el número de un párrafo numerado: [clic derecho sobre el párrafo numerado / párrafo / párrafo / reiniciar numeración / empezar por].

Empiezo una lista con un número distinto de 1.

6. Elemento 6
7. elemento 7

Ici Rompo la lista para decir algo, pero luego quiero seguir numerando.

8. Elemento 8
9. elemento 9

Luego empiezo una nueva lista numerada:

11. elemento 11
12. elemento 12

2.2. Bloques de códigos

Mis cursos contienen muchos bloques de código. A menudo se trata de códigos enriquecidos (negrita, colores de las palabras clave) por IDE (Eclipse, PyCharm, WebStorm, Netbeans). Estos códigos enriquecidos son reproducidos de forma idéntica por el conversor.

Cuando el código no está enriquecido (código del Bloc de notas o ...), el Convertidor Gemini / ChatGPT lo reconoce (Java, C#, XML, HTML, ...) utilizando palabras clave de idioma en un archivo de configuración. Cuando reconoce un idioma, inserta un marcador (valla) para MkDocs de modo que éste adapta el resaltado sintáctico del código al lenguaje utilizado en el bloque de código.

2.2.1. Bloques de código enriquecidos (Eclipse, Visual Studio, etc.)

He aquí algunos bloques de código enriquecidos con diferentes IDE :

Java

```
1. package istia.st.spring.core;
2.
3. import java.util.ArrayList;
```

```

4.     import java.util.List;
5.
6.     import org.springframework.context.ApplicationContext;
7.     import org.springframework.context.support.ClassPathXmlApplicationContext;
8.
9.     public class Demo01 {
10.
11.         @SuppressWarnings({ "unchecked", "resource" })
12.         public static void main(String[] args) {
13.             // recuperación del contexto de Spring
14.             ApplicationContext ctx = new ClassPathXmlApplicationContext("config-01.xml");
15.             // recuperamos las judías
16.             Personne p01 = ctx.getBean("personne_01", Personne.class);
17.             Personne p02 = ctx.getBean("personne_02", Personne.class);
18.             List<Personne> club = ctx.getBean("club", new
ArrayList<Personne>().getClass());
19.             Appartement appart01 = ctx.getBean(Appartement.class);
20.             ...

```

C#

```

11.     using System;
12.
13.     namespace Chap1 {
14.         class Impots {
15.             static void Main(string[] args) {
16.                 // tablas de datos necesarios para calcular el impuesto
17.                 decimal[] limites = { 4962M, 8382M, 14753M, 23888M, 38868M, 47932M, 0M
};
18.
19.                 decimal[] coeffR = { 0M, 0.068M, 0.191M, 0.283M, 0.374M, 0.426M,
0.481M };
20.
21.                 decimal[] coeffN = { 0M, 291.09M, 1322.92M, 2668.39M, 4846.98M,
6883.66M, 9505.54M };
22.
23.                 // se restablece el estado civil
24.                 bool OK = false;
25.                 string reponse = null;
26.                 while (!OK) {
27.                     Console.WriteLine("Etes-vous marié(e) (O/N) ? ");
28.                     reponse = Console.ReadLine().Trim().ToLower();
29.                     if (reponse != "o" && reponse != "n")
30.                         Console.Error.WriteLine("Réponse incorrecte.
Recommencez");
31.                     else OK = true;
32.                 } //mientras que
33.                 bool marie = reponse == "o";
34.             }
35.         }
36.     }
37.

```

Python

```

21. # -----
22. def affiche(chaine):
23.     # cartel cadena
24.     print("chaine=%s" % chaine)
25.
26. # -----
27. def affiche_type(variable):
28.     # muestra el tipo de variable
29.     print("type[%s]=%s" % (variable, type(variable)))
30.
31. # -----
32. def f1(param):
33.     # añade 10 a param
34.     return param + 10
35.

```

```

38.
39. # -----
40. def f2():
41.     # devuelve una tupla de 3 valores
42.     return "un", 0, 100
43.
44.
45. # ----- programa principal -----
46. ...

```

PHP

```

31. <?php
32.
33. // tipos estrictos para los parámetros de las funciones
34. declare(strict_types=1);
35.
36. // constantes globales
37. define("PLAFOND_QF_DEMI_PART", 1551);
38. define("PLAFOND_REVENUS_CELIBATAIRE_POUR_REDUCTION", 21037);
39. define("PLAFOND_REVENUS_COUPLE_POUR_REDUCTION", 42074);
40. define("VALEUR_REDOC_DEMI_PART", 3797);
41. define("PLAFOND_DECOTE_CELIBATAIRE", 1196);
42. define("PLAFOND_DECOTE_COUPLE", 1970);
43. define("PLAFOND_IMPOT_COUPLE_POUR_DECOTE", 2627);
44. define("PLAFOND_IMPOT_CELIBATAIRE_POUR_DECOTE", 1595);
45. define("ABATTEMENT_DIXPOURCENT_MAX", 12502);
46. define("ABATTEMENT_DIXPOURCENT_MIN", 437);
47.
48. // definición de constantes locales
49. $DATA = "taxpayersdata.txt";
50. $RESULTATS = "resultats.txt";
51. $limites = array(9964, 27519, 73779, 156244, 0);
52. $coeffR = array(0, 0.14, 0.3, 0.41, 0.45);
53. $coeffN = array(0, 1394.96, 5798, 13913.69, 20163.45);
54.
55. // datos de lectura
56. $data = fopen($DATA, "r");
57. if (!$data) {
58.     print "Impossible d'ouvrir en lecture le fichier des données [$DATA]\n";
59.     exit;
60. }
61.
62. ...

```

ECMAScript

```

41. 'use strict';
42. // este es un comentario
43. // constante
44. const nom = "dupont";
45. // una pantalla de visualización
46. console.log("nom : ", nom);
47. // una matriz con elementos de distintos tipos
48. const tableau = ["un", "deux", 3, 4];
49. // su número de elementos
50. let n = tableau.length;
51. // un bucle
52. for (let i = 0; i < n; i++) {
53.     console.log("tableau[" + i + "] = ", tableau[i]);
54. }
55. // inicializar 2 variables con el contenido de un array
56. let [chaine1, chaine2] = ["chaine1", "chaine2"];
57. // concatenación de las 2 cadenas
58. const chaine3 = chaine1 + chaine2;
59. // visualización de resultados
60. console.log([chaine1, chaine2, chaine3]);

```

61. ...

VBScript

```
51. ' cálculo de la deuda tributaria del contribuyente
52. ' el programa debe llamarse con tres parámetros: salario de los hijos casados
53. ' casado: carácter Y si está casado, N si no lo está
54. ' hijos: número de hijos
55. ' salario: salario anual sin céntimos
56.
57. ' declaración obligatoria de variables
58. Option Explicit
59. Dim erreur
60.
61. ' recuperar los argumentos y comprobar su validez
62. Dim marie, enfants, salaire
63. erreur=getArguments(marie,enfants,salaire)
64. ' ¿Error?
65. If erreur(0)<>0 Then wscript.echo erreur(1) : wscript.quit erreur(0)
66.
67. ' recuperamos los datos necesarios para calcular los impuestos
68. Dim limites, coeffR, coeffN
69. erreur=getData(limites,coeffR,coeffN)
70. ' ¿Error?
71. If erreur(0)<>0 Then wscript.echo erreur(1) : wscript.quit 5
72.
73. ' se muestra el resultado
74. wscript.echo "impôt=" & calculerImpot(marie,enfants,salaire,limites,coeffR,coeffN)
75.
76. ' salimos sin error
77. wscript.quit 0
```

XML

```
61. <?xml version="1.0" encoding="utf-8" ?>
62. <configuration>
63.
64.     <configSections>
65.         <sectionGroup name="spring">
66.             <section name="context" type="Spring.Context.Support.ContextHandler, Spring.Core" />
67.             <section name="objects" type="Spring.Context.Support.DefaultSectionHandler,
Spring.Core" />
68.         </sectionGroup>
69.     </configSections>
70.
71.     <spring>
72.         <context>
73.             <resource uri="config://spring/objects" />
74.         </context>
75.         <objects xmlns="http://www.springframework.net">
76.             <object name="dao" type="Dao.DataBaseImpot, ImpotsV7-dao">
77.                 <constructor-arg index="0" value="MySQL.Data.MySqlClient"/>
78.                 <constructor-arg index="1"
value="Server=localhost;Database=bdimpots;Uid=admimpots;Pwd=mdpimpots;"/>
79.                 <constructor-arg index="2" value="select limite, coeffr, coeffn from tranches"/>
80.             </object>
81.             <object name="metier" type="Metier.ImpotMetier, ImpotsV7-metier">
82.                 <constructor-arg index="0" ref="dao"/>
83.             </object>
84.         </objects>
85.     </spring>
86. </configuration>
```

2.2.2. Bloques de códigos bruto (plain text)

He aquí algunos ejemplos de código bruto :

Resultados

Observe que el código no empieza por la línea 1.

```
7. C:\Data\st-2020\dev\python\cours-2020\python3-flask-2020\venv\Scripts\python.exe
   C:/Data/st-2020/dev/python/cours-2020/python3-flask-2020/bases/bases_01.py
8. nom=dupont
9. liste[0]=un
10. liste[1]=deux
11. liste[2]=3
12. liste[3]=4
13. [chaine1,chaine2,chaine1chaine2]
14. chaine=chaine1
15. type[4]=<class 'int'>
16. type[chaine1]=<class 'str'>
17. type[['un', 'deux', 3, 4]]=<class 'list'>
18. type[a changé]=<class 'str'>
19. res1=14s
20. (res1,res2,res3)=[un,0,100]
21. liste[0]=un
22. liste[1]=0
23. liste[2]=100
24. liste[0]=8
25. liste[1]=5
26. somme=13
27.
28. Process finished with exit code 0
```

Este caso requirió docenas de iteraciones. El conversor nunca encontraba el número de la primera línea de código. Finalmente le pregunté a Gemini cómo formatear el bloque de código para que lo reconociera. Esto es lo que me dijo

Dans LibreOffice :

1. Placez votre curseur sur la **première ligne** de votre bloc de code numéroté (celle qui doit être le n°5).
2. Faites un **Clic-Droit > Paragraphe > Paragraphe...**
3. Allez dans l'onglet **Plan & numérotation** (ou *Outline & Numbering*).
4. Cochez la case **Recommencer la numérotation** (*Restart numbering at this paragraph*).
5. Cochez/Réglez **Commencer à** (*Start with*) sur **5**.
6. Cliquez sur **OK** et sauvegardez votre ODT.

Pourquoi ça marche ? Cette méthode oblige LibreOffice à inscrire un attribut explicite sur le paragraphe ou l'item de liste (`text:start-value="5"`), ce que le script détecte immédiatement. La méthode via l'icône de liste ("Personnaliser") modifie parfois seulement le style visuel sans changer la valeur logique sous-jacente dans le XML, surtout sur les documents importés de Word.

He seguido este procedimiento y ha funcionado. Utilicé el método a través del icono de la lista (Personalizar). Así que puede que tenga que modificar ciertos elementos del Documento ODT / DOCX para obtener lo que desea.

El conversor puede manejar bloques de código sin numerar.

```
liste[0]=un
liste[1]=0
liste[2]=100
liste[0]=8
```

```
liste[1]=5
somme=13
```

He aquí los mismos ejemplos que en el párrafo 2.2.1 pero no enriquecido. Por tanto, son las palabras clave del archivo de configuración las que guiarán al conversor hacia el idioma adecuado.

Java

```
1. package istia.st.spring.core;
2.
3. import java.util.ArrayList;
4. import java.util.List;
5.
6. import org.springframework.context.ApplicationContext;
7. import org.springframework.context.support.ClassPathXmlApplicationContext;
8.
9. public class Demo01 {
10.
11.     @SuppressWarnings({ "unchecked", "resource" })
12.     public static void main(String[] args) {
13.         // recuperación del contexto de Spring
14.         ApplicationContext ctx = new ClassPathXmlApplicationContext("config-01.xml");
15.         // recuperamos las judías
16.         Personne p01 = ctx.getBean("personne_01", Personne.class);
17.         Personne p02 = ctx.getBean("personne_02", Personne.class);
18.         List<Personne> club = ctx.getBean("club", new ArrayList<Personne>().getClass());
19.         Appartement appart01 = ctx.getBean(Appartement.class);
20.         ...
```

C#

```
11. using System;
12.
13. namespace Chap1 {
14.     class Impots {
15.         static void Main(string[] args) {
16.             // tablas de datos necesarios para calcular el impuesto
17.             decimal[] limites = { 4962M, 8382M, 14753M, 23888M, 38868M, 47932M, 0M
18. };
19.             decimal[] coeffR = { 0M, 0.068M, 0.191M, 0.283M, 0.374M, 0.426M,
20. 0.481M };
21.             decimal[] coeffN = { 0M, 291.09M, 1322.92M, 2668.39M, 4846.98M,
22. 6883.66M, 9505.54M };
23.
24.             // se restablece el estado civil
25.             bool OK = false;
26.             string reponse = null;
27.             while (!OK) {
28.                 Console.Write("Etes-vous marié(e) (O/N) ? ");
29.                 reponse = Console.ReadLine().Trim().ToLower();
30.                 if (reponse != "o" && reponse != "n")
31.                     Console.Error.WriteLine("Réponse incorrecte.
32. Recommencez");
33.                 else OK = true;
34.             } //mientras que
35.             bool marie = reponse == "o";
36.             ...
```

Python

```
21. # -----
22. def affiche(chaine):
23.     # cartel cadena
24.     print("chaine=%s" % chaine)
25.
26.
```

```

27.  # -----
28.  def affiche_type(variable):
29.      # muestra el tipo de variable
30.      print("type[%s]=%s" % (variable, type(variable)))
31.
32.
33.  # -----
34.  def f1(param):
35.      # añade 10 a param
36.      return param + 10
37.
38.
39.  # -----
40.  def f2():
41.      # devuelve una tupla de 3 valores
42.      return "un", 0, 100
43.
44.
45.  # ----- programa principal -----
46.  ...

```

PHP

```

31.  <?php
32.
33.  // tipos estrictos para los parámetros de las funciones
34.  declare(strict_types=1);
35.
36.  // constantes globales
37.  define("PLAFOND_QF_DEMI_PART", 1551);
38.  define("PLAFOND_REVENUS_CELIBATAIRE_POUR_REDUCTION", 21037);
39.  define("PLAFOND_REVENUS_COUPLE_POUR_REDUCTION", 42074);
40.  define("VALEUR_REDOC_DEMI_PART", 3797);
41.  define("PLAFOND_DECOTE_CELIBATAIRE", 1196);
42.  define("PLAFOND_DECOTE_COUPLE", 1970);
43.  define("PLAFOND_IMPOT_COUPLE_POUR_DECOTE", 2627);
44.  define("PLAFOND_IMPOT_CELIBATAIRE_POUR_DECOTE", 1595);
45.  define("ABATTEMENT_DIXPOURCENT_MAX", 12502);
46.  define("ABATTEMENT_DIXPOURCENT_MIN", 437);
47.
48.  // definición de constantes locales
49.  $DATA = "taxpayersdata.txt";
50.  $RESULTATS = "resultats.txt";
51.  $limites = array(9964, 27519, 73779, 156244, 0);
52.  $coeffR = array(0, 0.14, 0.3, 0.41, 0.45);
53.  $coeffN = array(0, 1394.96, 5798, 13913.69, 20163.45);
54.
55.  // datos de lectura
56.  $data = fopen($DATA, "r");
57.  if (!$data) {
58.      print "Impossible d'ouvrir en lecture le fichier des données [$DATA]\n";
59.      exit;
60.  }
61.
62.  ...

```

ECMAScript

```

41.  'use strict';
42.  // este es un comentario
43.  // constante
44.  const nom = "dupont";
45.  // una pantalla de visualización
46.  console.log("nom : ", nom);
47.  // una matriz con elementos de distintos tipos
48.  const tableau = ["un", "deux", 3, 4];
49.  // su número de elementos

```

```

50. let n = tableau.length;
51. // un bucle
52. for (let i = 0; i < n; i++) {
53.     console.log("tableau[" + i + "] = " + tableau[i]);
54. }
55. // inicializar 2 variables con el contenido de un array
56. let [chaine1, chaine2] = ["chaine1", "chaine2"];
57. // concatenación de las 2 cadenas
58. const chaine3 = chaine1 + chaine2;
59. // visualización de resultados
60. console.log([chaine1, chaine2, chaine3]);
61. ...

```

VBScript

```

51. ' cálculo de la deuda tributaria del contribuyente
52. ' el programa debe llamarse con tres parámetros: salario de los hijos casados
53. ' casado: carácter Y si está casado, N si no lo está
54. ' hijos: número de hijos
55. ' salario: salario anual sin céntimos
56.
57. ' declaración obligatoria de variables
58. Option Explicit
59. Dim erreur
60.
61. ' recuperar los argumentos y comprobar su validez
62. Dim marie, enfants, salaire
63. erreur=getArguments(marie,enfants,salaire)
64. ' ¿Error?
65. If erreur(0)<>0 Then wscript.echo erreur(1) : wscript.quit erreur(0)
66.
67. ' recuperamos los datos necesarios para calcular los impuestos
68. Dim limites, coeffR, coeffN
69. erreur=getData(limites,coeffR,coeffN)
70. ' ¿Error?
71. If erreur(0)<>0 Then wscript.echo erreur(1) : wscript.quit 5
72.
73. ' se muestra el resultado
74. wscript.echo "impôt=" & calculerImpot(marie,enfants,salaire,limites,coeffR,coeffN)
75.
76. ' salimos sin error
77. wscript.quit 0

```

XML

```

61. <?xml version="1.0" encoding="utf-8" ?>
62. <configuration>
63.
64.     <configSections>
65.         <sectionGroup name="spring">
66.             <section name="context" type="Spring.Context.Support.ContextHandler, Spring.Core" />
67.             <section name="objects" type="Spring.Context.Support.DefaultSectionHandler,
Spring.Core" />
68.         </sectionGroup>
69.     </configSections>
70.
71.     <spring>
72.         <context>
73.             <resource uri="config://spring/objects" />
74.         </context>
75.         <objects xmlns="http://www.springframework.net">
76.             <object name="dao" type="Dao.DataBaseImpot, ImpotsV7-dao">
77.                 <constructor-arg index="0" value="MySQL.Data.MySqlClient"/>
78.                 <constructor-arg index="1"
value="Server=localhost;Database=bdimpots;Uid=admimpots;Pwd=mdpimpots;"/>
79.                 <constructor-arg index="2" value="select limite, coeffr, coeffn from tranches"/>
80.             </object>

```



```

81.         <object name="metier" type="Metier.ImpotMetier, ImpotsV7-metier">
82.             <constructor-arg index="0" ref="dao"/>
83.         </object>
84.     </objects>
85. </spring>
86. </configuration>

```

HTML

```

71. <!DOCTYPE HTML>
72. <HTML>
73.     <head>
74.         <title>Laragon</title>
75.
76.         <link href="https://fonts.googleapis.com/css?family=Karla:400" rel="stylesheet"
77.             type="text/css">
78.
79.         <style>
80.             HTML, body {
81.                 height: 100%;
82.             }
83.
84.             body {
85.                 margin: 0;
86.                 padding: 0;
87.                 width: 100%;
88.                 display: table;
89.                 font-weight: 100;
90.                 font-family: 'Karla';
91.             }
92.
93.             .container {
94.                 text-align: center;
95.                 display: table-cell;
96.                 vertical-align: middle;
97.             }
98.
99.             .content {
100.                 text-align: center;
101.                 display: inline-block;
102.             }
103.
104.             .title {
105.                 font-size: 96px;
106.             }
107.
108.             .opt {
109.                 margin-top: 30px;
110.             }
111.
112.             .opt a {
113.                 text-decoration: none;
114.                 font-size: 150%;
115.             }
116.
117.             a:hover {
118.                 color: red;
119.             }
120.     </style>
121. </head>
122. <body>
123.     <div class="container">
124.         <div class="content">
125.             <div class="title" title="Laragon">Laragon</div>
126.
127.             <div class="info"><br />
128.                 Apache/2.4.35 (Win64) OpenSSL/1.1.0i PHP/7.2.11<br />
129.                 PHP version: 7.2.11 <span><a title="phpinfo()" href="/?
130.                 q=info">info</a></span><br />

```

```
129. Document Root: C:/myprograms/laragon-lite/www<br />
130.
131. </div>
```

2.3. Enlaces

El Gemini / ChatGPT sabe cómo mantener los enlaces externos en el Documento ODT / DOCX. Por ejemplo [Gemini 3](#) o [\[Generar un script Python con herramientas de IA\]](#).

Puede gestionar un enlace a un capítulo [Enlace a un capítulo](#)

Un referencia cruzada a un capítulo : 2.1.1.

Referencia a un marcador de texto que precede a : Gemini 3

Referencia a un marcador de texto que sigue : GitHub

2.4. Enriquecimiento del texto

El conversor puede manejar negrita, cursiva, subrayado y resaltado. Respeta el color del resalte.

Un texto con palabras en **grasa** en *cursiva*, subrayado o destacado o destacado o destacado.

Lo mismo ocurre con los enlaces: [\[Genere a script Python con herramientas de IA\]](#).

Le **convertidor** *gestiona* también el **color** de **caracteres**.

También gestiona los bordes superior e inferior de los párrafos.

- También gestiona los bordes superior e inferior de los párrafos.
 - También gestiona los bordes superior e inferior de los párrafos.
 - También gestiona los bordes superior e inferior de los párrafos.
-

1. También gestiona los bordes superior e inferior de los párrafos.
 2. También gestiona los bordes superior e inferior de los párrafos.
 3. También gestiona los bordes superior e inferior de los párrafos.
-

10. Un texto con palabras en **grasa** en *cursiva*, subrayado o destacado o destacado o destacado.
11. Lo mismo ocurre con los enlaces: [\[Genere a script Python con herramientas de IA\]](#).
12. Le **convertidor** *gestiona* también el **color** de **caracteres**.

- Un texto con palabras en **grasa** en *cursiva*, subrayado o destacado o destacado o destacado.
- Lo mismo ocurre con los enlaces: [\[Genere a script Python con herramientas de IA\]](#).
- Le **convertidor** *gestiona* también el **color** de **caracteres**.

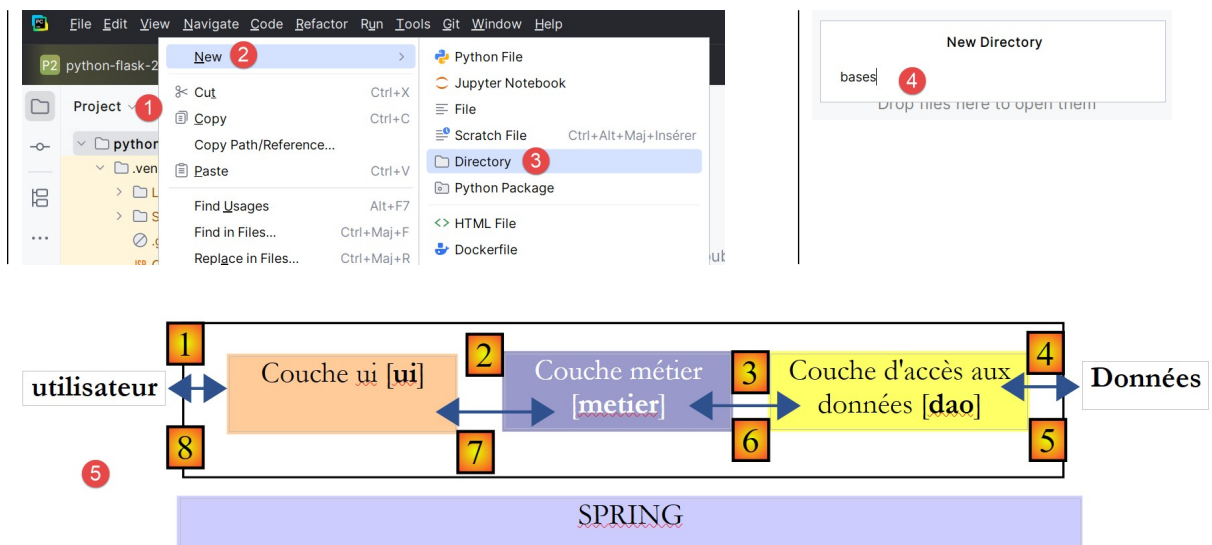
También gestiona el fondo del párrafo

- También gestiona el fondo del párrafo
- 1. también gestiona el fondo del párrafo

2.5. A título puede ser también *enriquecido*.

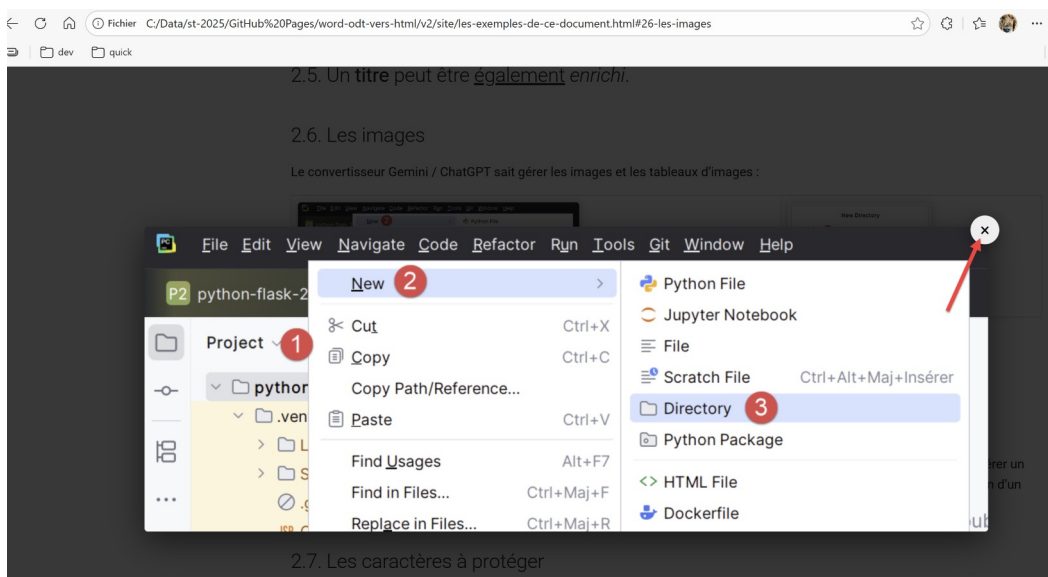
2.6. Las imágenes

El Gemini / ChatGPT sabe gestionar imágenes y tablas de imágenes :



Los planos son habituales en los documentos ODT. A pesar de docenas de intentos, Gemini no consiguió generar un script que generara la imagen (como una captura de pantalla) del dibujo. Así que arriba, la imagen 5 es una captura de pantalla de un dibujo de un documento ODT.

Puede hacer clic en todas las imágenes para ampliarlas. Si hace clic en la imagen [1-3] anterior, obtendrá la siguiente ampliación:



2.7. Los caracteres que hay que proteger

Una página web MkDocs¹ tiene páginas cuyo contenido no es HTML sino Markdown. Si el Documento ODT / DOCX contiene caracteres que existen en Markdown, pueden ser interpretados por MkDocs y por lo tanto no ser renderizados como se espera. He aquí dos ejemplos:

¹MkDocs

El asterisco * significa Markdown. La siguiente línea puede malinterpretarse:

El impuesto I es igual a **0.15*R - 2072.5*nbParts**.

Otro ejemplo es cuando quieres insertar un bloque de código Markdown en tu documento como este:

```
1.      # Convertidor de Word/ODT a sitio HTML (MkDocs)
2.
3.
4.      🔗 **[Ver el sitio de demostración generado](https://stahe.github.io/word-odt-vers-html-
janv-2026/)**
5.
6.
7.      ---
8.
9.      ## 📄 Descripción
10.
11.
12.      El objetivo de este proyecto es poner a disposición del lector un conversor en Python de
documentos Word u ODT a un sitio web HTML estático.
13.
14.
15.      Cuando el documento ODT/DOCX es adecuado, el conversor genera un sitio web HTML mediante
**MkDocs** que tiene el aspecto profesional de los sitios web generados por Pandoc.
16.
17.      ## 📄 Contexto de creación
18.
19.      Este conversor ha sido creado íntegramente por la IA **Gemini 3** (con una suscripción pro). Es
el resultado de sucesivas iteraciones para gestionar con precisión la estructura de los documentos ODT
(OpenDocument Text).
20.
21.      ## ✨ Funcionalidades
22.
23.      El script `convert.py` realiza las siguientes acciones:
24.
25.
26.      * **Conversión de ODT a Markdown**: Analiza el archivo `.odt` (XML) para extraer su
estructura.
27.      * **Gestión de títulos**: Genera automáticamente la tabla de contenidos (TOC) y la navegación
lateral.
28.
29.      * **Bloques de código**: Detección automática de lenguajes, coloración sintáctica y **gestión
precisa de la numeración de líneas** (atributos `start-value`).
30.      * **Listas**: Compatibilidad con listas con viñetas y numeradas con la sangría correcta.
31.
32.      * **Formato**: Compatibilidad con *negrita*, *cursiva*, *subrayado* y *resaltado* (respetando
los colores originales).
33.      * **Imágenes**: Extracción e integración automática de las imágenes contenidas en el
documento.
34.      * **Configuración**: Personalización mediante un archivo `config.py` (pie de página, Google
Analytics, etc.).
35.
36.
37.      ## 🚀 Instalación
38.
39.
40.      ### Requisitos previos
41.
42.
43.      * Python 3.x
44.      * Las siguientes bibliotecas:
45.
46.
47.      ```bash
48.      pip install odfpy unicode mkdocs mkdocs-material
49.
50.
```

2.8. Los cuadros

Una tabla puede tener diferentes contenidos:

1

2



```

1. package istia.st.spring.core;
2.
3. import java.util.ArrayList;
4. import java.util.List;
5.
6. import
org.springframework.context.ApplicationContext;
7. import
org.springframework.context.support.ClassPathXmlApplication
Context;
8.
9. public class Demo01 {
10.
11.

```

3

4



```

1. # -----
2. def affiche(chaine):
3.     # cartel cadena
4.     print("chaine=%s" % chaine)
5.
6.
7. # -----
8. def affiche_type(variable):
9.     # muestra el tipo de variable
10.    print("type[%s]=%s" % (variable,
type(variable)))
11.
12.

```

Una tabla con enlaces :

El Gemini / ChatGPT sabe cómo mantener enlaces externos en el Documento ODT / DOCX. Por ejemplo [Gemini 3](#) o [\[Generar un script Python con herramientas de IA\]](#).

Un referencia cruzada a un capítulo : 2.1.1.

Referencia a un marcador de texto que precede a : Gemini 3

Referencia a un marcador de texto que sigue : GitHub

Puede gestionar un enlace a un capítulo [Enlace a un capítulo](#)

Una tabla cuyas celdas contienen otra tabla :

`html:form` se utiliza para generar la etiqueta HTML `<formulario>` y facilitar información al responsable del tratamiento que deberá tramitar este formulario:

<code>action</code>	url donde se enviarán los valores del formulario
<code>name</code>	nombre del formulario HTML. Este es también el nombre del bean que almacenará sus valores
<code>type</code>	nombre de la clase que debe instanciarse para obtener el frijol de almacenamiento de formularios

Observe que no se especifica el método para enviar los parámetros del formulario (GET/POST) al controlador. Esto podría hacerse utilizando el método **method**. En su defecto, el **POST** que se utiliza por defecto.

`html:text` se utiliza para generar la etiqueta `<input type="text" value="...">` :

`property` nombre del campo bean del formulario que se asociará a la zona de entrada. Cuando el formulario se envía al servidor (cliente -> servidor), el campo bean tomará el valor del campo de entrada. Cuando se muestre el formulario (servidor -> cliente), el valor contenido en el campo bean se mostrará en el campo convulsión.

También gestiona el fondo del párrafo en una tabla

- También gestiona el fondo del párrafo en una tabla
- 1. también gestiona el fondo del párrafo en una tabla

2.9. Notas a pie de página

El convertidor Gemini² / ChatGPT gestiona las notas a pie de página. He aquí otra nota³ pie de página.

3. ¿Qué existe? en internet

Antes de empezar, puede preguntar al IA qué hay disponible en el mercado para convertir gratuitamente un documento Word o ODT en un sitio estático HTML.

Preguntemos a Gemini:

Existe-t-il sur internet un logiciel gratuit capable de convertir un document Word en site statique HTML ?

He aquí su respuesta:

Sí, existen varias soluciones gratuitas, pero la elección depende de su definición de "sitio estático" (una simple página HTML frente a un sitio completo con navegación) y de sus conocimientos técnicos.

Aquí están las mejores opciones clasificadas por método:

- **1. La solución "todo en uno" (para crear un sitio realmente navegable)**

Si desea transformar un largo documento de Word (con capítulos y subcapítulos) en un sitio web completo con un menú de navegación a la izquierda, éste es el mejor option.

- **Software : HelpNDoc** (Versión gratuita para uso personal)
- **¿Cómo funciona?** Se trata de un software de creación de documentación. Usted importa su archivo Word, él analiza la estructura (Títulos 1, Títulos 2, etc.) y genera automáticamente un sitio estático y responsivo HTML5.
- **Ventajas :** Cree un sitio real con menú, barra de búsqueda e índice sin escribir una sola línea de código.
- **El menos :** La versión gratuita añade un pequeño banner "Generado por HelpNDoc" en la parte inferior de la página.
- **2. La solución "Limpiador en línea" (Para convertir página por página)**

²[Google Gemini](#)

³La nota a pie de página

Si tiene un documento corto y sólo quiere obtener el código HTML para ponerlo en una página existente, los conversores en línea son ideales para limpiar el código "sucio" generado por Word.

- **Herramienta : WordToHTML.net** (o Word2CleanHTML)
- **¿Cómo funciona?**
 1. Copia tu texto en Word.
 2. Péguelo en el editor visual de la parte izquierda del sitio.
 3. El sitio genera un código HTML limpio (limpio de etiquetas Microsoft innecesarias) en la ventana de la derecha.
- **Ventajas :** Muy sencillo, visual, elimina la "grasa" del código de Microsoft.
- **El menos :** Ideal para una sola página, no para generar la arquitectura de todo un sitio.
- **3. La solución "Pro / Developer" (La mejor calidad)**

Este es el método recomendado por los expertos para obtener un código ultralimpio y duradero.

- **Software : Pandoc** (Línea de comandos)
- **¿Cómo funciona?** Se trata de un pequeño programa sin interfaz gráfica (para ser utilizado a través de la terminal / símbolo del sistema). Convierte formatos con mucha precisión.
- **Pedido estándar :** `pandoc mon_document.docx -t html -o mon_site.html`
- **Ventajas :** Código HTML5 perfecto, gratuito, de código abierto, maneja muy bien las imágenes y las fórmulas matemáticas (LaTeX).
- **El menos :** Requiere comodidad con un símbolo del sistema (pantalla en negro).
- **4. La solución "Système D" (Microsoft Word)**

Puede hacerlo directamente desde Word, pero tenga cuidado con los resultados.

- **Método :** Archivo > Guardar como > Elegir "**Página Web filtrada (*.htm; *.html)**".
- **Información importante:** No elija "Página Web con archivo único (.mht)", elija "Filtrado". El option "Filtrado" reduce (un poco) el código superfluo que Word suele añadir.
- **El menos :** El código suele ser engorroso y difícil de modificar a posteriori.

- **Cuadro comparativo rápido**

Solución	Tipo de resultado	Código de calidad	Fácil	Ideal para...
HelpNDoc	Sitio web completo (con menú)	Bien	Media	Manuales, tesis, documentación
WordToHTML	Una página	Excelente	Muy fácil	Entradas de blog, páginas individuales
Pandoc	Página única (en bruto)	Perfecto	Difícil	Desarrolladores, archivo limpio

Solución	Tipo de resultado	Código de calidad	Fácil	Ideal para...
Palabra (Nativo)	Una página	Equivocado	Fácil	Solución rápida de problemas

- **Mi consejo**

Si su objetivo es publicar este documento en línea de manera profesional, le sugiero que **HelpNDoc** si eres principiante y quieres una estructura de sitio de inmediato.

Si sólo desea el contenido limpio para pegarlo en otro lugar (WordPress, etc.), utilice **WordToHTML.net**.

¿Quiere que le detalle el procedimiento paso a paso de una de estas herramientas (por ejemplo WordToHTML)?

A veces utilizaba [Pandoc], pero me resultaba difícil de usar. Recuerdo que tenía que "preparar" el documento de Word para que Pandoc lo manejara correctamente. Pero el resultado fue excelente. **HelpNDoc** parece un programa atractivo, ya que genera un sitio con un menú. Lo probé en uno de mis documentos de Word y el resultado no fue satisfactorio.

Ahora vamos a ver si existe algún software de conversión para documentos ODT:

Existe-t-il sur internet un logiciel gratuit capable de transformer un document ODT en site HTML statique ?

La respuesta de Gemini es la siguiente:

Sí, así es. Al igual que con Word, el **ODT** (utilizado por LibreOffice y OpenOffice) tiene sus propias soluciones. Aquí tienes las mejores opciones gratuitas específicas para archivos ODT, ordenadas por método:

- **1. La solución "Extensión" (Mejor calidad integrada)**

Si utilizas LibreOffice o OpenOffice, la exportación básica de HTML es (como con Word) bastante pobre. Sin embargo, hay una gran extensión gratuita diseñada específicamente para esto.

- **Herramienta : Escritor2xhtml** (Ampliación de LibreOffice)

- **¿Cómo funciona?**

1. Descargue e instale la extensión en LibreOffice.
2. Archivo > Exportar y aparecerá un nuevo option "XHTML 1.1 + MathML" (o similar).

- **Ventajas :** El código es mucho más limpio que el de la exportación estándar y maneja muy bien las fórmulas matemáticas y los formatos complejos.

- **El menos :** Necesita instalar una extensión (operación técnica puntual).

- **2. La solución "Pro / Order line" (Universal)**

Es la misma herramienta que para Word, porque también destaca con el formato ODT.

- **Software : Pandoc**

- **¿Cómo funciona?** A través del terminal de control.

- **Cómo hacer un pedido :** `pandoc mon_fichier.odt -t html -o mon_site.html`

- **Ventajas :** Este es sin duda el conversor más potente del mercado para el formato ODT a HTML. "Entiende" la estructura del documento ODT mejor que nadie.
- **3. La solución "Sitio completo con navegación" (Vía HelpNDoc)**

HelpNDoc (mencionado anteriormente) sigue siendo el mejor option gratuito para crear un sitio real con un menú a la izquierda.

Sin embargo, HelpNDoc no siempre importa de forma nativa el formato .odt así como el .docx.

- **El truco:**
 1. Abra su archivo en LibreOffice.
 2. Guardar como... y seleccione **"Word 2007-365 (.docx)"**.
 3. Importe este archivo .docx en HelpNDoc para generar su sitio.
- **¿Por qué?** Esto garantiza que la jerarquía de sus capítulos (Títulos 1, Títulos 2) se reconozca perfectamente al crear el menú del sitio.
- **4. La solución en línea (Limpieza rápida)**

Algunas herramientas en línea aceptan ODT directamente y emiten un HTML limpio.

- **Herramienta : WordToHTML.net** (También acepta ODT)
- **¿Cómo funciona?** Puede cargar su archivo ODT directamente o pegar su texto.
- **Ventajas :** Muy práctico para convertir un solo elemento sin instalar nada.
- **Resumen de mis consejos para ODT :**
 1. **Para un sitio web real (Documentación/Tesis) :** Convierte tu ODT en **DOCX** a través de LibreOffice, entonces utilice **HelpNDoc** (este es el método más fiable para crear un menú de navegación).
 2. **Para una página limpia y sencilla:** Utiliza el **Escritor2xhtml** si lo haces a menudo, o **Pandoc** si desea automatizar la tarea.

¿Quiere que le explique cómo instalar el **Escritor2xhtml** en LibreOffice?

He probado la extensión LibreOffice [Writer2xhtml] en uno de mis documentos. Da algo legible pero no lo suficientemente profesional. No hay tabla de contenidos a la izquierda. Los bloques de código pierden su resaltado de sintaxis.

Al final, creo que el convertidor generado por Gemini es una alternativa como se ha propuesto anteriormente. El resultado es comparable al producido por Pandoc si el documento ODT es adecuado para el conversor.

4. El mensaje inicial a Gemini 3

Se supone ici que el lector tiene un conocimiento inicial de IA [Gemini](#). Si no es el caso, le aconsejamos que lea el documento [\[ref1\]](#). El prompt inicial dado a Gemini estaba en la forma del siguiente archivo de texto [Prompt User.txt]:

Solicitudes del usuario

- 1.
2. - Crea un script de Python [convert] para generar un sitio MkDocs con el tema «material» a partir de un documento ODT. El nombre del documento ODT será el único argumento a convertir.
- 3.
4. - A continuación, crea un script de Python [build] que utilizará el archivo [mkdocs.yml] creado por [convert] para generar un sitio HTML estático

5. - al final, el script [build] mostrará la página de inicio del sitio en un navegador.
- 6.
7. - el sitio tendrá dos columnas independientes. La de la izquierda contendrá la tabla de contenidos, la de la derecha el contenido del documento ODT.
8. - El índice incluirá todos los títulos del documento ODT (Título 1, Título 2, Título 3, etc.), su numeración y su jerarquía
- 9.
10. - Las imágenes suelen estar en tablas y se han redimensionado.
11. Explorarás las tablas en busca de estas imágenes y respetarás su redimensionamiento.
12. - Hay bloques de código en el documento ODT. Se pueden identificar por sus estilos [Código fuente numerado resultados, Código fuente]
13. Los demás atributos del párrafo de código deben ignorarse. Por ejemplo, si el párrafo de código es también una lista numerada, debes ignorar este atributo.
14. Todos estos bloques de código deben tratarse como código en el sitio MkDocs. Las líneas de código deben estar numeradas en el resultado HTML.
- 15.
16. - Para todos los bloques de código, utilizarás el resaltado sintáctico de Python.
- 17.
18. - No debes modificar las líneas de código. En particular, no debes eliminar las sangrías ni los espacios que preceden al primer carácter de la línea de código.
19. - Debes localizar todas las listas con viñetas. ODT tiene varias formas de crearlas. Debes explorarlas todas. Las listas con viñetas son
20. a veces anidadas unas dentro de otras. Debes respetar esta anidación.
- 21.
22. - El título del sitio será «Generar un script de Python con herramientas de IA»
23. - Debes registrar cada imagen extraída del documento ODT, así como cada capítulo MkDocs creado
- 24.
25. - Debes respetar los enlaces que encuentres en el documento ODT. Deben seguir siendo enlaces en el documento HTML generado.
- 26.
27. - Debes respetar los textos en negrita, subrayados o en cursiva. Deben reproducirse tal cual en el documento HTML
- 28.

- línea 19: la prueba inicial se realizó en un curso publicado anteriormente que contenía Python ;
- que iniciaba el chat. Tras unas decenas de iteraciones, quedó obsoleto;

5. Crear un entorno de trabajo Python

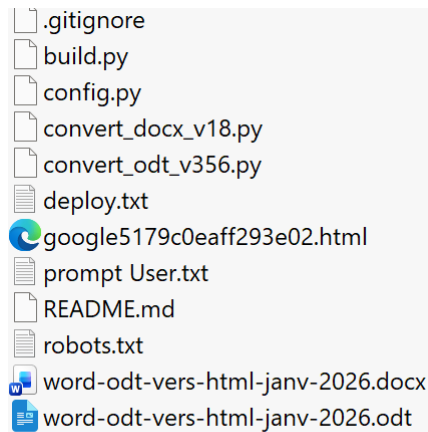
Gemini y ChatGPT tienen generar dos scripts Python. Así que necesitamos un script Python :

- La instalación de Python se presenta en [apartado 3.1](#) de [\[ref1\]](#) ;
- La instalación PyCharm se presenta al [apartado 3.2](#) de [\[ref1\]](#). [\[ref1\]](#) presenta una larga configuración de PyCharm. Ici, PyCharm sólo se utilizarán para ejecutar scripts Python generados por Gemini y ChatGPT. Por lo tanto, no es necesario configurar PyCharm. PyCharm también se utilizará para presentar los archivos requeridos por el convertidor;

6. El expediente de trabajo del convertidor

Puede descargar el archivo de trabajo | [AQUÍ](#) |.

El expediente de trabajo es siguiente:



.gitignore
build.py
config.py
convert_docx_v18.py
convert_odt_v356.py
deploy.txt
google5179c0eaff293e02.html
prompt User.txt
README.md
robots.txt
word-odt-vers-html-janv-2026.docx
word-odt-vers-html-janv-2026.odt

- Ignoraremos por el momento los siguientes archivos [.gitignore, deploy.txt, google*.html, README.md, robots.txt, deploy.txt]. Estos archivos se utilizarán para desplegar el sitio HTML generado localmente en GitHub;
- [prompt User.txt] es el prompt con el que empecé a iterar con Gemini ;
- El archivo [word-odt-vers-html-janv-2026.odt] es el documento ODT de este artículo. Se convierte en HTML mediante script [convert_odt_v356.py] ;
- El fichero [word-odt-vers-html-janv-2026.docx] es el documento DOCX de este artículo. Se convierte en HTML mediante script [convert_docx_v18.py] ;
- Guiones [convert*.py] y [build.py] son los dos scripts Python generados por el IA ;
 - [convert*] convierte un documento ODT o DOCX in situ [MkDocs]. Un sitio MkDocs es un sitio estático cuyas páginas se escriben utilizando la sintaxis [Markdown]. [MkDocs] proporciona un servidor capaz de mostrar sitios MkDocs;
 - [build] convierte el sitio MkDocs en un sitio estático estándar HTML. Al final de esta conversión, abre un navegador para mostrar la raíz del sitio;

En ningún momento miraremos el código Python generado. Consideraremos los dos scripts como dos cajas negras. En ningún momento modifiqué el código a mano. Siempre pregunté a Gemini / ChatGPT para corregir su script por sí mismo. Por eso no necesitas saber Python para usar el conversor.

- [config.py] es un archivo de configuración para scripts [convert*]. Inicialmente [convert*] no tenía un archivo de configuración. Luego, a medida que cambiaba los documentos a convertir, me fui dando cuenta de que había elementos que me pedían el IA para generar (por ejemplo el nombre del sitio que cambia para cada documento) que sería mejor colocar en un archivo de configuración que el propio usuario pudiera modificar. También lo construí iterativamente con el IA ;

7. Los archivos de configuración de los convertidores

- 1  config.py
- 2  config_code.py
- 3  config_colors.py
- 4  config_presentation.py
- 5  config_styles.py
- 6  config_texts.py
- 7  config_urls.py

El archivo de configuración [config.py] reúne información que puede cambiar de un documento a otro. En utiliza otros 6 archivos de configuración. Éstos contienen los parámetros que es más probable que cambien al pasar de un documento a otro para su conversión.

El fichero [config.py] importa el contenido de los otros 6 ficheros de configuración:

```
1. from config_urls import URLS
2. from config_texts import TEXTS
3. from config_code import CODE
4. from config_styles import STYLES
5. from config_colors import COLORS
6. from config_presentation import PRESENTATION
```

config_urls

Este archivo contiene los tres URL de la configuración [config.py]. Estos cambian con cada documento.

```
1. URLS = {
2.     "site_url": "https://stahe.github.io/word-odt-vers-html-janv-2026/",
3.     "repo_url": "https://stahe.github.io/word-odt-vers-html-janv-2026",
4.     "author_site": "https://tahe.developpez.com",
5. }
```

- línea 2: el URL del sitio web donde va a desplegar el sitio HTML producido por el conversor;
- línea 3: el URL de su repositorio Git (véase el párrafo 12) ;
- línea 4: el sitio web del creador del sitio web. Este URL se coloca en la parte inferior de las páginas del sitio web. Puede estar vacío.

config_texts

Este archivo contiene los textos [config.py] que pueden cambiar al cambiar el idioma del sitio HTML.

```
1. TEXTS = {
2.     "home_label": "Bienvenido",
3.     "site_name": "Convertir un documento Word u ODT en un sitio web HTML estático compatible con MkDocs utilizando las IA Gemini 3 y ChatGPT 5.2",
4.     "site_description": "Convertir un documento Word u ODT a un sitio HTML estático compatible con MkDocs utilizando las IA Gemini 3 y ChatGPT 5.2",
5.     "toggle_to_dark": "Cambiar a modo oscuro",
6.     "toggle_to_light": "Cambiar a modo claro",
7.     "footer_license_sentence": (
```

```

8.      "Este curso-tutorial escrito por <strong>Serge Tahé</strong> se pone a disposición del público
según los términos de la\n"
9.      "      <em>Licencia Creative Commons Reconocimiento - Sin uso comercial -\n"
10.     "      Compartir en las mismas condiciones 3.0 no adaptada</em>.\n"
11.     ),
12.     "copy_label": "Copiar",
13.     "copy_copied_label": "Copiado",
14.     }

```

- línea 2: el nombre del primer enlace del índice. Su contenido incluye todo lo que hay en el documento antes del primer título de nivel 1 (Título 1). Es el principio del documento;
- línea 3: el título del documento. Se muestra en el banner superior del sitio web generado ;
- línea 4: también el título del documento, esta vez para los motores de búsqueda;
- líneas 5-6: las burbujas que aparecen al pasar el ratón por encima del icono de la barra superior de la web para pasar del modo claro al oscuro o viceversa;
- líneas 7-11: la licencia del sitio. Se muestra en la página pied del sitio generado;
- líneas 12-13: el código mostrado del sitio, junto con un botón que permite copiar el código en el portapapeles. La línea 12 es la etiqueta del botón antes de copiar, la línea 13 la etiqueta cuando el código ha sido copiado;

Este es el aspecto del archivo para la versión inglesa del documento:

```

1.      TEXTS = {
2.          "home_label": "Welcome",
3.          "site_name": "Convert a Word or ODT document into a static HTML website using Gemini 3 and
ChatGPT 5.2",
4.          "site_description": "Convert a Word or ODT document into a static HTML website using Gemini
3 and ChatGPT 5.2 AI",
5.          "toggle_to_dark": "Switch to dark mode",
6.          "toggle_to_light": "Switch to light mode",
7.          "footer_license_sentence": (
8.              'This tutorial written by <strong>Serge Tahé</strong> is made available to the public
under the terms of the\n'
9.              '<em>Creative Commons Attribution - Non-Commercial -\n'
10.             'ShareAlike 3.0 Unported</em>.'
11.          ),
12.          "copy_label": "Copy",
13.          "copy_copied_label": "Copied",
14.      }

```

config_styles

Este archivo se utiliza para nombrar el estilo del título del documento. Para un documento DOCX, generalmente es "Título", pero para un documento ODT, cambia con cada documento.

```

1.      STYLES = {
2.          "style_names": [
3.              "P1"
4.          ]
5.      }

```

- línea 2: [styles_names] es una tabla. Puedes poner los nombres de los estilos que puede tener el título del documento;
- línea 3: el documento [word-odt-vers-html-janv-2026.odt] tiene un título con estilo P1 ;

config_code

Los bloques de código del documento DOCX o ODT pueden encontrarse de dos formas:

- código enriquecido (colores, negrita, cursiva, etc.). En este caso, se copia tal cual en el sitio HTML;
- código plano (plain text). En este caso, damos instrucciones a los conversores para que ellos mismos encuentren el lenguaje utilizado en el bloque de código. Para ello, les damos las palabras clave :

```

1. CODE = {
2.     # Automatic language detection rules based on content
3.     "detection_rules": {
4.         "csharp": [
5.             "using", "Console.WriteLine", "public static void Main", "WebMethod",
6.             "TryParse", "EventArgs", "String.Format", "System.Web.Services"
7.         ],
8.         "java": [
9.             "System.out.println", "public static void main(String", "package",
10.            "JUnitTest", "Class.forName", "PreparedStatement", "private static void",
11.            "private void", "getAgendaMedecinJour", "@PostConstruct", "@ResponseBody",
12.            "@RequestMapping", "getMedecin", "@Entity", "@Autowired", "@Bean",
13.            "Serializable", "getClient", "getCreneau", "getRv", "PostAjouterRv",
14.            "PostSupprimerRv", "@EnableJpaRepositories", "@Component", "getAgendaMedecinJour",
15.            "getResponse", "getMessagesForException", "getBase64", "ajouterRv",
16.            "Reponse", "getPartialViewAgenda", "setModelForAgenda", "ActionContext",
17.            "getActionContext", "PostLang", "PostUser", "PostGetAgenda", "@NotNull",
18.            "@EnableAutoConfiguration", "HttpSecurity"
19.        ],
20.        "html": [
21.            "<html>", "</div>", "<body>", "<script>", "href=", "<span>", "<p>",
22.            "<h2>", "<form", "<table", "<input"
23.        ],
24.        "sql": [
25.            "SELECT", "INSERT INTO", "UPDATE", "DELETE FROM", "WHERE",
26.            "CREATE TABLE", "ALTER TABLE"
27.        ],
28.        "python": [
29.            "def", "import", "print(", "from"
30.        ],
31.        "xml": [
32.            "<?xml", "<project", "<version>", "<configuration>", "<build>",
33.            "<dependency>", "<properties>", "<configuration>", "<start-class>"
34.        ],
35.        "javascript": [
36.            "use strict", "console.log", "let", "constructor", "async", "export"
37.        ],
38.        "php": [
39.            "<?php", "declare", "require"
40.        ],
41.        "vbscript": [
42.            "Option", "Dim", "Explicit"
43.        ],
44.        "markdown": [
45.            "# ", "## ", "### ", "****", "___", "![", "]( " # Typical Markdown markers
46.        ],
47.    },
48. }

```

Una vez más, estas reglas de detección sólo se aplican a bloques de código en "plain text" y no a bloques de código enriquecido. Si descubre en su sitio que hay bloques de código que no han sido coloreados, ahí es donde tiene que intervenir.

Para cada lengua indicamos cadenas de caracteres utilizadas para identificar idioma. Determinadas palabras clave pueden encontrarse en distintos idiomas. El conversor selecciona la lengua para la que ha encontrado más palabras clave. El principio es sencillo: usted mira sus códigos, selecciona las palabras clave características y las pone en estas líneas para el idioma correcto. Tenga en cuenta que no puede poner cualquier cosa como nombre de idioma: tiene que utilizar los nombres reconocidos por MkDocs ;

config presentation

Este archivo establece la apariencia de varios elementos del sitio web generado. No es algo que usted necesita cambiar a menudo :

```

1. PRESENTATION = {
2.     # -----
3.     # Document title

```

```

4.      # -----
5.      "document_title": {
6.          "font_size": "28px",
7.          "font_weight": "bold",
8.          "margin_bottom": "1em",
9.          "line_height": "1.2"
10.     },
11.
12.     # -----
13.     # Rich code blocks
14.     # -----
15.     "code": {
16.         "rich_line_height": "12px",
17.         "rich_font_family": "Consolas, 'Courier New', monospace",
18.         "rich_font_size": "15px"
19.     },
20.
21.     # -----
22.     # Copy button
23.     # -----
24.     "copy_button": {
25.         "container": "position: relative;",
26.         "btn": (
27.             "position: absolute; top: .5rem; right: .5rem; "
28.             "display: inline-flex; align-items: center; justify-content: center; "
29.             "gap: .35rem; "
30.             "padding: .25rem .6rem; "
31.             "font-size: .72rem; font-weight: 600; letter-spacing: .01em; "
32.             "line-height: 1.2; "
33.             "border-radius: 999px; "
34.             "box-shadow: 0 1px 2px rgba(0,0,0,.10); "
35.             "backdrop-filter: saturate(180%) blur(6px); "
36.             "cursor: pointer; user-select: none; "
37.             "transition: transform .08s ease, box-shadow .12s ease, background .12s ease; "
38.             "z-index: 5;"
39.         ),
40.         "btn_hover": (
41.             "box-shadow: 0 3px 10px rgba(0,0,0,.18); "
42.             "transform: translateY(-1px);"
43.         ),
44.         "btn_copied": "opacity: .85;"
45.     },
46.
47.     # -----
48.     # Images
49.     # -----
50.     "images": {
51.         "shadow": {
52.             "enabled": True,
53.             "border_radius": "8px"
54.         }
55.     },
56.
57.     # -----
58.     # Frame / header border
59.     # -----
60.     "frame": {
61.         "header_top_border_width": "4px"
62.     }
63. }

```

Si lo desea, puede cambiar el aspecto :

- líneas 5-10: del título del documento ;
- líneas 15-19: líneas de código mejorado ;
- líneas 24-45: el botón [Copiar] para guardar el código en el portapapeles;
- líneas 50-55: sombras de la imagen ;
- líneas 60-62: el marco que bordea la barra superior de las páginas del sitio;

config_colors

Aquí es donde controlas los colores de tu sitio, principalmente el color de fondo del banner superior de las páginas:

```
1.  COLORS = {
2.      "theme": {
3.          "palette": {
4.              "light": {
5.                  "media": "(prefers-color-scheme: light)",
6.                  "scheme": "default",
7.                  "primary": "teal",
8.                  "accent": "purple"
9.              },
10.             "dark": {
11.                 "media": "(prefers-color-scheme: dark)",
12.                 "scheme": "slate",
13.                 "primary": "teal",
14.                 "accent": "purple"
15.             }
16.         }
17.     },
18.
19.     "frame": {
20.         # "header_bg_light": "#1E88E5", (azul)
21.         "header_bg_light": "#0B3D91",
22.         "header_bg_dark": "#0B3D91",
23.         "header_top_border_light": "",
24.         "header_top_border_dark": ""
25.     },
26.
27.     "document_title": {
28.         "color": "#2c3e50"
29.     },
30.
31.     "copy_button": {
32.         "border": "rgba(0,150,136,.45)",
33.         "background": "rgba(0,150,136,.12)",
34.         "text": "rgb(0,150,136)",
35.         "background_hover": "rgba(0,150,136,.20)"
36.     },
37.
38.     "images": {
39.         "shadow": {
40.             "zoomable": "0 8px 24px rgba(0,0,0,.28), 0 18px 60px rgba(0,0,0,.20)",
41.             "zoomable_hover": "0 12px 30px rgba(0,0,0,.32), 0 26px 80px rgba(0,0,0,.22)",
42.             "lightbox": "0 24px 80px rgba(0,0,0,.65)"
43.         }
44.     }
45. }
```

Hay que saber un poco de CSS, que no es mi caso. Sólo he utilizado los colores en las líneas 7 y 13, que establecen el color del banner superior.

config_files_to_copy

Este archivo enumera los archivos que deben copiarse en la raíz del sitio web generado :

```
1.  FILES_TO_COPY =[
2.      "google5179c0eaff293e02.html",
3.      "robots.txt",
4.      "es-word-odt-vers-html-janv-2026.pdf",
5.      "es-word-odt-vers-html-janv-2026.zip"
6.  ]
```

- líneas 2-3: son necesarias para el seguimiento de Google del sitio generado (véase el párrafo 13) ;
- líneas 4-5: los archivos que desea copiar en la raíz de su sitio web para ponerlos a disposición de sus

visitantes;

config

El archivo [config.py] es el archivo de configuración utilizado por los dos convertidores. Reúne toda la información ellos necesidad. Se trata de un fichero que no debe modificarse nunca. Es complejo, por lo que hemos decidido extraer los parámetros de configuración susceptibles de cambio y colocarlos en ficheros externos.

```
1.  from config_urls import URLS
2.  from config_texts import TEXTS
3.  from config_code import CODE
4.  from config_styles import STYLES
5.  from config_colors import COLORS
6.  from config_presentation import PRESENTATION
7.  from config_files_to_copy import FILES_TO_COPY
8.
9.  config = {
10.     "toc": {
11.         "home_label": TEXTS["home_label"]
12.     },
13.
14.     "mkdocs": {
15.         "site_name": TEXTS["site_name"],
16.         "site_url": URLS["site_url"],
17.         "site_description": TEXTS["site_description"],
18.         "site_author": "Serge Tahé",
19.         "repo_url": URLS["repo_url"],
20.         "repo_name": "GitHub",
21.         "use_directory_urls": False,
22.
23.         "theme": {
24.             "name": "material",
25.             "custom_dir": "overrides",
26.             "features": [
27.                 "navigation.sections",
28.                 "navigation.indexes",
29.                 "navigation.expand",
30.                 "toc.integrate",
31.                 "navigation.top"
32.             ],
33.             "palette": [
34.                 {
35.                     "media": COLORS["theme"]["palette"]["light"]["media"],
36.                     "scheme": COLORS["theme"]["palette"]["light"]["scheme"],
37.                     "primary": COLORS["theme"]["palette"]["light"]["primary"],
38.                     "accent": COLORS["theme"]["palette"]["light"]["accent"],
39.                     "toggle": {
40.                         "icon": "material/brightness-7",
41.                         "name": TEXTS["toggle_to_dark"]
42.                     }
43.                 },
44.                 {
45.                     "media": COLORS["theme"]["palette"]["dark"]["media"],
46.                     "scheme": COLORS["theme"]["palette"]["dark"]["scheme"],
47.                     "primary": COLORS["theme"]["palette"]["dark"]["primary"],
48.                     "accent": COLORS["theme"]["palette"]["dark"]["accent"],
49.                     "toggle": {
50.                         "icon": "material/brightness-4",
51.                         "name": TEXTS["toggle_to_light"]
52.                     }
53.                 }
54.             ]
55.         },
56.
57.         "markdown_extensions": [
58.             "admonition",
59.             "attr_list",
60.             "pymdownx.superfences",
```

```

61.         "pymdownx.mark",
62.         {
63.             "pymdownx.highlight": {
64.                 "anchor_linenums": True,
65.                 "linenums": None
66.             }
67.         },
68.         "md_in_html",
69.         "footnotes"
70.     ],
71.
72.     "extra_javascript": [
73.         "javascripts/focus.js"
74.     ],
75.     "extra_css": [
76.         "stylesheets/focus.css"
77.     ]
78. },
79.
80. "footer": (
81.     "{% block footer %}\n"
82.     " <div class=\"md-footer-meta md-typeset\">\n"
83.     " <div class=\"md-footer-meta__inner\">\n\n"
84.     " <div>\n"
85.     f" <a href=\"{URLS['author_site']}\" target=\"_blank\">\n"
86.     f" {URLS['author_site']}\n"
87.     " </a>\n"
88.     " <br>\n"
89.     f" {TEXTS['footer_license_sentence']}\n"
90.     " </div>\n\n"
91.     " </div>\n"
92.     " </div>\n"
93.     "{% endblock %}"
94. ),
95.
96. "extra": {
97.     "analytics": {
98.         "provider": "google",
99.         "property": "G-XXXXXXX"
100.    }
101. },
102.
103. "document_title": {
104.     "style_names": STYLES["style_names"],
105.     "css": (
106.         f"font-size: {PRESENTATION['document_title']['font_size']}; "
107.         f"font-weight: {PRESENTATION['document_title']['font_weight']}; "
108.         f"margin-bottom: {PRESENTATION['document_title']['margin_bottom']}; "
109.         f"line-height: {PRESENTATION['document_title']['line_height']}; "
110.         f"color: {COLORS['document_title']['color']}; "
111.     )
112. },
113.
114. "code": {
115.     "case_insensitive_languages": ["sql", "html", "vbnet", "vbscript"],
116.     "style_keywords": ["code"],
117.     "default_language": "text",
118.     "rich_line_height": PRESENTATION["code"]["rich_line_height"],
119.     "rich_font_family": PRESENTATION["code"]["rich_font_family"],
120.     "rich_font_size": PRESENTATION["code"]["rich_font_size"],
121.     "detection_rules": CODE["detection_rules"],
122.     "copy_button": True,
123.     "copy_label": TEXTS["copy_label"],
124.     "copy_copied_label": TEXTS["copy_copied_label"],
125.     "copy_only_recognized_language": True,
126.     "copy_min_lines": 4,
127.     "copy_allow_pygments_heuristic": True,
128.     "copy_style": {
129.         "container": PRESENTATION["copy_button"]["container"],
130.         "btn": (

```

```

131.         PRESENTATION["copy_button"]["btn"]
132.         + f"border:1px solid {COLORS['copy_button']['border']}}; "
133.         + f"background:{COLORS['copy_button']['background']}}; "
134.         + f"color:{COLORS['copy_button']['text']}}; "
135.     ),
136.     "btn_hover": (
137.         f"background:{COLORS['copy_button']['background_hover']}}; "
138.         + PRESENTATION["copy_button"]["btn_hover"]
139.     ),
140.     "btn_copied": PRESENTATION["copy_button"]["btn_copied"]
141. },
142. },
143.
144.     "images": {
145.         "shadow": {
146.             "enabled": PRESENTATION["images"]["shadow"]["enabled"],
147.             "border_radius": PRESENTATION["images"]["shadow"]["border_radius"],
148.             "zoomable": COLORS["images"]["shadow"]["zoomable"],
149.             "zoomable_hover": COLORS["images"]["shadow"]["zoomable_hover"],
150.             "lightbox": COLORS["images"]["shadow"]["lightbox"],
151.         }
152.     },
153.
154.     "frame": {
155.         "header_bg_light": COLORS["frame"]["header_bg_light"],
156.         "header_bg_dark": COLORS["frame"]["header_bg_dark"],
157.         "header_top_border_light": COLORS["frame"]["header_top_border_light"],
158.         "header_top_border_dark": COLORS["frame"]["header_top_border_dark"],
159.         "header_top_border_width": PRESENTATION["frame"]["header_top_border_width"]
160.     },
161.
162.     "files_to_copy": FILES_TO_COPY,
163.
164.     "debug": True
165. }

```

- líneas 1-7: importar todos los archivos de configuración ;
- línea 9 : el fichero de configuración es un script de Python. Define una única variable [config]. Se trata de un diccionario que contiene todos los valores de configuración;
- línea 11: la redacción del enlace "Inicio" ;
- línea 14 el diccionario [mkdocs] configurará el archivo [mkdocs.yml] generado por el Gemini / convertidor ChatGPT. Este archivo es utilizado por script [build] para construir un sitio HTML a partir del sitio MkDocs generado por el conversor ;
- líneas 15-20 configurar el sitio GitHub, que albergará el sitio HTML creado a partir del documento ODT / DOCX (véase el apartado 12)
- línea 21 esta línea es importante. Si falta, en lugar de mostrar una página HTML, el navegador abrirá la carpeta de esta página;
- línea 24 : MkDocs propone varios temas para un sitio MkDocs / HTML. Ici hemos elegido el tema "material", línea 24 ;
- líneas 27-31 configuración de la navegación del sitio. Para ello se utiliza un índice situado en la columna izquierda de la página mostrada (línea 30) ;
- líneas 33-54 definen dos paletas de colores: modo luz (líneas 35-42) o modo oscuro (líneas 45-52). Un icono en la barra superior de las páginas mostradas permite pasar de una a otra;
- líneas 57-70 extensiones del lenguaje Markdown utilizadas por MkDocs. Estas extensiones fueron generadas por Gemini en respuesta a algunas de mis peticiones;
- línea 73 el script [focus.js] es un script Javascript generado por Gemini. Está asociado al botón de la barra superior del sitio para mostrar u ocultar el índice;
- línea 76 el CSS utilizado por este botón ;
- líneas 80-95 la definición de la página pied en el sitio HTML. Fue generada por Gemini a partir de un ejemplo de texto que le di;
- líneas 97-100 definición de la etiqueta de Google Analytics (GA) que se utilizará para realizar un seguimiento de las visitas al sitio;
- línea 99 pon tu marcador GA ahí;

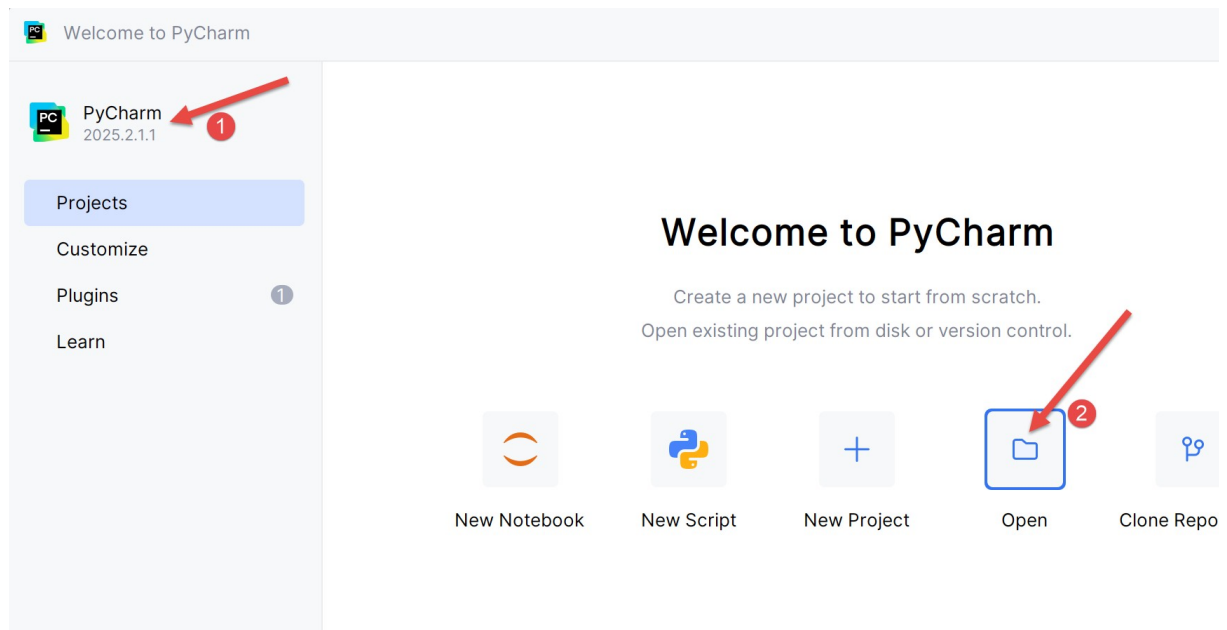
- líneas 103-112 el estilo del título del documento, el presente en el documento ODT /DOCX antes del primer título de nivel 1. El estilo mostrado línea 104 cambia con cada ODT Documento. Sin embargo, puede ser constante ("Título") para los documentos DOCX. Por defecto, el conversor registra los estilos de todos los párrafos del documento ODT / DOCX que precede al primer título de nivel 1. El primer paso es localizar el estilo del párrafo que desea utilizar como título, a continuación, establezca en el archivo [config_styles];;
- líneas 105-111 el estilo CSS que desea dar al título del documento. Este título se encuentra en la página "Inicio", la primera página que aparece al abrir el sitio;
- línea 115: aquí es donde se ponen los idiomas que no distinguen entre mayúsculas y minúsculas;
- línea 116 los bloques de código se identifican por sus estilos en el documento ODT / DOCX. Puede haber más de una. Línea 116 en la sección "Código", ponemos todas las palabras clave que se pueden utilizar para detectar un estilo de código. En mis documentos, todos mis estilos de código tienen la palabra "código" en su nombre. Y ningún otro estilo tiene esta palabra en su nombre. Así que puedo usar una sola palabra clave. Si hay varias, sepáralas con comas 116 ;
- línea 117 establece el idioma por defecto. En el caso de un bloque de código "plain text", si no se detecta ningún idioma, el idioma por defecto será "text". En el caso de MkDocs, se trata de un bloque de código sin resaltado de sintaxis. Este es el caso, por ejemplo, de ;
- línea 121 plain text": esta configuración se utiliza para los códigos "plain text" que inicialmente no tienen resaltado de sintaxis. El propósito de estas líneas es en dar un en función del lenguaje utilizado en el bloque de código. **Ten en cuenta que si todo tu código está enriquecido porque proviene de un IDE como Eclipse, Visual Studio, etc. y no tienes un bloque de código "plain text" asociado a un idioma, entonces no necesitas poner nada en esta parte de la configuración. Puede dejar vacías las tablas asociadas a los idiomas.** Todo esto tiene lugar en el archivo de configuración del código [config_code];
- líneas 118-120 bloques de código enriquecidos (negrita, cursiva, subrayado, resaltado, color de los caracteres): estas líneas se refieren a bloques de código enriquecidos (negrita, cursiva, subrayado, resaltado, color de los caracteres). Estos bloques no se tratan de la misma manera que los bloques de código "normal text". Se representan de forma idéntica en HTML ;
 - línea 118 establece la altura de línea de las líneas de código ;
 - línea 119 establece la fuente CSS del bloque de código ;
 - línea 120 establece el tamaño de la fuente ;
- líneas 144-152: establecer la apariencia de las imágenes ;
- líneas 154-160: establece la apariencia del marco en el que se muestra la página web;
- línea 162 la lista de archivos que se copiarán en la raíz del sitio HTML;
- línea 164 debug] en True autoriza los registros de estilo de los párrafos que preceden al primer título de nivel 1 en el Documento ODT / DOCX. En estos registros encontrará el estilo exacto del párrafo que se utilizará como título en la página de inicio del sitio;

Al final, cuando se pasa de un Documento ODT / DOCX al otro, no cambiaremos nada en el archivo [config.py]. Sólo modificaremos los otros archivos de configuración.

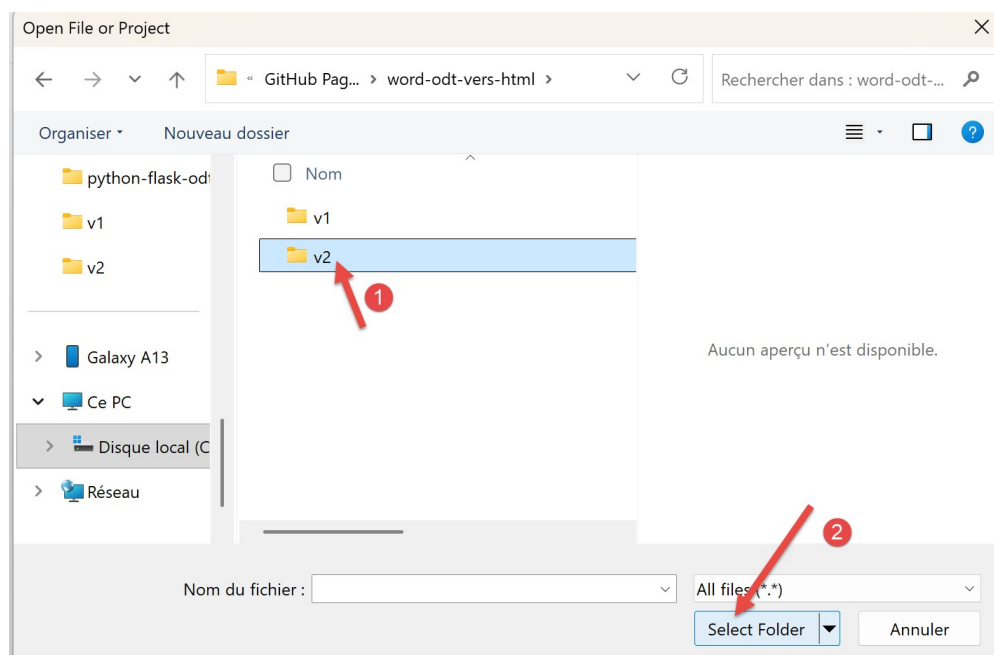
8. Cómo usar el conversor de ODT a HTML

El archivo de trabajo se encuentra en [| AQUÍ |](#).

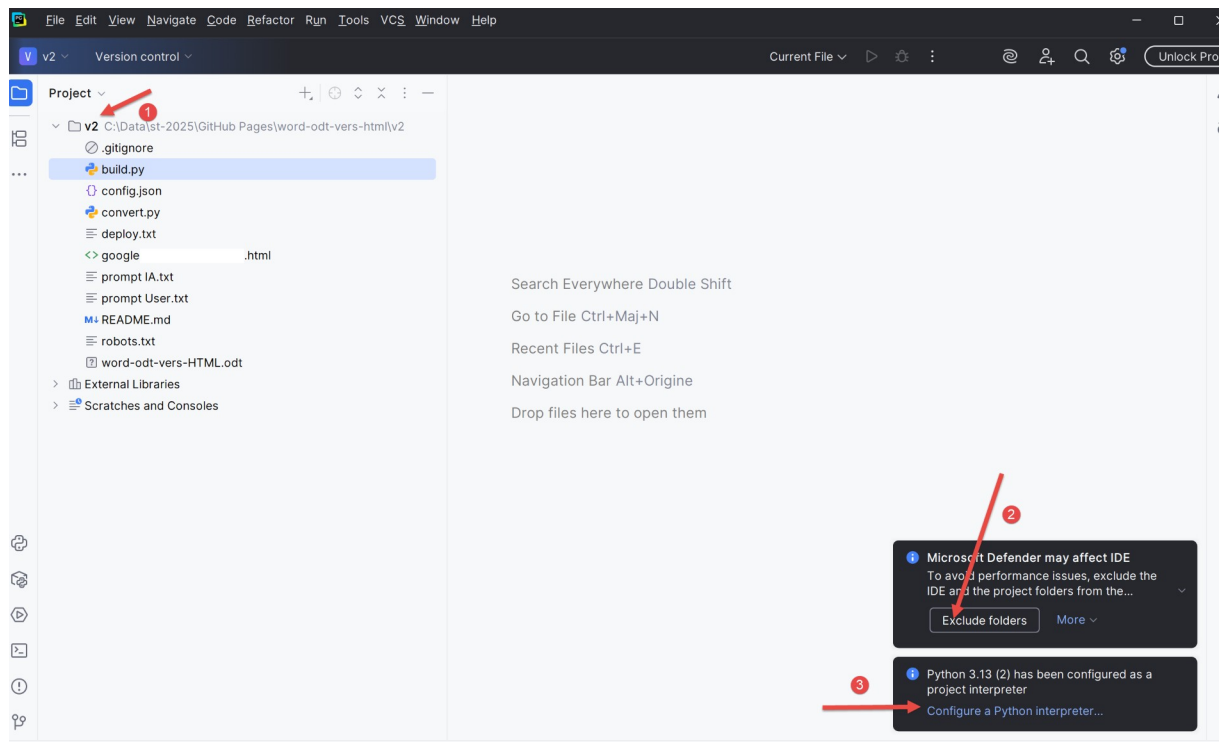
Vamos a ejecutar PyCharm :



- En [2], abra un nuevo proyecto ;

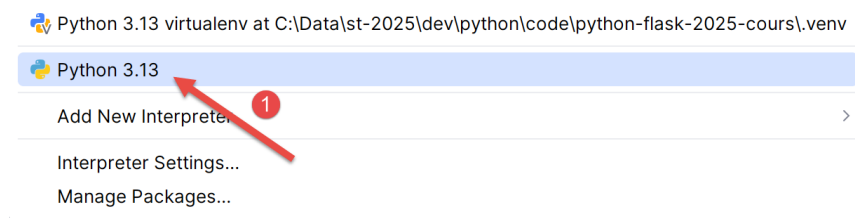


- En [1], seleccione la carpeta donde guardó la carpeta de trabajo;
- En [2], confirman esta elección;



- En [1], su Python ;
- En [2], validar para no ser molestado por el sistema de seguridad de Windows;
- En [3], elija un intérprete de Python ;

Python Interpreter



- En [1], elija el intérprete de Python que instaló en el párrafo 5 ;

El Gemini / ChatGPT utiliza dependencias de Python que debes instalar. Preguntaremos a Gemini cuáles:

rappelle moi toutes les dépendances Python nécessaires à ton convertisseur [convert]

La respuesta de Gemini es la siguiente:

Commande d'installation rapide

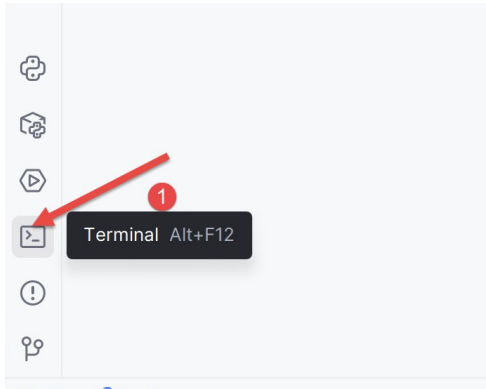
Vous pouvez installer tout le nécessaire en une seule commande dans votre terminal :

Bash



```
pip install odfpy unicode mkdocs mkdocs-material
```

Vamos a instalar estos :



- En [1], abre un terminal;



- En [2], escriba el comando dado por Gemini ;

La respuesta es la siguiente:

```
1. PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> pip install odfpy unicode mkdocs
mkdocs-material
2. Defaulting to user installation because normal site-packages is not writeable
3. Requirement already satisfied: odfpy in c:\users\serge\appdata\roaming\python\python313\site-
packages (1.4.1)
4. Requirement already satisfied: unicode in c:
\users\serge\appdata\roaming\python\python313\site-packages (1.4.0)
5. Requirement already satisfied: mkdocs in c:\users\serge\appdata\roaming\python\python313\site-
packages (1.6.1)
6. Requirement already satisfied: mkdocs-material in c:
\users\serge\appdata\roaming\python\python313\site-packages (9.7.0)
7. Requirement already satisfied: defusedxml in c:
\users\serge\appdata\roaming\python\python313\site-packages (from odfpy) (0.7.1)
```

```

8. Requirement already satisfied: click>=7.0 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (8.3.1)
9. Requirement already satisfied: colorama>=0.4 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (0.4.6)
10. Requirement already satisfied: ghp-import>=1.0 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (2.1.0)
11. Requirement already satisfied: jinja2>=2.11.1 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (3.1.6)
12. Requirement already satisfied: markdown>=3.3.6 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (3.10)
13. Requirement already satisfied: markupsafe>=2.0.1 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (3.0.2)
14. Requirement already satisfied: mergedeep>=1.3.4 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (1.3.4)
15. Requirement already satisfied: mkdocs-get-deps>=0.2.0 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (0.2.0)
16. Requirement already satisfied: packaging>=20.5 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (25.0)
17. Requirement already satisfied: pathspec>=0.11.1 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (0.12.1)
18. Requirement already satisfied: pyyaml-env-tag>=0.1 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (1.1)
19. Requirement already satisfied: pyyaml>=5.1 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (6.0.2)
20. Requirement already satisfied: watchdog>=2.0 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs) (6.0.0)
21. Requirement already satisfied: babel>=2.10 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs-material) (2.17.0)
22. Requirement already satisfied: backrefs>=5.7.post1 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs-material) (6.1)
23. Requirement already satisfied: mkdocs-material-extensions>=1.3 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs-material) (1.3.1)
24. Requirement already satisfied: paginate>=0.5 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs-material) (0.5.7)
25. Requirement already satisfied: pygments>=2.16 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs-material) (2.19.2)
26. Requirement already satisfied: pymdown-extensions>=10.2 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs-material) (10.17.2)
27. Requirement already satisfied: requests>=2.26 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs-material) (2.32.5)
28. Requirement already satisfied: python-dateutil>=2.8.1 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from ghp-import>=1.0->mkdocs)
(2.9.0.post0)
29. Requirement already satisfied: platformdirs>=2.2.0 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from mkdocs-get-deps>=0.2.0->mkdocs)
(4.5.0)
30. Requirement already satisfied: six>=1.5 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from python-dateutil>=2.8.1->ghp-
import>=1.0->mkdocs) (1.17.0)
31. Requirement already satisfied: charset_normalizer<4,>=2 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from requests>=2.26->mkdocs-material)
(3.4.3)
32. Requirement already satisfied: idna<4,>=2.5 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from requests>=2.26->mkdocs-material)
(3.10)
33. Requirement already satisfied: urllib3<3,>=1.21.1 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from requests>=2.26->mkdocs-material)
(2.5.0)
34. Requirement already satisfied: certifi>=2017.4.17 in c:
\users\serge\appdata\roaming\python\python313\site-packages (from requests>=2.26->mkdocs-material)
(2025.8.3)
35. PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2>

```

En mi PC, todo estaba ya instalado. Si este no es su caso, se instalarán todas las dependencias solicitadas.

Ahora podemos utilizar el conversor. En el terminal abierto, escriba el siguiente comando:


```

1. PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> python .\convert_odt_v356.py .\word-odt-
vers-html-janv-2026.odt .\config.py
2. --- ODT to MkDocs Converter V356 ---
3. Traitement de .\word-odt-vers-html-janv-2026.odt...
4. Copié : google5179c0eaff293e02.html
5. Copié : robots.txt
6. Copié : word-odt-vers-html-janv-2026.pdf
7. Copié : word-odt-vers-html-janv-2026.zip
8. Scan des renvois...
9. Génération Markdown...
10. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
11. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
12. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
13. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
14. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
15. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
16. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
17. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
18. [DEBUG PRE-H1] Tag=text:p Style='Standard' Text='...'
19. [DEBUG PRE-H1] Tag=text:p Style='Standard (WW)' Text='...'
20. [DEBUG PRE-H1] Tag=text:p Style='Standard (WW)' Text='...'
21. [DEBUG PRE-H1] Tag=text:p Style='P1' Text='Convertir un document Word ou ODT vers un site
sta...'
22. >>> TITRE DOCUMENT TROUVÉ : Convertir un document Word ou ODT vers un site statique HTML
compatible MkDocs avec les IA Gemini 3 et ChatGPT 5.2
23. [DEBUG PRE-H1] Tag=text:p Style='P2' Text='...'
24. [DEBUG PRE-H1] Tag=text:p Style='P2' Text='...'
25. [DEBUG PRE-H1] Tag=text:p Style='P2' Text='...'
26. [DEBUG PRE-H1] Tag=text:p Style='P3' Text='Serge Tahé, janvier 2026...'
27. [DEBUG PRE-H1] Tag=text:p Style='P2' Text='...'
28. [DEBUG PRE-H1] Tag=text:p Style='P2' Text='...'
29. [DEBUG PRE-H1] Tag=text:p Style='P2' Text='...'
30. [DEBUG PRE-H1] Tag=text:p Style='P4' Text='...'
31. [DEBUG PRE-H1] Tag=text:p Style='P4' Text='...'
32. [DEBUG PRE-H1] Tag=text:p Style='P4' Text='...'
33. [DEBUG PRE-H1] Tag=text:p Style='P4' Text='Ce site a été créé avec le convertisseur [Word
ou ...'
34. [DEBUG PRE-H1] Tag=text:h Style='P5' Text='Introduction...'
35. >>> CHAPITRE 1: 1 Introduction
36. >>> CHAPITRE 1: 2 Les exemples de ce document
37. >>> CHAPITRE 2: 2.1 Les listes
38. >>> CHAPITRE 3: 2.1.1 Listes à puces
39. >>> CHAPITRE 3: 2.1.2 Listes numérotées
40. >>> CHAPITRE 3: 2.1.3 Listes mixtes 1
41. >>> CHAPITRE 3: 2.1.4 Listes mixtes 2
42. >>> CHAPITRE 2: 2.2 Les blocs de code
43. >>> CHAPITRE 3: 2.2.1 Blocs de code enrichi (Eclipse, Visual Studio, ...)
44. >>> CHAPITRE 3: 2.2.2 Blocs de code brut (plain text)
45. >>> CHAPITRE 2: 2.3 Les liens
46. >>> CHAPITRE 2: 2.4 L'enrichissement de texte
47. >>> CHAPITRE 2: 2.5 Un titre peut être également enrichi.
48. >>> CHAPITRE 2: 2.6 Les images
49. >>> CHAPITRE 2: 2.7 Les caractères à protéger
50. >>> CHAPITRE 2: 2.8 Les tableaux
51. >>> CHAPITRE 2: 2.9 Les notes de bas de page
52. >>> CHAPITRE 1: 3 Ce qui existe sur internet
53. >>> CHAPITRE 1: 4 Le prompt initial à Gemini 3
54. >>> CHAPITRE 1: 5 Créer un environnement de travail Python
55. >>> CHAPITRE 1: 6 Le dossier de travail du convertisseur
56. >>> CHAPITRE 1: 7 Le fichier de configuration du convertisseur
57. >>> CHAPITRE 1: 8 Utilisation du convertisseur ODT → HTML
58. >>> CHAPITRE 1: 9 Utilisation du convertisseur DOCX → HTML
59. >>> CHAPITRE 1: 10 Construction du site HTML statique
60. >>> CHAPITRE 1: 11 Examen du site HTML généré
61. >>> CHAPITRE 2: 11.1 La barre supérieure du site
62. >>> CHAPITRE 2: 11.2 Le bas de page du site
63. >>> CHAPITRE 2: 11.3 La page d'accueil
64. >>> CHAPITRE 2: 11.4 Les listes à puces
65. >>> CHAPITRE 2: 11.5 Les listes numérotées
66. >>> CHAPITRE 3: 11.5.1 Listes mixtes 1

```

```

67. >>> CHAPITRE 3: 11.5.2 Listes mixtes 2
68. >>> CHAPITRE 2: 11.6 Les blocs de code enrichis
69. >>> CHAPITRE 3: 11.6.1 Exemple 1
70. >>> CHAPITRE 3: 11.6.2 Exemple 2
71. >>> CHAPITRE 3: 11.6.3 Exemple 3
72. >>> CHAPITRE 2: 11.7 Les blocs de code brut (plain text)
73. >>> CHAPITRE 3: 11.7.1 Exemple 1
74. >>> CHAPITRE 3: 11.7.2 Exemple 2
75. >>> CHAPITRE 3: 11.7.3 Exemple 3
76. >>> CHAPITRE 2: 11.8 Autres blocs de code
77. >>> CHAPITRE 2: 11.9 Les liens
78. >>> CHAPITRE 2: 11.10 L'enrichissement de texte
79. >>> CHAPITRE 2: 11.11 Les images
80. >>> CHAPITRE 2: 11.12 Les caractères protégés
81. >>> CHAPITRE 2: 11.13 Les tableaux
82. >>> CHAPITRE 2: 11.14 Notes de bas de page
83. >>> CHAPITRE 2: 11.15 Anomalies connues
84. >>> CHAPITRE 2: 11.16 Autres cas
85. >>> CHAPITRE 1: 12 Héberger le site HTML sur GitHub
86. >>> CHAPITRE 1: 13 Assurer le suivi du site avec Google Analytics et Google Search Console
87. >>> CHAPITRE 1: 14 Conclusion
88. Terminé.

```

- Línea 1: el comando que convierte el documento ODT en el sitio MkDocs [`python convertir_odt_v356.py . \word-odt-vers-html-janv-2026.odt .\config.py`]. Adapte el número de versión (íci 356) a la versión que ha descargado. El primer parámetro del convertidor es el documento ODT a convertir, el segundo el fichero de configuración del convertidor ;
- 4- líneas7 que el conversor copia en la raíz del sitio MkDocs que crea;
- líneas 10-34 depurar los estilos de los párrafos que preceden al primer título de nivel 1. Estos párrafos constituirán la página de inicio. Uno de los párrafos actúa como título de la página de inicio y, por tanto, del sitio. Este es el párrafo de la línea 21. Observamos su estilo P1. Tenemos que poner este estilo en el archivo de configuración [`config_styles`] :

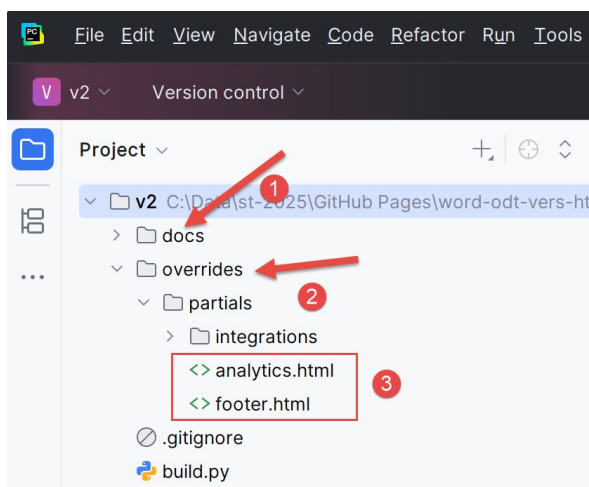
```

1. STYLES = {
2.     "style_names": [
3.         "P1"
4.     ]
5. }

```

- líneas 35-88 pregunté el IA para registrar todos los capítulos que encontrara;

Esta ejecución ha modificado tu carpeta de trabajo:

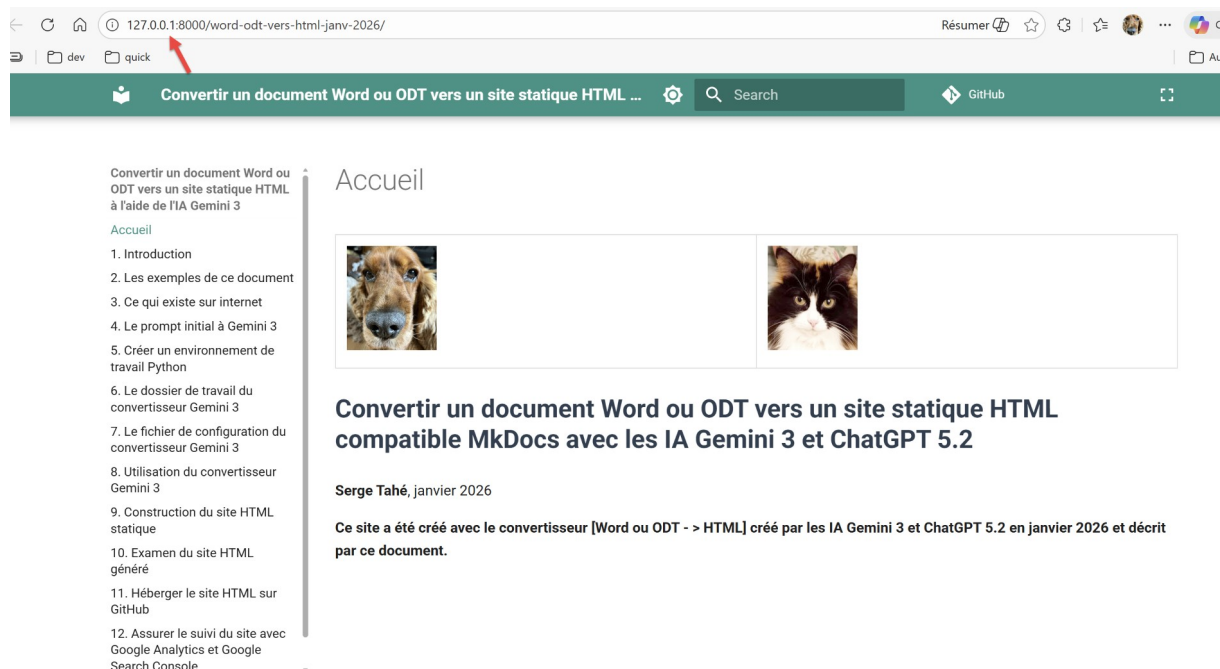


- En [1], [docs] es el sitio MkDocs que el convertidor Gemini / ChatGPT ha creado. Quizá le interese visitarlo;
- en [2], se ha creado una carpeta [overrides]. Será utilizada por el constructor [build] del sitio HTML;
- en [3]: [analytics.html] se utilizará para el seguimiento del sitio por parte de Google Analytics. [footer.html] es el pie de página que definió en el archivo [config.py] ;

Ahora mismo podemos utilizar el sitio MkDocs. El comando `[python -m mkdocs serve]` permite visualizarlo. Puede probar :

```
1. PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> python -m mkdocs serve
2. INFO - Building documentation...
3. INFO - Cleaning site directory
4. INFO - Doc file 'les-exemples.md' contains a link '#_Les_exemples', but there is no such anchor on this page.
5. INFO - Documentation built in 0.41 seconds
6. INFO - [15:46:06] Serving on http://127.0.0.1:8000/word-odt-vers-html-janv-2026/
```

Ctrl-chaga en el enlace de la línea 6. El sitio MkDocs debe aparecer :



Este es el resultado de mucho trabajo. Para detener el servidor MkDocs, basta con pulsar Ctrl-C en el terminal que lo inició.

9. Utilizar el conversor DOCX → HTML

El comando para convertir un documento Word DOCX es muy similar al de convertir un documento LibreOffice ODT. Vamos a cambiar el estilo del título del documento en `[config_styles.py]` :

```
1. STYLES = {
2.     "style_names": [
3.         "Titre"
4.     ]
5. }
```

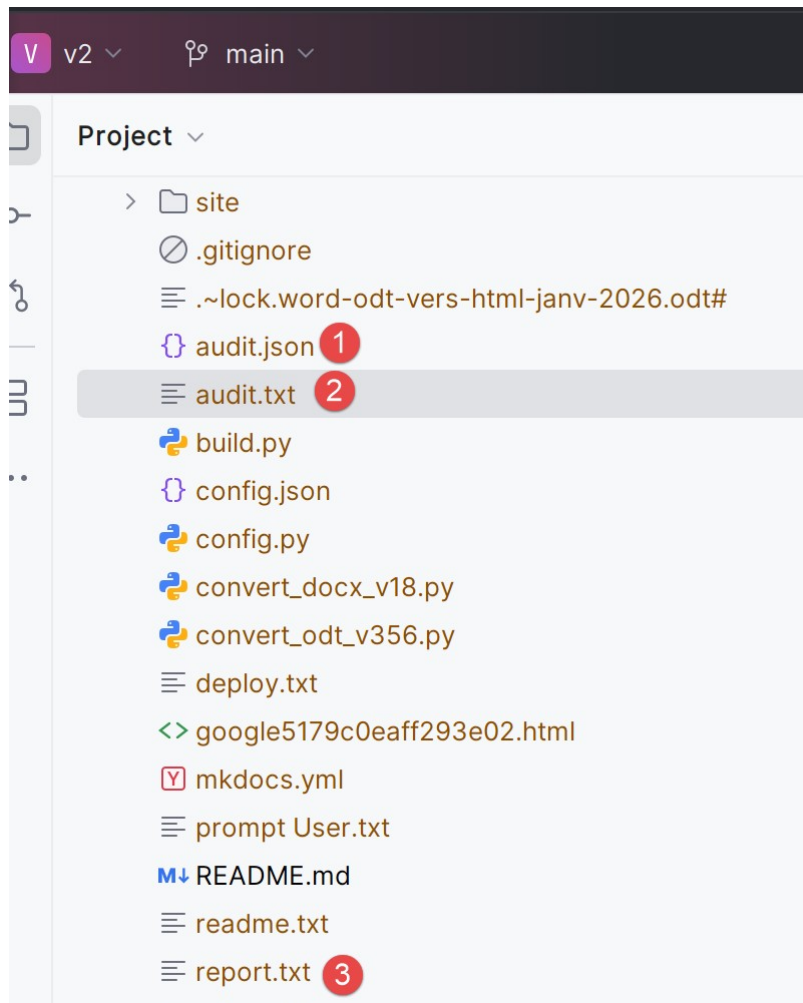
- línea 3 establecido en 'Título'. Este es el estilo del documento DOCX que va a convertir. Lo veremos en las líneas de depuración del conversor.

Todavía en el terminal PyCharm, escriba el siguiente comando:

```
1. PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> python .\convert_docx_v18.py .\word-odt-vers-html-janv-2026.docx .\config.py
2. C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2\convert_docx_v18.py:976: SyntaxWarning: invalid escape sequence '\h'
3. - REF Bookmark \h
4. --- DOCX to MkDocs Converter V16 ---
5. Copié : google5179c0eaff293e02.html
6. Copié : robots.txt
7. Copié : word-odt-vers-html-janv-2026.pdf
8. Copié : word-odt-vers-html-janv-2026.zip
9. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
10. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
11. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
12. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
13. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
14. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
15. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
16. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
17. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
18. [DEBUG PRE-H1] style=StandardWW heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
19. [DEBUG PRE-H1] style=StandardWW heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
20. [DEBUG PRE-H1] style=Titre heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='<span>Convertir un document Word ou ODT vers un site statique HTML compatible Mk...'
21. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
22. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
23. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
24. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='<b>Serge Tahé</b><span>, janvier 2026</span>...'
25. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
26. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
27. [DEBUG PRE-H1] style=Standard heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
28. [DEBUG PRE-H1] style=StandardWW heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
29. [DEBUG PRE-H1] style=StandardWW heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
30. [DEBUG PRE-H1] style=StandardWW heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='...'
31. [DEBUG PRE-H1] style=StandardWW heading=None rebase=0 numId=None ilvl=None list=None:0/None txt='<b>Ce site a été créé avec le convertisseur &#91;Word ou ODT - &gt; HTML&#93; cr...'
32. [DEBUG PRE-H1] style=Titre1 heading=1 rebase=0 numId=1 ilvl=0 list=numPr:1/ordered txt='<span>Introduction</span>...'
33. Terminé. (audit.json, audit.txt, report.txt générés)
```

- línea 1: el comando es el siguiente: `[python .\convert_docx_v18.py .\word-odt-vers-html-janv-2026.docx .\config.py]` (Adapte el número de versión (ici 18) a la versión que tenga descargado) :

- el primer parámetro `[\convert_docx_v18.py]` es el convertidor DOCX → HTML
- el segundo parámetro `[\word-odt-vers-html-janv-2026.docx]` es el nombre del documento DOCX a convertir ;
- el tercer parámetro `[\config.py]` es el ;
- línea 33: el conversor informa de que se han generado tres archivos:



El archivo `[audit.txt]` es el siguiente:

```

1. Version: V16
2. Paragraphs: 2029
3. Tables: 97
4. Images (blips): 2
5. Headings detected (raw): 53
6. Min heading level detected (raw): 1
7. Rebase offset applied: 0
8.
9. Top paragraph styles:
10.   - SourceCodenumrot: 1054
11.   - StandardWW: 594
12.   - Standard: 146
13.   - Paragraphedeliste: 113
14.   - SourceCodenumrotrsltats: 33
15.   - codenouveau: 28
16.   - Titre2: 25
17.   - Titre1: 14
18.   - Titre3: 14

```

```

19.      - StandardWWW: 6
20.      - Textebrut: 1
21.      - Titre: 1
22.
23.      List paragraphs:
24.      - with numPr: 1329
25.      - by style fallback: 49
26.      - not recognized: 0

```

- línea 2: el número de párrafos del documento Word ;
- línea 3: número de mesas ;
- líneas 9-21: estilos encontrados en el documento ;
 - líneas 10, 14, 15: el estilo de los bloques de código. Probablemente habría bastado con un único estilo;
 - líneas 11-12, 19: estilo de párrafo estándar. Probablemente habría bastado con un único estilo;
 - líneas 16-18, 21: los estilos de título del documento. En la línea 21, sólo un párrafo tiene el estilo "Título". Se trata del título del documento que precede al primer "Título1";

Esta auditoría de documentos Word es una buena forma de juzgar la calidad del documento. Aquí veo que he utilizado demasiados estilos diferentes para la misma cosa en mi documento de Word.

El fichero [audit.json] es idéntico al fichero [audit.txt] pero con la forma JSON :

```

1.      {
2.          "version": "V16",
3.          "file": "word-odt-vers-html-janv-2026.docx",
4.          "counts": {
5.              "paragraphs": 2029,
6.              "tables": 97,
7.              "image_blips": 2,
8.              "headings_raw": 53
9.          },
10.         "lists": {
11.             "with_numpr": 1329,
12.             "by_style": 49,
13.             "unrecognized": 0
14.         },
15.         "heading": {
16.             "min_level_raw": 1,
17.             "rebase_offset": 0
18.         },
19.         "top_styles": [
20.             [
21.                 "SourceCodenumrot",
22.                 1054
23.             ],
24.             [
25.                 "StandardWW",
26.                 594
27.             ],
28.             [
29.                 "Standard",
30.                 146
31.             ],
32.             [
33.                 "Paragraphedeliste",
34.                 113
35.             ],
36.             [
37.                 "SourceCodenumrotrsltats",
38.                 33
39.             ],
40.             [
41.                 "codenouveau",

```

```

42.         28
43.     ],
44.     [
45.         "Titre2",
46.         25
47.     ],
48.     [
49.         "Titre1",
50.         14
51.     ],
52.     [
53.         "Titre3",
54.         14
55.     ],
56.     [
57.         "StandardWWW",
58.         6
59.     ],
60.     [
61.         "Textebrut",
62.         1
63.     ],
64.     [
65.         "Titre",
66.         1
67.     ]
68. ]
69. }

```

El archivo [report.txt] es éste:

```

1. [SUMMARY] Listes détectées via fallback "par style" (agregé)
2.   - Paragraphedeliste -> level=1 type=unordered: 49
3.
4. [SUMMARY] Blocs Word ignorés (agregé)
5.   - <w:sectPr>: 1

```

No lo entendí..

Es posible solicitar sólo una auditoría del documento Word para juzgar su calidad con el parámetro [--audit]:

```
python .\convert_docx_v18.py .\word-odt-vers-html-janv-2026.docx .\config.py --audit
```

En este caso, sólo se realiza la auditoría de documentos. El sitio MkDocs no se genera.

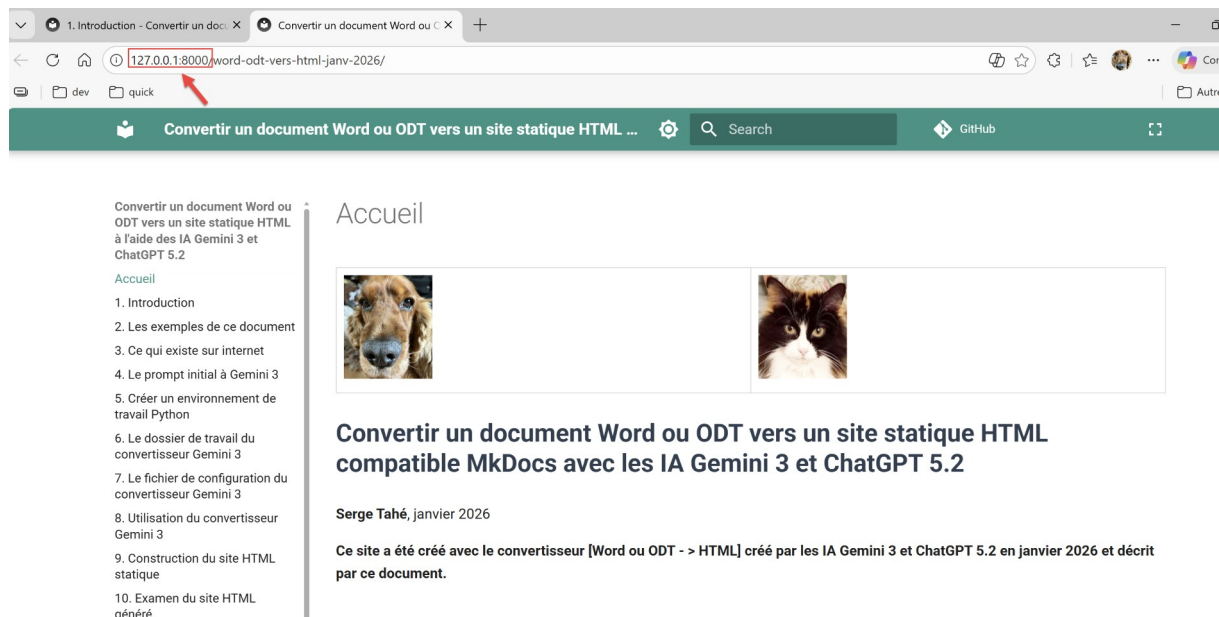
Como ya hemos visto, puede utilizar las siguientes opciones para visualizar el sitio MkDocs generado por el convertidor :

```

1. PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> python -m mkdocs serve
2. INFO - Building documentation...
3. INFO - Cleaning site directory
4. INFO - Documentation built in 0.59 seconds
5. INFO - [06:05:48] Serving on http://127.0.0.1:8000/word-odt-vers-html-janv-2026/

```

Ctrl-Clic en el URL en la línea 5 :



10. Construcción de obras HTML estático

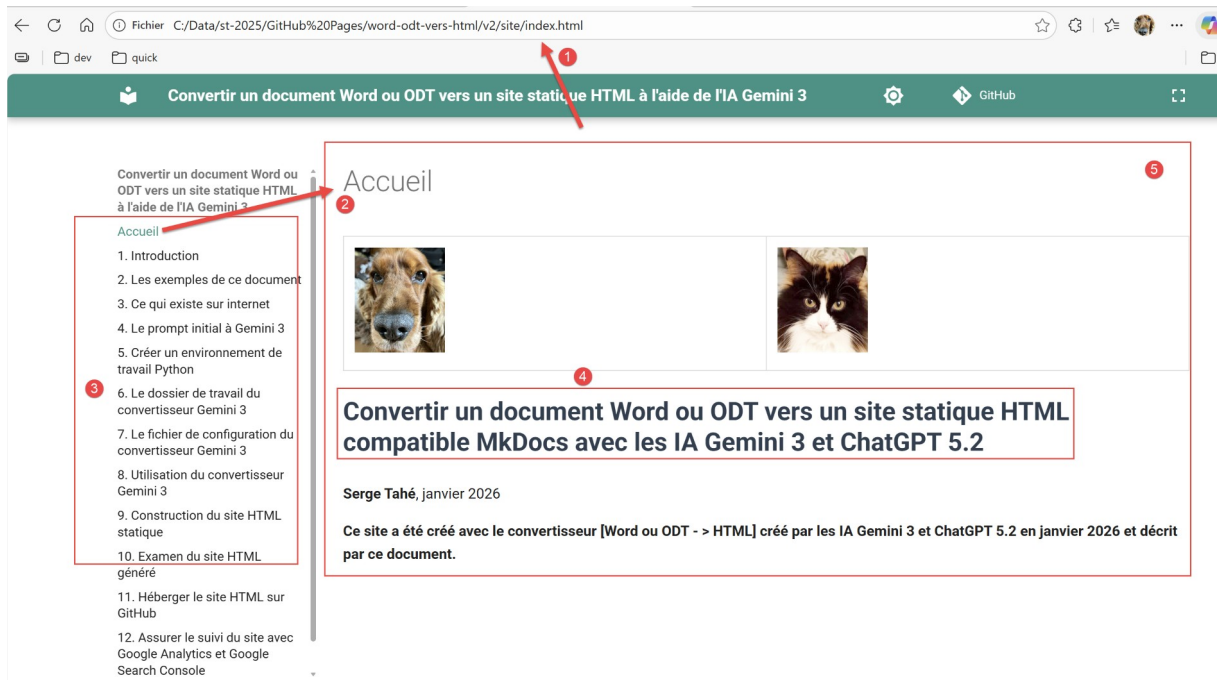
A partir de ahora no hay diferencia entre los documentos ODT y DOCX. Trabajamos en el sitio MkDocs construido por uno u otro de los convertidores.

Ahora vamos a construir el sitio HTML. Esto se hace con script [build]. Todavía en el terminal, escriba el siguiente comando:

```
1. PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> python .\build.py
2. Démarrage de la construction du site MkDocs...
3. Construction réussie ! Le site est dans : C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2\site
4. Ouverture de C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2\site\index.html dans le navigateur...
```

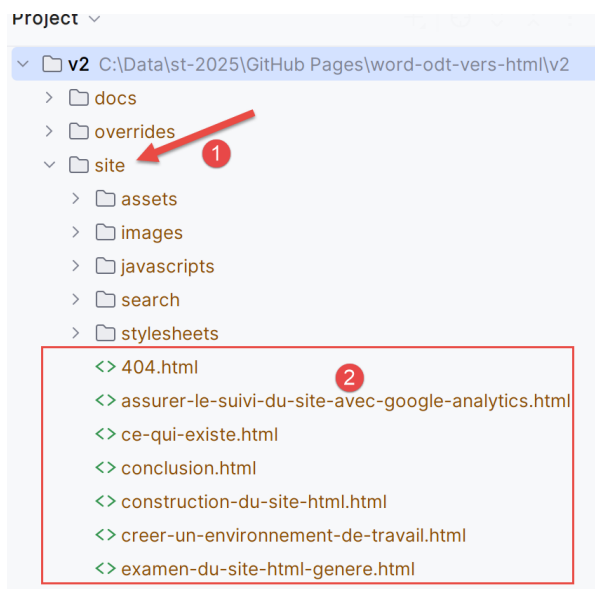
- línea 1: el comando [python build] crea el sitio HTML a partir del sitio MkDocs ;
- línea 4: el sitio se visualiza en un navegador ;

El resultado es el mismo que para el sitio MkDocs :



- En [1], vemos que se muestra una página HTML;
- En [2], esta página corresponde a la página de inicio del sitio;
- En [3], el índice del documento HTML ;
- En [4], el título del documento se formateaba según su configuración en [config.py] ;
- En [5] la primera página que aparece es [Inicio]. El contenido de esta página es el contenido del Documento ODT / DOCX que **precede al primer capítulo en el nivel del Título 1** ;

Ejecutar [build] ha modificado tu carpeta de trabajo:



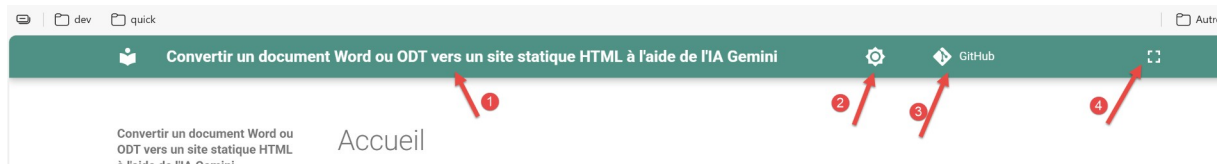
- En [1], se ha creado una nueva carpeta [site]. Contiene su sitio estático HTML ;
- En [2], las páginas de su sitio ;

11. Revisión de sitio web HTML generado

Ahora examinaremos la representación HTML de este documento ODT / DOCX. Ya hemos visto que el conversor respeta el índice.

11.1. La barra superior del sitio

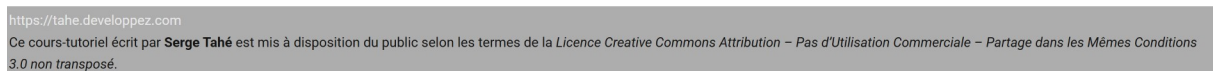
Rechemos un vistazo a la barra superior del sitio:



- En [1], el nombre del sitio definido en [config.py] ;
- En [2], el icono para cambiar al modo oscuro o claro;
- En [3], el icono que es un enlace al repositorio GitHub donde se exportará el sitio HTML. También definido en [config.py] ;
- En [4], el icono que permite ocultar/mostrar el índice;

11.2. Pie de página

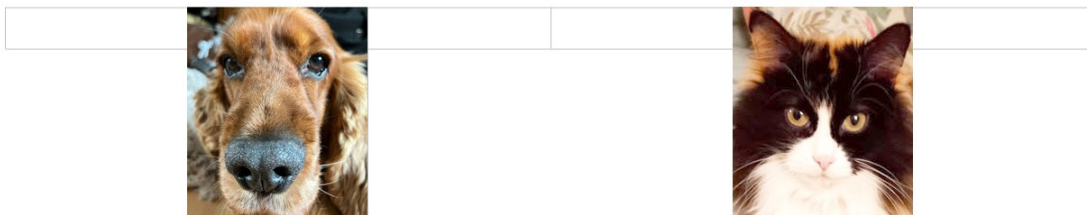
Ahora echemos un vistazo al pie de página:



Este es el pie de página definido en el archivo [config.py].

11.3. La página de inicio

La portada del Documento ODT / DOCX era la siguiente:



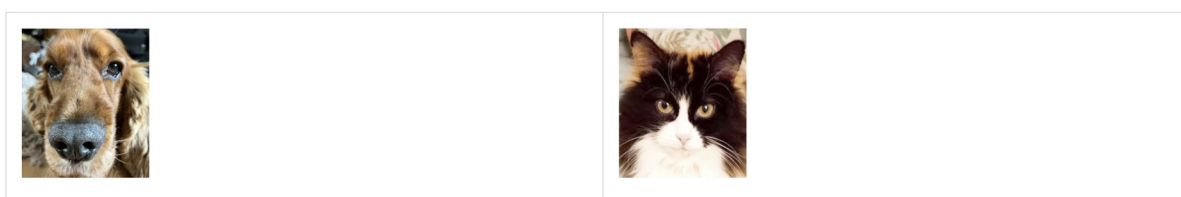
Convertir un document Word ou ODT vers un site statique HTML compatible MkDocs avec les IA Gemini 3 et ChatGPT 5.2

Serge Tahé, janvier 2026

Ce site a été créé avec le convertisseur [Word ou ODT - > HTML] créé par les IA Gemini 3 et ChatGPT 5.2 en janvier 2026 et décrit par ce document.

Esta portada del Documento ODT / DOCX se convierte en la página de inicio del sitio HTML :

Accueil



Convertir un document Word ou ODT vers un site statique HTML compatible MkDocs avec les IA Gemini 3 et ChatGPT 5.2 1

Serge Tahé, janvier 2026

Ce site a été créé avec le convertisseur [Word ou ODT - > HTML] créé par les IA Gemini 3 et ChatGPT 5.2 en janvier 2026 et décrit par ce document.

El Gemini 3 / convertidor ChatGPT pone todo lo que le precede en el Documento ODT / DOCX, el título de primer nivel, en la página de Inicio 1este es el estilo "Título 1". Si añades imágenes como arriba, las mostrará. Así que puedes imaginar darle a tu sitio una portada como la de un libro de verdad. En [1], este es el título principal del documento. Su representación está controlada por las siguientes líneas en el archivo de configuración [config_styles.py]:

```
1.     STYLES = {
2.         "style_names": [
3.             "P4"
4.         ]
5.     }
```

- líneas [6-8]: la lista de posibles estilos para el título de su documento. Cuando miro este documento, el style LibreOffice de mi título es "Título principal". Pero el convertidor Gemini no lo encontró. Registró los estilos que encontró y mostró [P1]. Este es un gran problema con LibreOffice: los nombres mostrados para los estilos no coinciden eno se corresponden con los nombres internos utilizados por el software. Sólo están ahí para adaptarse a la lengua del usuario;
- línea 10 una vez detectado el título principal, puede establecer los parámetros de representación. Yo quería un tamaño de fuente de 28 (font-size: 28px;) y negrita (font-weight: bold);

Con las imágenes y el estilo del título, puedes crear una página de inicio atractiva.

Es posible que el título principal de su documento no tenga uno de los estilos definidos en las líneas [6-8]. Para encontrar el estilo de su título principal, utilice la siguiente línea del archivo [config.py]

```
1.     "debug": True
```

Con el valor [true], se mostrará el estilo de los párrafos que preceden al primer título de nivel 1, es decir, los párrafos de la página de inicio, cuando se ejecute el convertidor Gemini / ChatGPT. Para un documento distinto de éste, obtuve los siguientes registros :

```
1.     [DEBUG PRE-H1] Style='Standard (WW)' (Clean='standard (ww)') | Texte='...'
2.     [DEBUG PRE-H1] Style='P1' (Clean='p1') | Texte='...'
3.     [DEBUG PRE-H1] Style='P1' (Clean='p1') | Texte='...'
4.     [DEBUG PRE-H1] Style='P1' (Clean='p1') | Texte='...'
5.     [DEBUG PRE-H1] Style='P3' (Clean='p3') | Texte='...'
6.     [DEBUG PRE-H1] Style='P1' (Clean='p1') | Texte='...'
7.     [DEBUG PRE-H1] Style='P1' (Clean='p1') | Texte='...'
8.     [DEBUG PRE-H1] Style='P1' (Clean='p1') | Texte='...'
9.     [DEBUG PRE-H1] Style='P1' (Clean='p1') | Texte='...'
10.    [DEBUG PRE-H1] Style='P4' (Clean='p4') | Texte='Introduction au langage PHP7 par l'exemp...'
11.    >>> TITRE DOCUMENT DETECTÉ : Introduction au langage PHP7 par l'exemple
12.    [DEBUG PRE-H1] Style='Standard (WW)' (Clean='standard (ww)') | Texte='...'
13.    [DEBUG PRE-H1] Style='Standard (WW)' (Clean='standard (ww)') | Texte='...'
14.    [DEBUG PRE-H1] Style='Standard (WW)' (Clean='standard (ww)') | Texte='...'
15.    [DEBUG PRE-H1] Style='Standard (WW)' (Clean='standard (ww)') | Texte='...'
16.    [DEBUG PRE-H1] Style='Standard (WW)' (Clean='standard (ww)') | Texte='...'
17.    [DEBUG PRE-H1] Style='Standard (WW)' (Clean='standard (ww)') | Texte='...'
18.    [DEBUG PRE-H1] Style='P5' (Clean='p5') | Texte='Serge Tahé, juillet 2019...'
19.    [DEBUG PRE-H1] Style='P5' (Clean='p5') | Texte='...'
20.    [DEBUG PRE-H1] Style='P6' (Clean='p6') | Texte='...'
21.    [DEBUG PRE-H1] Style='Heading 1' (Clean='heading 1') | Texte='Introduction au langage PHP 7...'
```

- línea 10, el título del documento tiene el estilo 'P4' ;

En el archivo [config.py], tengo entonces ponga las siguientes líneas :

```
1.     "document_title": {
2.         "style_names": [
3.             "P4"
4.         ],
5.         "css": "font-size: 28px; font-weight: bold; margin-bottom: 1em; line-height: 1.2; color: #2c3e50;"
6.     },
```

- línea 3, el estilo que estaba buscando ;

Por eso el depurador muestra las líneas :

```
1. [DEBUG PRE-H1] Style='P4' (Clean='p4') | Texte='Introduction au langage PHP7 par
  1'exemp...'
2. >>> TITRE DOCUMENT DETECTÉ : Introduction au langage PHP7 par 1'exemple
```

Ha encontrado el estilo "P4" y ahora muestra que se ha encontrado el título del documento. Una vez encontrado el título, puedes establecer la tecla [debug] en [false] en [config.py] :

```
1. "debug": False
```

Echemos un vistazo a la conversión de capítulo [Ejemplos], que agrupa los ejemplos que el convertidor Gemini / ChatGPT sabe cómo gestionar :

11.4. Las listas

Document ODT / DOCX :

Listes à puces

- Élément 1 ;
- Élément 2 ;
- Élément 3 ;
 - Élément 3.1 ;
 - Élément 3.1.1
 - Élément 3.1.2
 - Élément 3.1.2.1
 - Élément 3.1.2.2
 - Élément 3.2 ;
- Élément 4 ;

Arriba, el texto [Listas con viñetas] está resaltado porque es una referencia asociada a una referencia cruzada.

Documento HTML :

Listes à puces

- Élément 1 ;
- Élément 2 :
- Élément 3 ;
 - Élément 3.1 ;
 - Élément 3.1.1
 - Élément 3.1.2
 - Élément 3.1.2.1
 - Élément 3.1.2.2
 - Élément 3.2 ;
- Élément 4 ;

Tenga en cuenta que mientras el Documento ODT / DOCX utiliza diferentes chips, el Documento HTML utiliza un único tipo de chip.

11.5. Listas numeradas

Documento ODT / DOCX :

Listes numérotées

1. Élément 1 ;
2. Élément 2 ;
 - a. Élément 2.1
 - i. Élément 2.1.1
 1. Élément 2.1.1.1
 2. Élément 2.1.1.2
 - ii. Élément 2.1.2
 - b. Élément 2.2
3. Élément 3 ;

Documento HTML :

2.1.2. Listes numérotées

1. Élément 1 ;
2. Élément 2 ;
 - a. Élément 2.1
 - i. Élément 2.1.1
 1. Élément 2.1.1.1
 2. Élément 2.1.1.2
 - ii. Élément 2.1.2
 - b. Élément 2.2
3. Élément 3 ;

11.5.1. Listas mixtas 1

Documento ODT / DOCX

2.1.3. Listes mixtes 1

- Élément 1 ;
- Élément 2 ;
- Élément 3 ;
 - Élément 3.1 ;
 1. Élément 3.1.1
 2. Élément 3.1.2
 - Élément 3.1.2.1
 - Élément 3.1.2.2
 - Élément 3.2 ;
- Élément 4 ;

Documento HTML

2.1.3. Listes mixtes 1

- Élément 1 ;
- Élément 2 :
- Élément 3 ;
 - Élément 3.1 ;
 1. Élément 3.1.1
 2. Élément 3.1.2
 - Élément 3.1.2.1
 - Élément 3.1.2.2
 - Élément 3.2 ;
- Élément 4 ;

También en este caso hay a veces diferencias entre los tipos de chip utilizados.

11.5.2. Listas mixtas 2

Documento ODT / DOCX

2.1.4. Listes mixtes 2

1. Élément 1 ;
2. Élément 2 ;
 - a. Élément 2.1
 - i. Élément 2.1.1
 - Élément 2.1.1.1
 - Élément 2.1.1.2
 - ii. Élément 2.1.2
 - b. Élément 2.2
3. Élément 3 ;

Documento HTML

2.1.4. Listes mixtes 2

1. Élément 1 ;
2. Élément 2 ;
 - a. Élément 2.1
 - i. Élément 2.1.1
 - Élément 2.1.1.1
 - Élément 2.1.1.2
 - ii. Élément 2.1.2
 - b. Élément 2.2
3. Élément 3 ;

11.6. En bloquea de código enrichécido

Los bloques de código enrichécido se representan de forma idéntica en el HTML (aparte del color de fondo). He aquí tres ejemplos:

11.6.1. Ejemplo 1

Documento ODT / DOCX

Java

```
1. package istia.st.spring.core;
2.
3. import java.util.ArrayList;
4. import java.util.List;
5.
6. import org.springframework.context.ApplicationContext;
7. import org.springframework.context.support.ClassPathXmlApplica
8.
9. public class Demo01 {
10.
11.     @SuppressWarnings({ "unchecked", "resource" })
12.     public static void main(String[] args) {
13.         // récupération du contexte Spring
14.         ApplicationContext ctx = new ClassPathXmlAppli
15.         // on récupère les beans
16.         Personne p01 = ctx.getBean("personne_01", Perso
17.         Personne p02 = ctx.getBean("personne_02", Perso
18.         List<Personne> club = ctx.getBean("club", new
19.         ArrayList<Personne>().getClass());
20.         Appartement appart01 = ctx.getBean(Appartement
```

Rendu HTML

Java

```
1 package istia.st.spring.core;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 import org.springframework.context.ApplicationContext;
7 import org.springframework.context.support.ClassPathXmlApplicat
8
9 public class Demo01 {
10
11     @SuppressWarnings({ "unchecked", "resource" })
12     public static void main(String[] args) {
13         // récupération du contexte Spring
14         ApplicationContext ctx = new ClassPathXmlApplicationCor
15         // on récupère les beans
16         Personne p01 = ctx.getBean("personne_01", Personne.class);
17         Personne p02 = ctx.getBean("personne_02", Personne.class);
18         List<Personne> club = ctx.getBean("club", new ArrayList<Personne>());
19         Appartement appart01 = ctx.getBean(Appartement.class);
```

11.6.2. Ejemplo 2

Documento ODT / DOCX :

Python

```
1. # -----
2. def affiche(chaine):
3.     # affiche chaine
4.     print("chaine=%s" % chaine)
5.
6.
7. # -----
8. def affiche_type(variable):
9.     # affiche le type de variable
10.    print("type[%s]=%s" % (variable, type(variable)))
11.
12.
13. # -----
14. def f1(param):
15.     # ajoute 10 à param
16.     return param + 10
17.
18.
19. # -----
20. def f2():
21.     # rend un tuple de 3 valeurs
22.     return "un", 0, 100
23.
24.
25. # -----
26. ...
```

Renderizado HTML

Python

```
1 # -----
2 def affiche(chaine):
3     # affiche chaine
4     print("chaine=%s" % chaine)
5
6
7 # -----
8 def affiche_type(variable):
9     # affiche le type de variable
10    print("type[%s]=%s" % (variable, type(variable)))
11
12
13 # -----
14 def f1(param):
15     # ajoute 10 à param
16     return param + 10
17
18
19 # -----
20 def f2():
21     # rend un tuple de 3 valeurs
22     return "un", 0, 100
```

11.6.3. Ejemplo 3

Documento ODT / DOCX

ECMAScript

```
1. 'use strict';
2. // ceci est un commentaire
3. // constante
4. const nom = "dupont";
5. // un affichage écran
6. console.log("nom : ", nom);
7. // un tableau avec des éléments de type diffère
8. const tableau = ["un", "deux", 3, 4];
9. // son nombre d'éléments
10. let n = tableau.length;
11. // une boucle
12. for (let i = 0; i < n; i++) {
13.     console.log("tableau[" + i + "] = ", tableau[i]);
14. }
15. // initialisation de 2 variables avec le conter
16. let [chaine1, chaine2] = ["chaine1", "chaine2"];
17. // concaténation des 2 chaînes
18. const chaine3 = chaine1 + chaine2;
19. // affichage résultat
20. console.log([chaine1, chaine2, chaine3]);
21. ...
```

Renderizado HTML

ECMAScript

```
1  'use strict';
2  // ceci est un commentaire
3  // constante
4  const nom = "dupont";
5  // un affichage écran
6  console.log("nom : ", nom);
7  // un tableau avec des éléments de type différent
8  const tableau = ["un", "deux", 3, 4];
9  // son nombre d'éléments
10 let n = tableau.length;
11 // une boucle
12 for (let i = 0; i < n; i++) {
13   console.log("tableau[" + i + "] = ", tableau[i]);
14 }
15 // initialisation de 2 variables avec le contenu d
16 let [chaine1, chaine2] = ["chaine1", "chaine2"];
17 // concaténation des 2 chaînes
18 const chaine3 = chaine1 + chaine2;
19 // affichage résultat
20 console.log([chaine1, chaine2, chaine3]);
21 ...
```

11.7. Bloques de código en bruto (sin formato text)

Los bloques de código sin procesar que se encuentran en el Documento ODT / DOCX están coloreados sintácticamente por MkDocs según el idioma encontrado en el bloque de código. Para ayudar al conversor a encontrar el idioma correcto, ponemos "cadenas clave" en el archivo [config.py] para cada idioma. El conversor cuenta las "cadenas clave" encontradas. A continuación, asocia el bloque de código al idioma con más "cadenas clave" encontradas.

Veamos algunos ejemplos.

11.7.1. Ejemplo 1

Documento ODT / DOCX (Java)

Java

```
1. package istia.st.spring.core;
2.
3. import java.util.ArrayList;
4. import java.util.List;
5.
6. import org.springframework.context.ApplicationContext;
7. import org.springframework.context.support.ClassPathXmlApplicationContext;
8.
9. public class Demo01 {
10.
11.     @SuppressWarnings({ "unchecked", "resource" })
12.     public static void main(String[] args) {
13.         // récupération du contexte Spring
14.         ApplicationContext ctx = new ClassPathXmlApplicationContext("config-01.xml");
15.         // on récupère les beans
16.         Personne p01 = ctx.getBean("personne_01", Personne.class);
17.         Personne p02 = ctx.getBean("personne_02", Personne.class);
18.         List<Personne> club = ctx.getBean("club", new ArrayList<Personne>().getClass());
19.         Appartement appart01 = ctx.getBean(Appartement.class);
20.         ...
    }
```

Renderizado HTML

Java

```
1 package istia.st.spring.core;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 import org.springframework.context.ApplicationContext;
7 import org.springframework.context.support.ClassPathXmlApplicationContext;
8
9 public class Demo01 {
10
11     @SuppressWarnings({ "unchecked", "resource" })
12     public static void main(String[] args) {
13         // récupération du contexte Spring
14         ApplicationContext ctx = new ClassPathXmlApplicationContext("config-01.xml");
15         // on récupère les beans
16         Personne p01 = ctx.getBean("personne_01", Personne.class);
17         Personne p02 = ctx.getBean("personne_02", Personne.class);
18         List<Personne> club = ctx.getBean("club", new ArrayList<Personne>().getClass());
19         Appartement appart01 = ctx.getBean(Appartement.class);
20         ...
    }
```

En el documento HTML podemos ver que el código Java ha sido coloreado sintácticamente.

11.7.2. Ejemplo 2

Documento ODT / DOCX (XML)

```

1.      <?xml version="1.0" encoding="utf-8" ?>
2.      <configuration>
3.
4.          <configSections>
5.              <sectionGroup name="spring">
6.                  <section name="context" type="Spring.Context.Support.Contextt
7.                  <section name="objects" type="Spring.Context.Support.DefaultS
Spring.Core" />
8.              </sectionGroup>
9.          </configSections>
10.
11.          <spring>
12.              <context>
13.                  <resource uri="config://spring/objects" />
14.              </context>
15.              <objects xmlns="http://www.springframework.net">
16.                  <object name="dao" type="Dao.DataBaseImpot, ImpotsV7-dao">
17.                      <constructor-arg index="0" value="MySQL.Data.MySqlClient"/>
18.                      <constructor-arg index="1"
value="Server=localhost;Database=bdimpots;Uid=admimpots;Pwd=mdpimpots;"/>
19.                      <constructor-arg index="2" value="select limite, coeffr, co
20.                  </object>
21.                  <object name="metier" type="Metier.ImpotMetier, ImpotsV7-met:
22.                      <constructor-arg index="0" ref="dao"/>
23.                  </object>
24.              </objects>
25.          </spring>
26.      </configuration>

```

Renderizado HTML

XML

```

1      <?xml version="1.0" encoding="utf-8" ?>
2      <configuration>
3
4          <configSections>
5              <sectionGroup name="spring">
6                  <section name="context" type="Spring.Context.Support
7                  <section name="objects" type="Spring.Context.Support
8              </sectionGroup>
9          </configSections>
10
11          <spring>
12              <context>
13                  <resource uri="config://spring/objects" />
14              </context>
15              <objects xmlns="http://www.springframework.net">
16                  <object name="dao" type="Dao.DataBaseImpot, ImpotsV
17                      <constructor-arg index="0" value="MySQL.Data.MySql
18                      <constructor-arg index="1" value="Server=localhos
19                      <constructor-arg index="2" value="select limite, c
20                  </object>
21                  <object name="metier" type="Metier.ImpotMetier, Impo
22                      <constructor-arg index="0" ref="dao"/>
23                  </object>
24              </objects>
25          </spring>
26      </configuration>

```

11.7.3. Ejemplo 3

Documento ODT / DOCX (HTML)

HTML

```
1. <!DOCTYPE HTML>
2. <HTML>
3.   <head>
4.     <title>Laragon</title>
5.
6.     <link href="https://fonts.googleapis.com/css?
   type="text/css">
7.
8.     <style>
9.       HTML, body {
10.        height: 100%;
11.      }
12.
13.      body {
14.        margin: 0;
15.        padding: 0;
16.        width: 100%;
17.        display: table;
18.        font-weight: 100;
19.        font-family: 'Karla';
20.      }
21.
22.      .container {
23.        text-align: center;
24.        display: table-cell;
25.        vertical-align: middle;
26.      }
27.
28.      .content {
29.        text-align: center;
30.        display: inline-block;
31.      }
```

Renderizado HTML

HTML

```
1  <!DOCTYPE HTML>
2  <HTML>
3      <head>
4          <title>Laragon</title>
5
6          <link href="<a href="view-source:https://font
7
8      <style>
9          HTML, body {
10             height: 100%;
11          }
12
13          body {
14             margin: 0;
15             padding: 0;
16             width: 100%;
17             display: table;
18             font-weight: 100;
19             font-family: 'Karla';
20          }
21
22          .container {
23             text-align: center;
24             display: table-cell;
25             vertical-align: middle;
26          }
27
```

11.8. Otros bloques de código

Documento ODT / DOCX

Un resultado de ejecución con una primera línea que no empieza por 1 :

Résultats d'exécution

On notera que le code ne commence pas avec la ligne n° 1.

```
7.  C:\Data\st-2020\dev\python\cours-2020\python3-flask-2020\ve
    C:/Data/st-2020/dev/python/cours-2020/python3-flask-2020/ba
8.  nom=dupont
9.  liste[0]=un
10. liste[1]=deux
11. liste[2]=3
12. liste[3]=4
13. [chaine1,chaine2,chaine1chaine2]
14. chaine=chaine1
15. type[4]=<class 'int'>
16. type[chaine1]=<class 'str'>
17. type[['un', 'deux', 3, 4]]=<class 'list'>
18. type[a changé]=<class 'str'>
```

Renderizado HTML

Résultats d'exécution

On notera que le code ne commence pas avec la ligne n° 1.

```
7 C:\Data\st-2020\dev\python\cours-2020\python3-flask-2
8 nom=dupont
9 liste[0]=un
10 liste[1]=deux
11 liste[2]=3
12 liste[3]=4
13 [chaine1,chaine2,chaine1chaine2]
14 chaine=chaine1
15 type[4]=<class 'int'>
16 type[chaine1]=<class 'str'>
17 type[['un', 'deux', 3, 4]]=<class 'list'>
18 type[a changé]=<class 'str'>
```

Un bloque de código sin numerar en ODT permanece sin numerar en HTML :

Documento ODT / DOCX

Le convertisseur sait gérer les blocs de code non numérotés.

```
liste[0]=un
liste[1]=0
liste[2]=100
liste[0]=8
liste[1]=5
somme=13
```

Renderizado HTML

Le convertisseur sait gérer les blocs de code non numérotés.

```
liste[0]=un
liste[1]=0
liste[2]=100
liste[0]=8
liste[1]=5
somme=13
```

11.9. Enlaces

Documento ODT / DOCX

2.3. Les liens

Le convertisseur Gemini sait conserver les liens externes du document ODT. Par exemple [Gemini 3](#) ou [\[Générer un script Python avec des outils d'IA\]](#).

Il sait gérer un lien vers un chapitre [Lien vers un chapitre](#)

Un renvoi vers un chapitre : [2.1](#).

Un renvoi vers un repère de texte qui précède : [Gemini 3](#)

Un renvoi vers un repère de texte qui suit : [GitHub](#)

Renderizado HTML

2.3. Les liens

Le convertisseur Gemini sait conserver les liens externes du document ODT. Par exemple [Gemini 3](#) ou [\[Générer un script Python avec des outils d'IA\]](#).

Il sait gérer un lien vers un chapitre [Lien vers un chapitre](#)

Un renvoi vers un chapitre : [2.1](#).

Un renvoi vers un repère de texte qui précède : [Gemini 3](#)

Un renvoi vers un repère de texte qui suit : [GitHub](#)

11.10. Enriquecimiento del texto

Documento ODT / DOCX

Un texte avec des mots en **gras**, en *italiques*, soulignés ou **surlignés** ou **surlignés** ou **surlignés**.

C'est vrai également pour les liens : [\[Générer un script Python avec des outils d'IA\]](#).

Le **convertisseur** **gère** également la **couleur** des **caractères**.

Il gère également les bordures supérieure et inférieure des paragraphes.

2.5. Un **titre** peut être également enrichi.

Renderizado HTML

2.4. L'enrichissement de texte

Le convertisseur sait gérer le gras, l'italique, le souligné et le surlignage. Il respecte la couleur du surlignage.

Un texte avec des mots en **gras**, en *italiques*, soulignés ou surlignés ou surlignés ou surlignés.

C'est vrai également pour les liens : [\[Générer un script Python avec des outils d'IA\]](#).

Le **convertisseur** **gère** également la **couleur** des **caractères**.

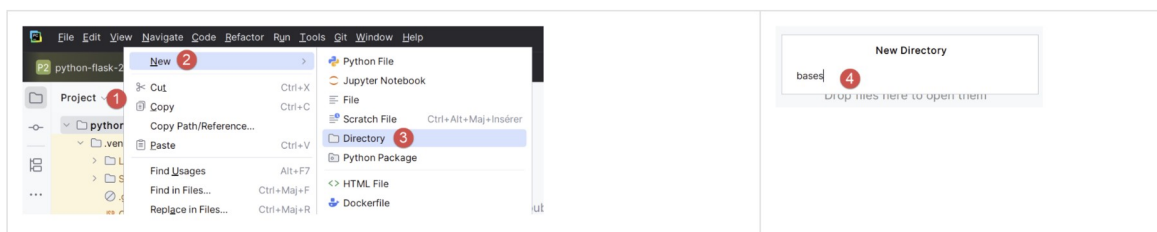
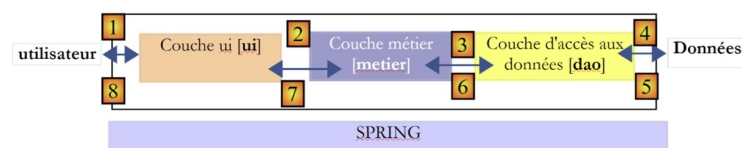
Il gère également les bordures supérieure et inférieure des paragraphes.

2.5. Un titre peut être également enrichi.

11.11. Las imágenes

2.5. Les images

Le convertisseur Gemini sait gérer les images et les tableaux d'images :



Cabe señalar que el Gemini / ChatGPT respeta el redimensionamiento de la imagen realizado en el Documento ODT / DOCX.

11.12. Caracteres protegidos

Documento ODT / DOCX

L'astérisque * a une signification Markdown. La ligne suivante peut être alors mal interprétée :

L'impôt I est alors égal à $0.15 \cdot R - 2072.5 \cdot \text{nbParts}$.

Renderizado HTML

L'astérisque * a une signification Markdown. La ligne suivante peut être alors mal interprétée :

L'impôt I est alors égal à **0.15*R – 2072.5*nbParts**.

Documento ODT / DOCX (Markdown)

```
5.      ---
6.
7.      ## 📄 Description
8.
9.      On se propose dans ce projet de mettre à disposition du
documents Word ou ODT vers un site statique HTML.
10.
11.      Lorsque le document ODT convient, le convertisseur prod
l'aspect professionnel des sites produits par Pandoc.
12.
13.      ## 📄 Contexte de création
14.
15.      Ce convertisseur a été entièrement construit par l'IA *
est le résultat d'itérations successives pour gérer finement la
(OpenDocument Text).
16.
17.      ## ✨ Fonctionnalités
18.
19.      Le script `convert.py` effectue les actions suivantes :
20.
21.      * **Conversion ODT vers Markdown** : Analyse le fichier
structure.
22.      * **Gestion des Titres** : Génère automatiquement la Ta
latérale.
23.      * **Blocs de Code** : Détection automatique des langage
précise de la numérotation des lignes** (attributs `start-value
24.      * **Listes** : Support des listes à puces et numérotées
25.      * **Mise en forme** : Support du *gras*, *italique*, *s
respect des couleurs d'origine).
```

Renderizado HTML

Un autre exemple est lorsque vous voulez insérer un bloc de code MarkDown dans votre document

```
1 # Convertisseur Word/ODT vers Site HTML (MkDocs)
2
3  **[Voir le site de démonstration généré](https://stahe.github.io/word-odt-v
4
5 ---
6
7 ##  Description
8
9 On se propose dans ce projet de mettre à disposition du lecteur un convertisseur
10
11 Lorsque le document ODT convient, le convertisseur produit un site HTML via **/
12
13 ##  Contexte de création
14
15 Ce convertisseur a été entièrement construit par l'IA **Gemini 3** (avec un abo
16
17 ##  Fonctionnalités
18
19 Le script `convert.py` effectue les actions suivantes :
20
```

- Il código MarkDown se ha conservado ;

11.13. Los cuadros

Documento ODT / DOCX

1	2		<pre>1. package istia.st.spring.core; 2. 3. import java.util.ArrayList; 4. import java.util.List; 5. 6. import org.springframework.context.ApplicationContext; 7. import org.springframework.context.support.ClassPathXmlApplication Context; 8. 9. public class Demo01 { 10. 11.</pre>
3	4		<pre>1. # ----- 2. def affiche(chaine): 3. # affiche chaine 4. print("chaine=%s" % chaine) 5. 6. 7. # ----- 8. def affiche_type(variable): 9. # affiche le type de variable 10. print("type[%s]=%s" % (variable, type(variable))) 11. 12.</pre>

Renderizado HTML

Un tableau peut contenir différents contenus :

1	2		<pre>1 package istia.st.spring.core; 2 3 import java.util.ArrayList; 4 import java.util.List; 5 6 import org.springframework.context.ApplicationContext; 7 import org.springframework.context.support.ClassPathXmlApplicationContext; 8 9 public class Demo01 { 10 11</pre>
3	4		<pre>1 # ----- 2 def affiche(chaine): 3 # affiche chaine 4 print("chaine=%s" % chaine) 5</pre>

11.14. Nnotas a pie de página

Le convertisseur Gemini² gère les notes de bas de page. Voici une autre note³ de bas de page.

-
1. MkDocs
 2. Google Gemini
 3. La note de bas de page

11.15. Anomalías conocidas

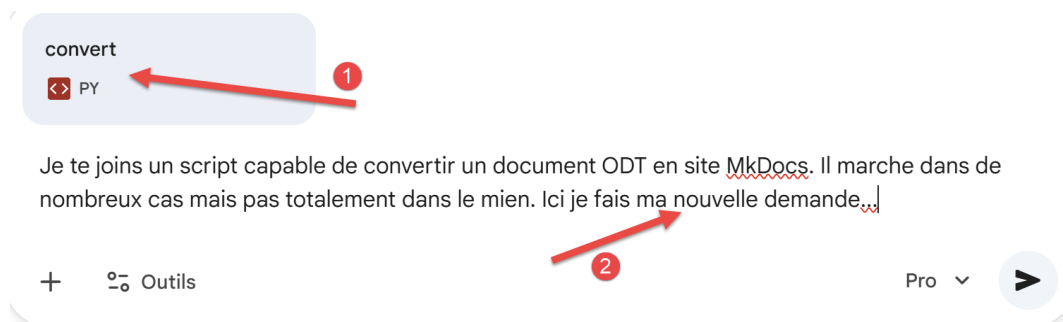
Algunos se han detectado anomalías que pueden corregirse modificando el ODT / DOCX :

- **los bloques de código deben ir seguidos de una línea vacía** de lo contrario, el bloque de código puede renderizarse mal. El caso identificado es un código inmediatamente seguido de un título sin estar separado de él por una línea vacía;
- las listas con viñetas no pueden tener borde inferior. **Para conseguirlo, añade una línea vacía detrás del último elemento de la lista ;**
- **las listas con viñetas deben ser jerárquicas.** Así pues, una lista de nivel 2 siempre debe estar incluida en una lista de nivel 1; de lo contrario, la lista de nivel 2 se mostrará como ;

Con cada nueva versión del convertidor, algunas de estas anomalías están destinadas a desaparecer. Las tres anomalías anteriores pueden evitarse corrigiendo el documento fuente.

11.16. Otros casos

Si su documento utiliza características distintas de las mencionadas, es muy probable que el conversor Gemini no las tenga en cuenta ChatGPT. ¿Qué debo hacer entonces? Puede enviar su nueva solicitud a uno de los IA dándole el convertidor de corriente :



- En [1], adjunto el convertidor de este documento;
- En [2], presento mi nueva solicitud;

Probablemente pasarás por muchas iteraciones. Cuando una versión sea estable, anota su número para poder devolvérsela a el IA en caso de regresión. También es aconsejable hacer una copia de cada versión estable. Un inconveniente importante de los dos IA es que ellos descensont con bastante facilidad. Todo lo que tienes que hacer es pedirle una nueva función y estará listo para funcionar el IA rompe el código que funcionaba antes. De ahí la importancia de anotar el número de versión de las versiones estables para poder volver a ellas. En enero de 2026, me pareció que ChatGPT 5.2 me teníanque Gemini 3.

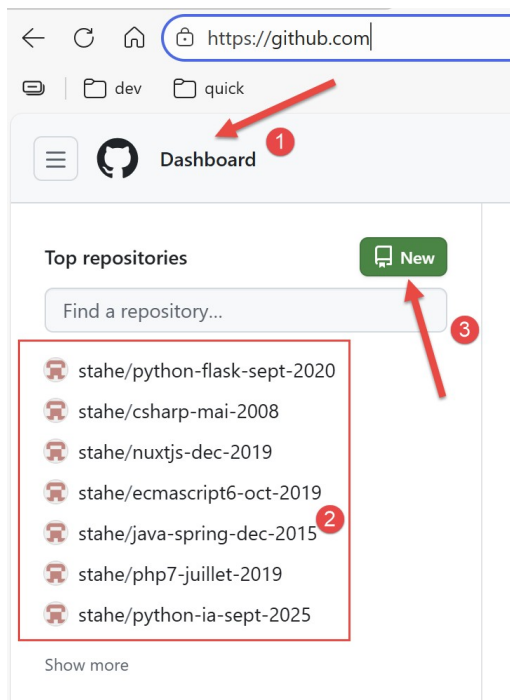
12. Aloje el sitio web HTML en GitHub

Fue el propio Gemini quien se ofreció a alojar el sitio HTML producido por sus dos guiones en GitHub⁴. No sabía que eso fuera posible. GitHub es un sitio que alberga proyectos de desarrollo. Exportar cursos de programación allí parece natural.

Primero necesitas tener una cuenta [GitHub](https://github.com). Si es necesario, créalo.

Conéctese a su cuenta GitHub:

⁴Nota a pie de página para GitHub



- En [2], sus repositorios existentes si los tiene;
- En [3], cree un nuevo repositorio GitHub ;

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1 General

Owner * stahe / Repository name * 1

✔ word-odt-vers-html-janv-2026 is available.

Great repository names are short and memorable. How about [sturdy-telegram](#)?

Description

2

84 / 350 characters

2 Configuration

Choose visibility * Public

Choose who can see and commit to this repository

Add README Off

READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

Add license No license

Licenses explain how others can use your code. [About licenses](#)

3 Create repository

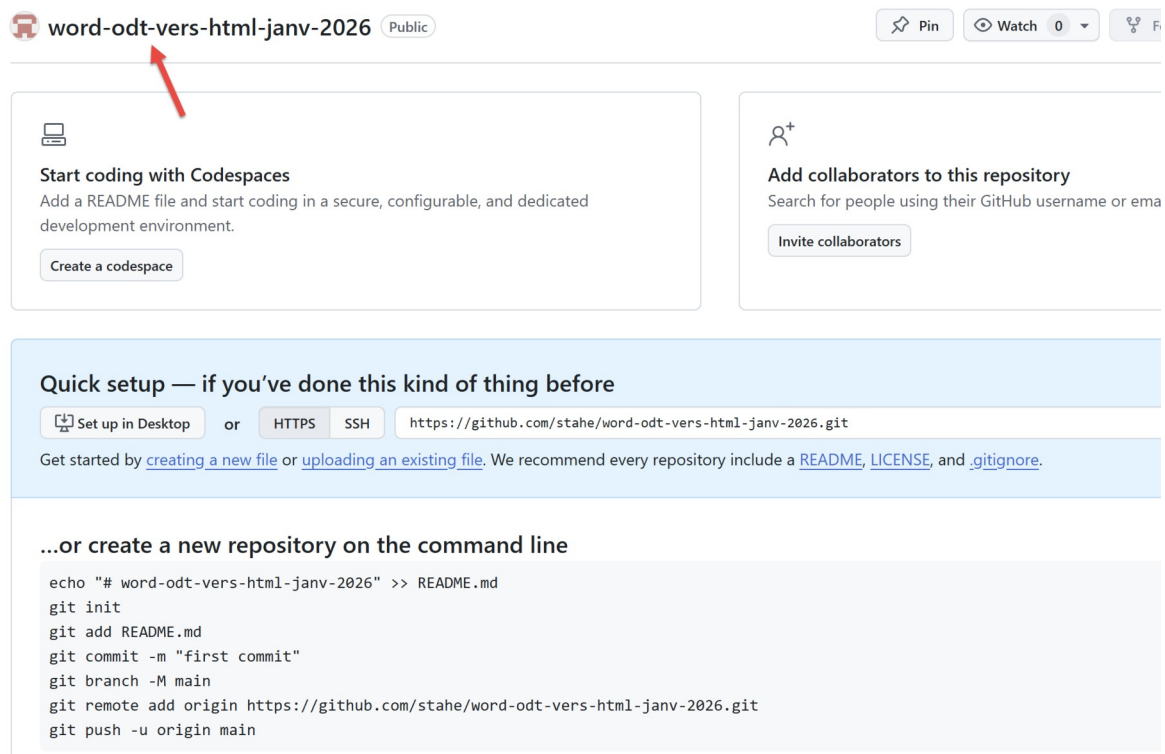
- En [1], utilice el nombre que introdujo en [config.py] :

```
1. "repo_url": "https://github.com/stahe/word-odt-vers-html-janv-2026",
```

- En [2], también puede introducir lo mismo que en [config.py] :

```
1. "site_description": "Convertir un document Word ou ODT vers un site statique HTML à l'aide de les IA Gemini 3 et ChatGPT 5.2",
```

- En [3], confirme la creación de su repositorio GitHub;

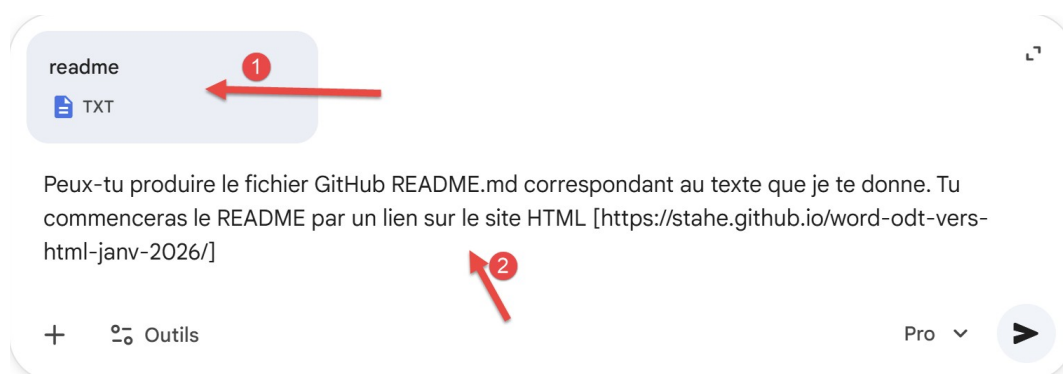


Es aconsejable que cada repositorio GitHub cree un archivo [README.md] que se mostrará en lugar de la pantalla anterior. Debe estar escrito en Markdown, lo que no es fácil. Así que escribiremos un fichero de texto con lo que queremos ver en README. Luego pediremos el IA para darnos el README.md correspondiente.

El archivo de texto [readme.txt] será el siguiente:

1. On se propose dans cet article de mettre à disposition du lecteur un convertisseur Python de documents Word ou ODT vers un site statique HTML.
- 2.
3. Ce convertisseur a été construit par l'IA Gemini 3.
- 4.
5. Lorsque le document ODT convient, le convertisseur produit un site HTML via MkDocs qui a l'aspect professionnel des sites produits par Pandoc.
- 6.
7. J'ai utilisé Gemini 3 avec un abonnement pro.

El indicador Gemini es el siguiente:



- En [1], el archivo adjunto [readme.txt] ;
- En [2], el prompt ;

Copia la respuesta que te da Gemini en el archivo [README.md] de tu carpeta de trabajo. Aquí está la respuesta que Gemini me dio:

```

1.      # Convertisseur Word/ODT vers Site HTML (MkDocs)
2.
3.      🔗 **[Voir le site de démonstration généré](https://stahe.github.io/word-odt-vers-html-
4.      janv-2026/)**
5.      ---
6.
7.      ## 📄 Description
8.
9.      On se propose dans ce projet de mettre à disposition du lecteur un convertisseur Python de
10.     documents Word ou ODT vers un site statique HTML.
11.
12.     Lorsque le document ODT convient, le convertisseur produit un site HTML via **MkDocs** qui a
13.     l'aspect professionnel des sites produits par Pandoc.
14.
15.     ## 📄 Contexte de création
16.
17.     Ce convertisseur a été entièrement construit par l'IA **Gemini 3** (avec un abonnement pro). Il
18.     est le résultat d'itérations successives pour gérer finement la structure des documents ODT
19.     (OpenDocument Text).
20.
21.     ## ✨ Fonctionnalités
22.
23.     Le script `convert.py` effectue les actions suivantes :
24.
25.     * **Conversion ODT vers Markdown** : Analyse le fichier `.odt` (XML) pour en extraire la
26.     structure.
27.     * **Gestion des Titres** : Génère automatiquement la Table des Matières (TOC) et la navigation
28.     latérale.
29.     * **Blocs de Code** : Détection automatique des langages, coloration syntaxique et **gestion
30.     précise de la numérotation des lignes** (attributs `start-value`).
31.     * **Listes** : Support des listes à puces et numérotées avec indentation correcte.
32.     * **Mise en forme** : Support du *gras*, *italique*, *souligné* et du *surlignage* (avec
33.     respect des couleurs d'origine).
34.     * **Images** : Extraction et intégration automatique des images contenues dans le document.
35.     * **Configuration** : Personnalisation via un fichier `config.py` (Bas de page, Google
36.     Analytics, etc.).
37.
38.     ## 🚀 Installation
39.
40.     ### Prérequis
41.
42.     * Python 3.x
43.     * Les bibliothèques suivantes :
44.
45.     ```bash
46.     pip install odfpy unicode mkdocs mkdocs-material
47.     ```
48.
49.     ### Structure du projet
50.
51.     Assurez-vous d'avoir les fichiers suivants :
52.
53.     * `convert.py` : Le script de conversion.
54.     * `config.py` : Votre fichier de configuration.
55.     * `votre-document.odt` : Le document source.
56.
57.     ## 📖 Utilisation
58.
59.     1. **Conversion**
60.
61.     Lancez le script en indiquant le fichier ODT source et le fichier de configuration :
62.
63.     ```bash

```

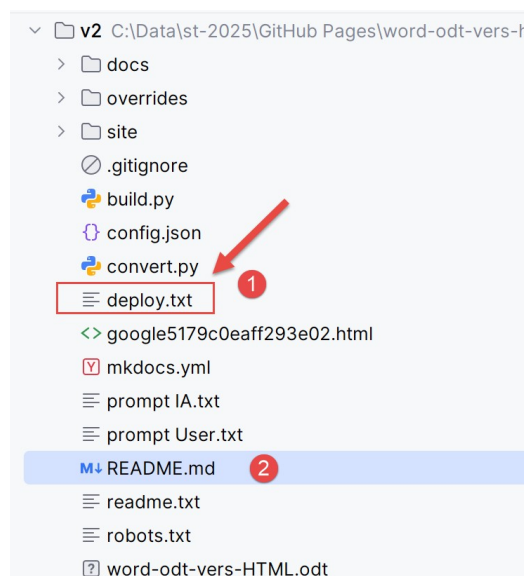
```

54. python convert.py votre-document.odt config.py
55.
56. ```
57.
58.
59. *Cela va générer un dossier `docs/` contenant les fichiers Markdown et un fichier
60. `mkdocs.yml`.
61. 2. **Prévisualisation**
62. Pour voir le site en local :
63. ```bash
64. mkdocs serve
65. ```
66.
67.
68. 3. **Génération**
69. Pour construire le site statique (dossier `site/`) :
70. ```bash
71. mkdocs build
72. ```
73.
74.
75.
76.
77. ## ⚙️ Configuration (`config.py`)
78.
79. Le fichier `config.py` permet de contrôler l'apparence du site :
80.
81. * **mkdocs** : Paramètres généraux du site (titre, description, thème Material).
82. * **footer** : Code HTML complet pour personnaliser le pied de page.
83. * **code** : Règles de détection des langages pour la coloration syntaxique.
84. * **extra** : Configuration de Google Analytics (GA4).
85.
86. ## 📄 Licence
87.
88. Ce cours tutoriel écrit par Serge Tahé est mis à disposition du public selon les termes de
89. la :
90. *Licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes
91. Conditions 3.0 non transposé.*

```

Es un README que me dio. Eso es porque Gemini conoce muy bien este proyecto y llevamos semanas trabajando en él. Probablemente obtendrás un [README.md] menos detallado.

Ahora volvamos a nuestro archivo de trabajo:



- En [2], el fichero README que acaba de modificar;
- En [1], el archivo [deploy.txt] explica cómo exportar su sitio HTML a su repositorio GitHub;

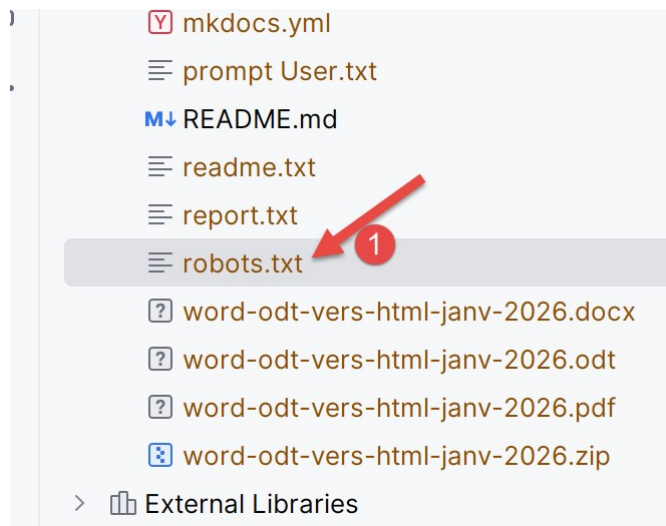
El contenido del fichero [deploy.txt] es el siguiente:

```
1. git init
2. git branch -M main
3. # vérifier .gitignore avant !
4. git add .
5. # premier commit
6. git commit -m "Initial commit: Source MkDocs"
7. git remote add origin https://github.com/stahe/word-odt-vers-html-janv-2026.git
8. git push -u origin main
9. python -m mkdocs gh-deploy
```

Esta es la secuencia de comandos que exportará su sitio HTML a su repositorio GitHub. Deberá modificar la línea 7 con el URL de su propio repositorio GitHub, presente en el fichero [config.py] :

```
1. "repo_url": "https://github.com/stahe/word-odt-vers-html-janv-2026",
```

Debe comprobar también otro URL en el fichero [robots.txt] :



El contenido del fichero [robots.txt] es el siguiente:

```
1. User-agent: *
2. Allow: /
3. Sitemap: https://stahe.github.io/word-odt-vers-html-janv-2026/sitemap.xml
```

En la línea 3, introduzca el URL de su sitio, el mismo que en el archivo [config.py] :

```
1. # URL de publication du site (ex: GitHub Pages)
2. "site_url": "https://stahe.github.io/word-odt-vers-html-janv-2026/",
```

Le archivo [robots.txt] no se utiliza cuando el sitio se construye localmente, pero sí cuando se aloja en GitHub.

El conjunto de comandos [deploy.txt] utiliza un software llamado Git. Necesitas instalarlo [[Git - Instalación para Windows](#)].

Una vez hecho esto, comprueba el archivo llamado [.gitignore] en tu carpeta de trabajo. Le dice a Git qué archivos debe ignorar. Mi archivo [.gitignore] es el siguiente:

```

1.      # 1. On ignore TOUS les fichiers et dossiers du projet
2.      *
3.
4.      # 3. On fait une exception (!) pour le seul fichier que vous voulez garder
5.      !README.md

```

Es extremadamente sencillo. Ignora todos los archivos (línea 2) excepto el archivo [README.md] (línea 5). GitHub está diseñado para alojar proyectos de desarrollo. En general, todo el proyecto de desarrollo se exporta a GitHub. Nosotros simplemente queremos exportar un sitio HTML, no un proyecto de desarrollo. El único archivo que queremos exportar a nuestro repositorio GitHub es el archivo [README.md] que explica a los visitantes lo que contiene nuestro sitio HTML.

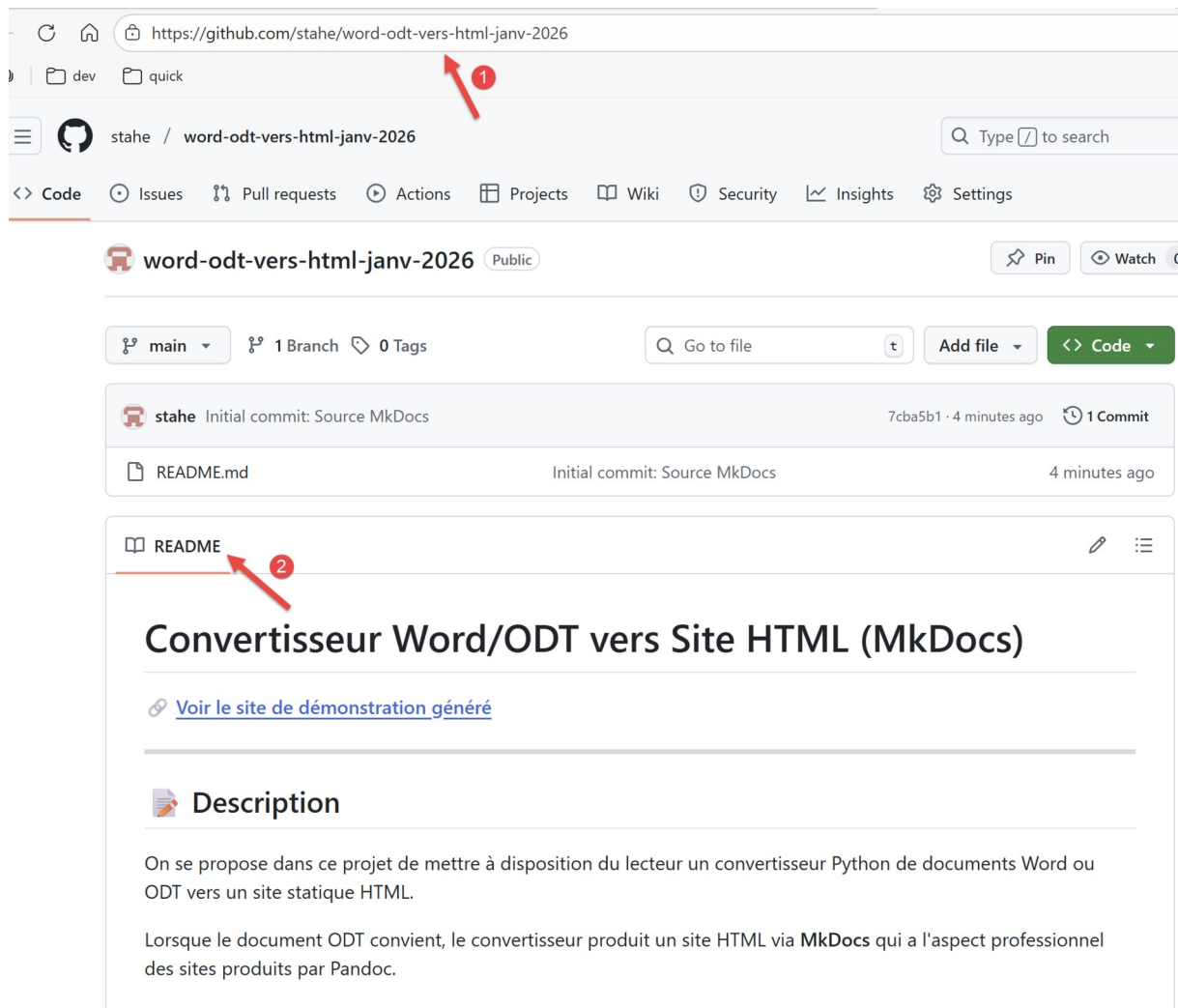
Ahora, en su terminal, escriba la siguiente secuencia de comandos en el orden indicado por [deploy.txt] hasta el comando 8. No ejecute el comando 9 por el momento.

```

1.      PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> git init
2.      Initialized empty Git repository in C:/Data/st-2025/GitHub Pages/word-odt-vers-html/v2/.git/
3.      PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> git branch -M main
4.      PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> git add .
5.      PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> git commit -m "Initial commit: Source
MkDocs"
6.      [main (root-commit) 7cba5b1] Initial commit: Source MkDocs
7.      1 file changed, 89 insertions(+)
8.      create mode 100644 README.md
9.      PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> git remote add origin
https://github.com/stahe/word-odt-vers-html-janv-2026.git
10.     PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> git push -u origin main
11.     Enumerating objects: 3, done.
12.     Counting objects: 100% (3/3), done.
13.     Delta compression using up to 8 threads
14.     Compressing objects: 100% (2/2), done.
15.     Writing objects: 100% (3/3), 1.70 KiB | 1.70 MiB/s, done.
16.     Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
17.     To https://github.com/stahe/word-odt-vers-html-janv-2026.git
18.      * [new branch]      main -> main
19.     branch 'main' set up to track 'origin/main'.

```

Ctrl-clíc en el URL en la línea 17. Esto le llevará a su repositorio GitHub:



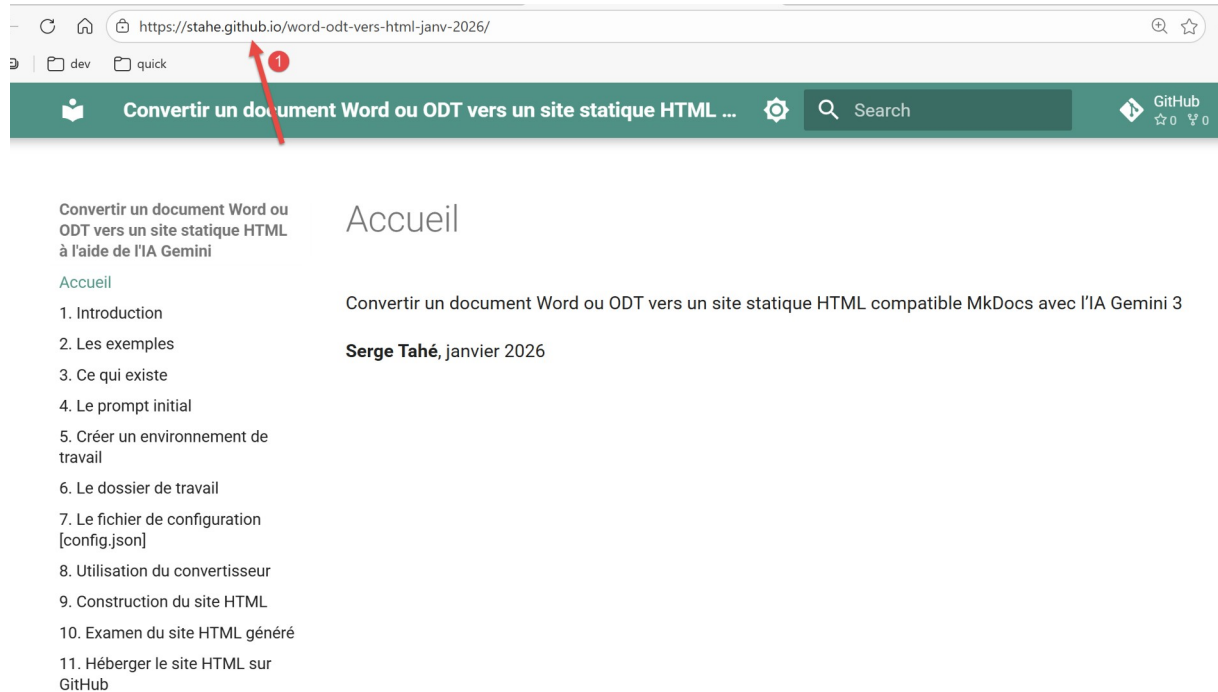
- En [1], el URL de su depósito GitHub ;
- En [2], el nuevo README.md ;

Ahora vamos al comando 9 en el archivo [deploy.txt]. Esto exporta el sitio HTML a GitHub :

```
1. PS C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2> python -m mkdocs gh-deploy
2. INFO - Cleaning site directory
3. INFO - Building documentation to directory: C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2\site
4. INFO - Doc file 'les-exemples.md' contains a link '#_Les_exemples', but there is no such anchor on this page.
5. INFO - Documentation built in 1.79 seconds
6. WARNING - Version check skipped: No version specified in previous deployment.
7. INFO - Copying 'C:\Data\st-2025\GitHub Pages\word-odt-vers-html\v2\site' to 'gh-pages' branch and pushing to GitHub.
8. Enumerating objects: 91, done.
9. Counting objects: 100% (91/91), done.
10. Delta compression using up to 8 threads
11. Compressing objects: 100% (85/85), done.
12. Writing objects: 100% (91/91), 1.64 MiB | 2.01 MiB/s, done.
13. Total 91 (delta 9), reused 0 (delta 0), pack-reused 0 (from 0)
14. remote: Resolving deltas: 100% (9/9), done.
15. remote:
16. remote: Create a pull request for 'gh-pages' on GitHub by visiting:
17. remote: https://github.com/stahe/word-odt-vers-html-janv-2026/pull/new/gh-pages
```

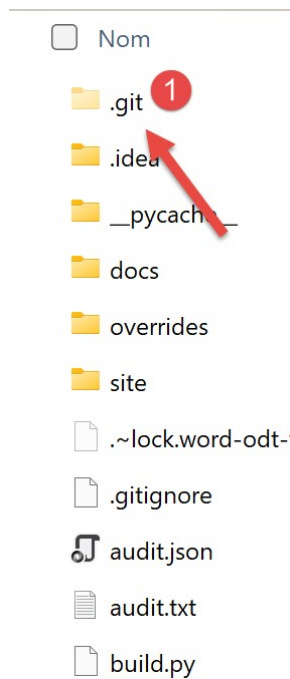
```
18. remote:
19. To https://github.com/stahe/word-odt-vers-html-janv-2026.git
20. * [new branch]      gh-pages -> gh-pages
21. INFO - Your documentation should shortly be available at: https://stahe.github.io/word-odt-vers-html-janv-2026/
```

Ctrl-clic en el URL en la línea 21. Esto le llevará a su nuevo sitio HTML en GitHub :



- En [1], vemos que está mostrando un sitio web en GitHub ;

Es extremadamente fácil cometer un error al ejecutar comandos [deploy.txt]. Luego es difícil volver atrás porque Git guarda la memoria de lo que hiciste (mal). Para empezar desde cero, consulta el archivo :



- En [1], elimine la carpeta [.git];

Luego vuelve a PyCharm y repite la serie de comandos desde [deploy.txt].

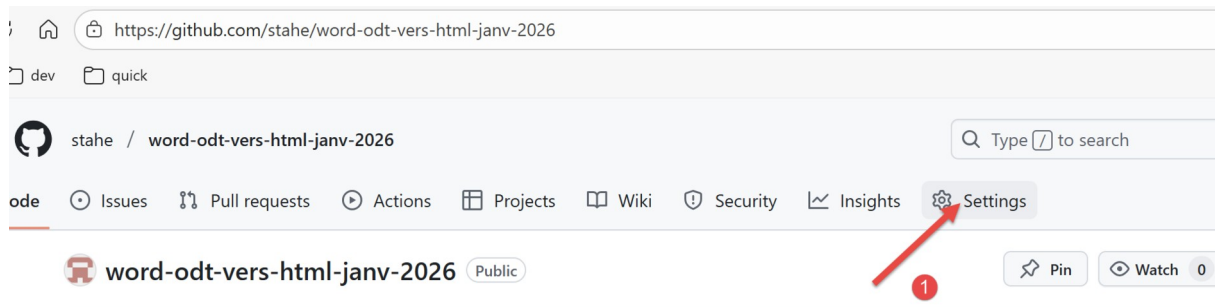
¿Qué debe hacer si modifica su Documento ODT / DOCX? Haga las 3 cosas siguientes:

1. convierte tu Documento ODT / DOCX de nuevo con [convert] ;
2. construir el sitio HTML con [build] ;
3. exporte el sitio HTML a GitHub con el comando [python -m mkdocs gh-deploy]. Este comando es suficiente siempre que no modifique el archivo [README.md]. Si modifica el archivo [README.md], tendrá que ejecutar algunos comandos más:

```
1. git add README.md
2. # commit
3. git commit -m "modif README"
4. git push -u origin main
5. python -m mkdocs gh-deploy
```

Si sólo ha modificado README, sólo son necesarios los comandos 1, 3 y 4. El comando 5 es inútil si ya ha desplegado el sitio HTML y no se ha movido desde entonces.

¿Qué debes hacer si quieres empezar de cero porque las cosas han ido mal? Puedes borrar tu repositorio GitHub y rehacer todas las operaciones del capítulo 12. El option para borrar un repositorio GitHub está bien escondido :



- En [1], vaya a los parámetros de configuración del repositorio ;

Danger Zone

Change repository visibility
 This repository is currently public.

Change visibility

Disable branch protection rules
 Disable branch protection rules enforcement and APIs

Disable branch protection rules

Transfer ownership
 Transfer this repository to another user or to an organization where you have the ability to create repositories.

Transfer

Archive this repository
 Mark this repository as archived and read-only.

Archive this repository

Delete this repository
 Once you delete a repository, there is no going back. Please be certain.

Delete this repository

Vaya a la parte inferior de la página [configuración]. Allí encontrarás el botón para borrar tu repositorio [2].

13. Aseguiamiento del sitio con Google Analytics y Google Search Console

Si aloja su sitio en GitHub probablemente estará interesado en conocer el número de visitantes del sitio. Para ello, necesita una etiqueta de Google Analytics (GA). Le pedimos [ChatGPT](#) cómo crear un marcador GA :

comment obtient-t-on un marqueur Google Analytics pour un site web ?

Pour un site web, le **marqueur Google Analytics** correspond aujourd'hui à l'**identifiant de mesure GA4** (format **G-xxxxxxxx**). Voici la procédure complète et à jour pour l'obtenir.

1. Créer ou utiliser un compte Google Analytics

1. Accédez à Google Analytics et connectez-vous avec votre compte Google.
2. Si aucun compte n'existe encore :
 - Cliquez sur **Créer un compte**
 - Donnez un nom au compte (ex. : *Entreprise X*)
3. Si un compte existe déjà, sélectionnez-le.

2. Créer une propriété GA4

1. Dans **Administration** (icône engrenage, en bas à gauche),
2. Colonne **Propriété** → **Créer une propriété**,
3. Renseignez :
 - Nom de la propriété
 - Fuseau horaire
 - Devise
4. Validez.

Depuis plusieurs années, **GA4 est le seul type de propriété disponible**. Les anciennes propriétés Universal Analytics ne sont plus utilisables.

3. Créer un flux de données Web

1. Toujours dans **Administration**,
2. Colonne **Propriété** → **Flux de données**,
3. Cliquez sur **Ajouter un flux** → **Web**,
4. Saisissez :
 - L'URL du site (ex. `https://www.monsite.fr`)
 - Un nom de flux
5. Validez.

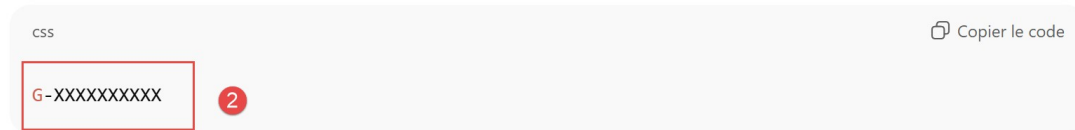


- En [1], établissez l'URL de votre site web en GitHub ;

4. Récupérer le marqueur (ID de mesure)

Après la création du flux web :

- Un **ID de mesure** est affiché, au format :



👉 C'est ce marqueur Google Analytics

Il identifie de manière unique votre site dans GA4.

- En [2], anote el marcador de su sitio ;

Coloque este marcador en el archivo de configuración [config.py] :

```
1.     "extra": {
2.         "analytics": {
3.             "provider": "google",
4.             "property": "G-XXXXXXXX"
5.         }
6.     },
```

En la línea 4, introduzca su marcador. MkDocs lo pondrá automáticamente en todas sus páginas HTML cuando genere el sitio HTML. Por lo tanto, no necesita seguir las otras instrucciones de ChatGPT que muestran cómo instalar el marcador GA en sus páginas HTML.

Vamos a pedir a ChatGPT instrucciones precisas para comprobar que el marcador funciona:



donne moi des instructions plus précises pour vérifier que le marqueur fonctionne. Donne moi l'URL à utiliser.

Voici une procédure **opérationnelle et vérifiable pas à pas** pour contrôler que votre marqueur GA4 fonctionne, avec l'**URL exacte à utiliser**.

Méthode recommandée : vérification via le rapport *Temps réel*

1. Accéder à Google Analytics

Ouvrez directement l'interface de Google Analytics à l'adresse suivante :

<https://analytics.google.com/> ➤

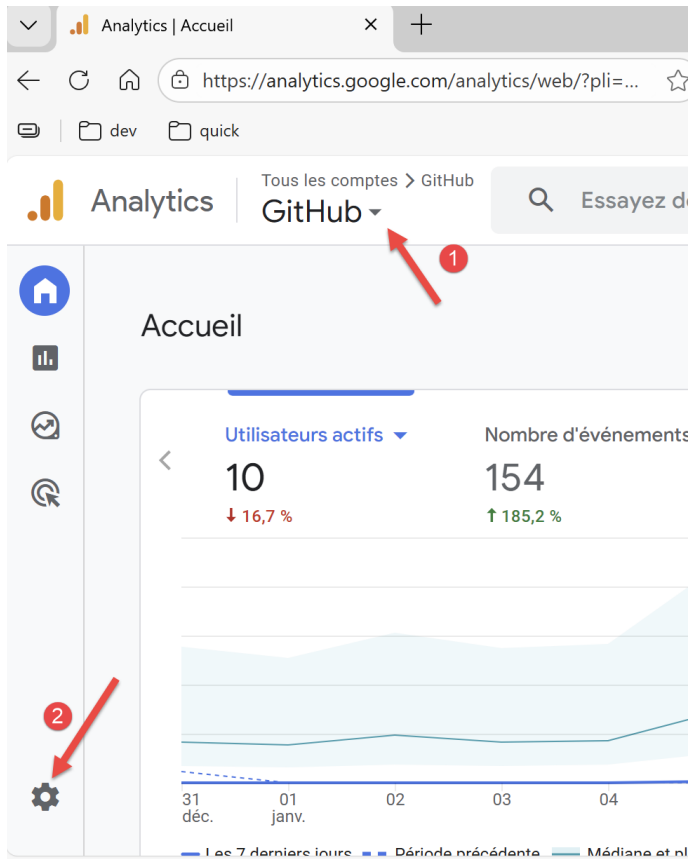
Connectez-vous avec le compte Google qui possède la propriété GA4.

2. Sélectionner la bonne propriété

En haut à gauche de l'interface :

1. Vérifiez que la **propriété GA4** correspondant à votre site est bien sélectionnée.
2. Si nécessaire, changez-la via le sélecteur de compte/propriété.

El paso anterior no está suficientemente claro. Aquí hay algunas capturas de pantalla para ayudarle:



- En [1], sélectionne la cuenta que desea seguir (si tiene más de una);
- En [2], sélectionne el botón de administración ;

Analytics | Tous les comptes > GitHub

Essayez de rechercher "comment créer un entonnoir de conversion"

Administrateur

Ces paramètres s'appliquent à tous les utilisateurs de ce compte et de cette propriété. Pour voir ceux qui ne s'appliquent qu'à vous, accédez à [Mes préférences](#) dans le panneau de navigation de gauche.

PARAMÈTRES DU COMPTE

Compte

Ces paramètres ont une incidence sur votre compte Analytics [Qu'est-ce qu'un compte ?](#)

- Détails du compte
- Gestion des accès au compte
- Historique des modifications du compte
- Historique des quotas de l'API Account data
- Corbeille

PARAMÈTRES DE LA PROPRIÉTÉ

Propriété

Ces paramètres ont une incidence sur votre propriété [Qu'est-ce qu'une propriété ?](#)

- Détails de la propriété
- Gestion des accès à la propriété

Collecte et modification des données

Ces paramètres déterminent la manière dont les données sont collectées et modifiées

- Flux de données
- Collecte des données

- En [3], sélectionnez [Flujo de datos];

Flux de données

Toutes les vues | iOS | Android | Web

Ajouter un flux

Propriété	ID de flux	Description
GitHub https://stahe.github.io	13112486249	La propriété a reçu du trafic au cours des dernières 48 heures.

- En [4], cliquez sur le flux de données ;

× Détails du flux Web

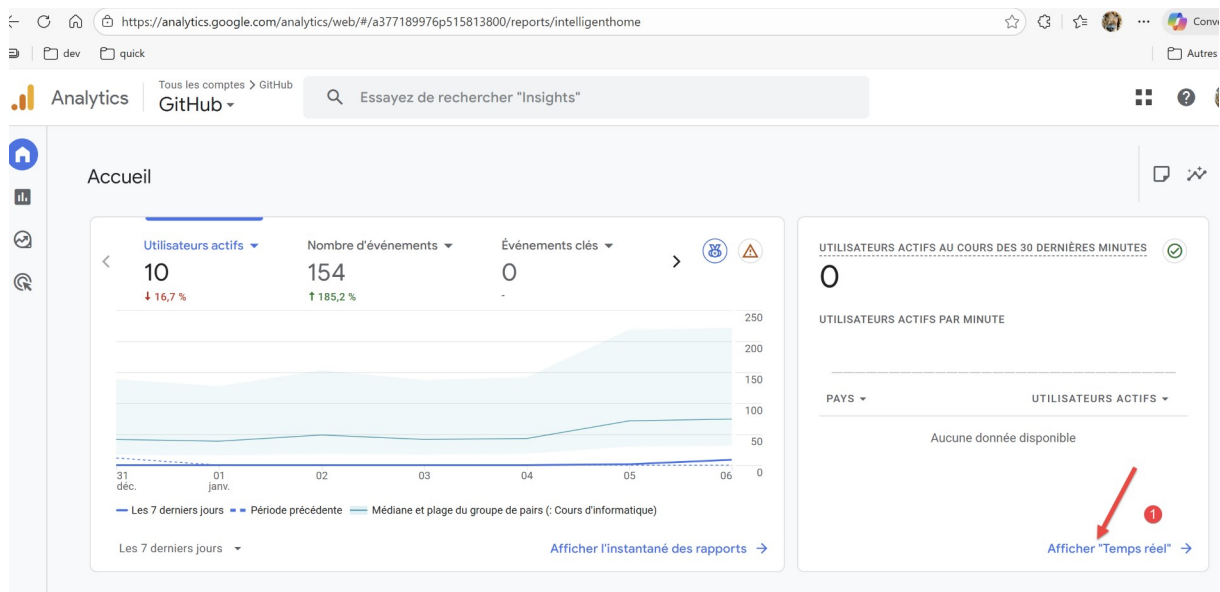
✓ La collecte de données a été active au cours des 48 dernières heures.

Détails du flux

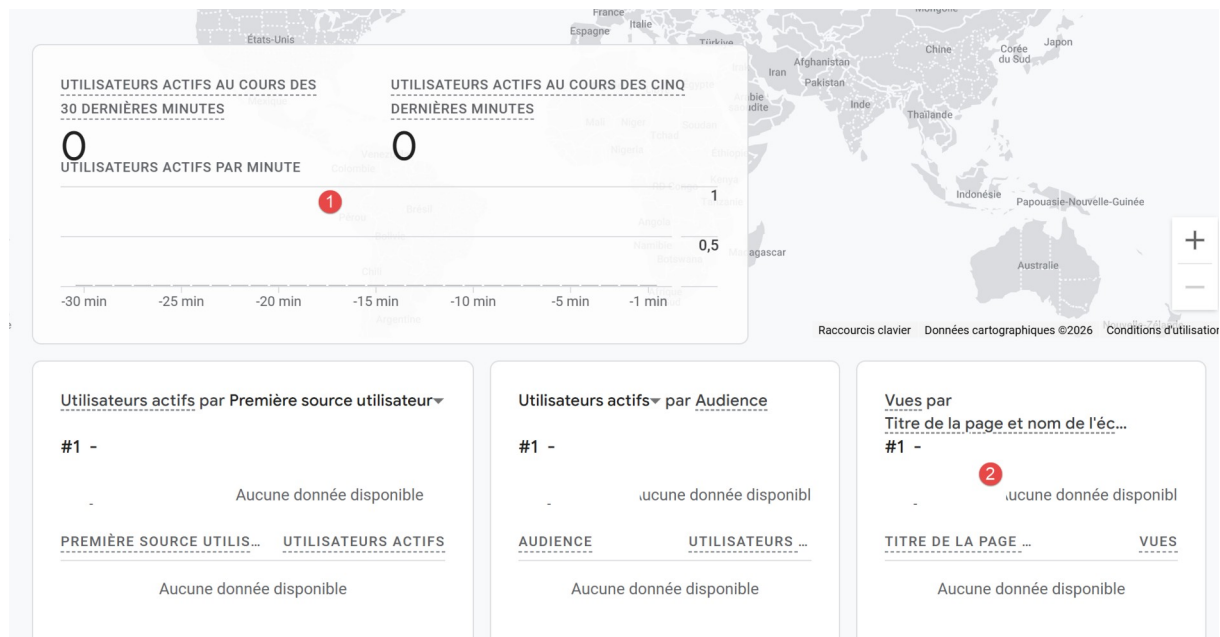
NOM DU FLUX	URL DE FLUX	ID DE FLUX	ID DE MESURE
GitHub	https://stahe.github.io	13112486249	G-EBZC

- En [5], se encuentra el marcador GA4 que ha creado;
- En [6], el ID de flujo. Este es un concepto diferente de la medición ID ;

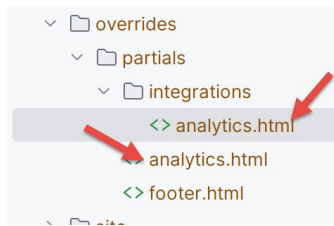
Volvamos a la bienvenida en GA :



- En [1], se solicita la visualización en tiempo real de las visitas;



- En [1], ves a tus visitantes;
- En [2], la página visitada. He pedido al conversor Gemini que asocie el nombre del sitio en lugar del nombre de la página a la etiqueta GA4. Es el archivo [analytics.html] el que realiza este cambio de nombre asociado a GA4 :



Mahora podemos continuar con las explicaciones de ChatGPT :

4. Générer une visite test propre

Pour éviter toute ambiguïté :

1. Ouvrez une **fenêtre de navigation privée** (Ctrl+Maj+N ou Ctrl+Maj+P).
2. Saisissez l'URL exacte de votre site (ex. `https://www.monsite.fr`).
3. Naviguez sur une ou deux pages.
4. Attendez 5 à 30 secondes.

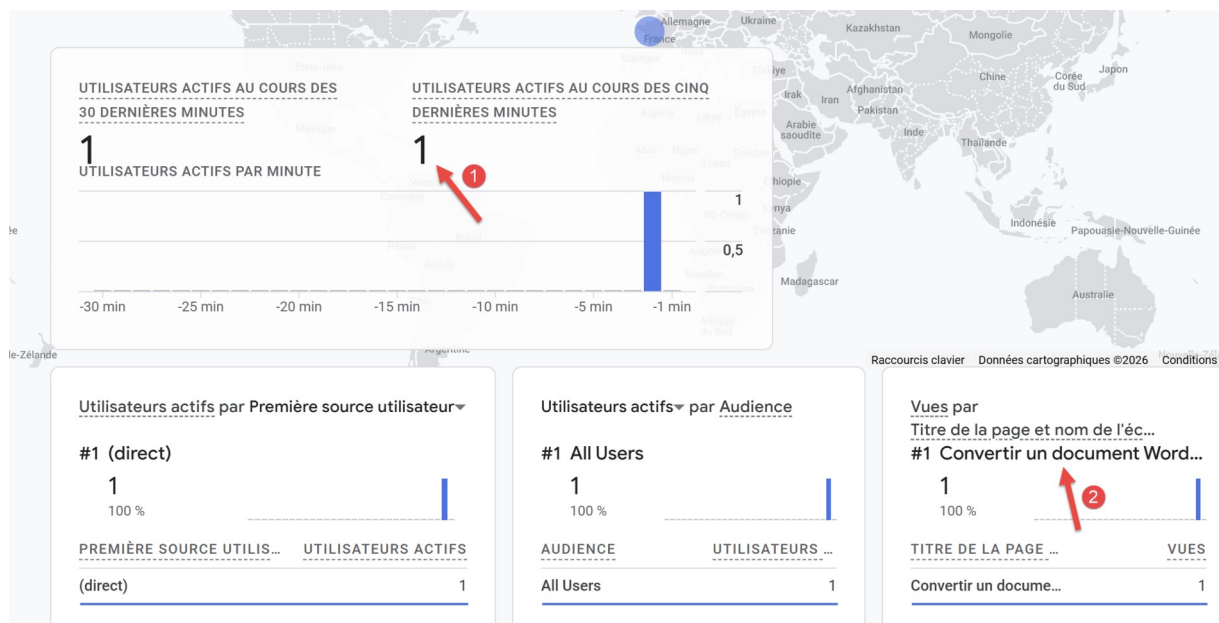
5. Résultat attendu

Dans le rapport *Temps réel*, vous devez voir au minimum :

- 1 utilisateur actif
- Une page consultée
- Une source de trafic (souvent *Direct*)

Si ces éléments apparaissent → le marqueur fonctionne correctement.

Hice lo que se pide en [4] sin pasar por una ventana privada. Y obtuve lo siguiente:



El marcador GA4 de Google Analytics es totalmente funcional.

Ahora vamos a presentar otra herramienta para monitorizar tu sitio [Google Search Console]. Preguntemos a ChatGPT para qué sirve esta herramienta:

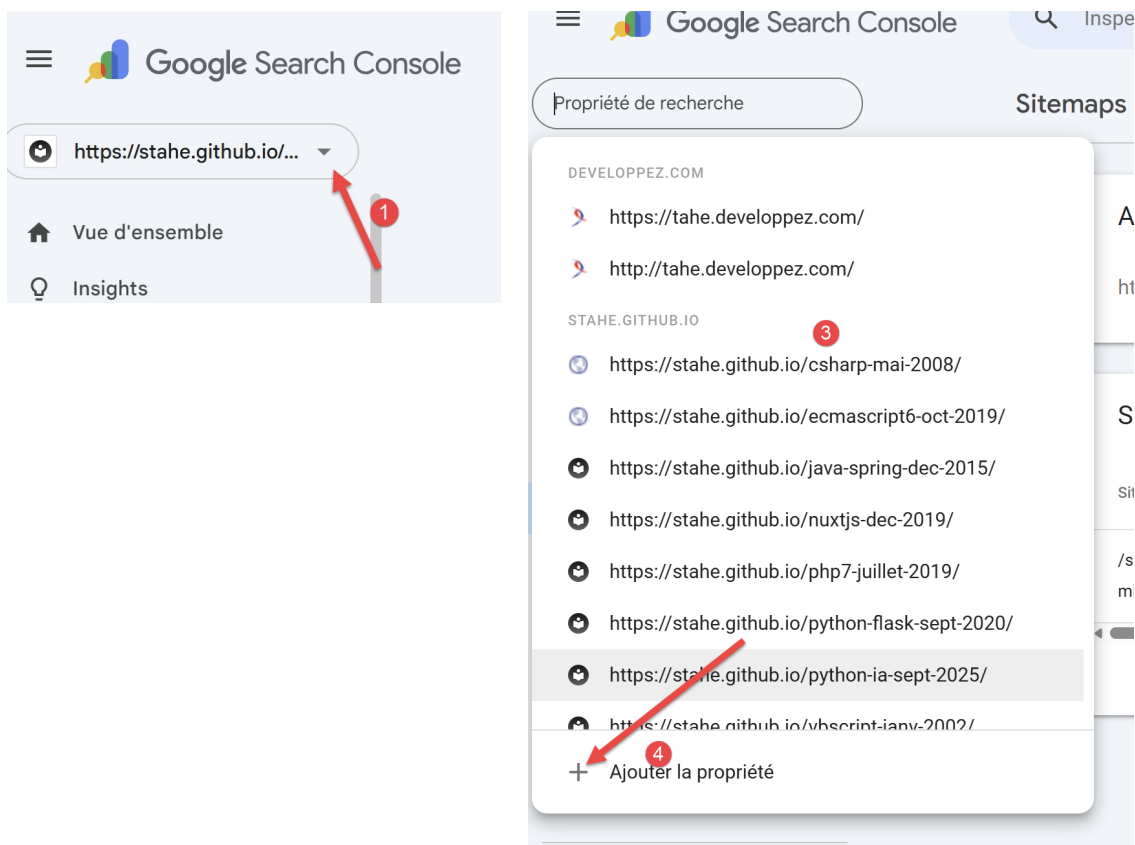
Conclusion

Google Search Console sert à :

- comprendre **comment Google perçoit votre site**,
- améliorer votre **référencement naturel (SEO)**,
- détecter rapidement les **problèmes bloquants**,
- piloter votre visibilité dans Google Search.

Casi nunca utilizo esta herramienta. Sólo la utilizo para animar al motor de búsqueda de Google a explorarlo e indexarlo para que la gente pueda encontrarlo.

El URL de la [<https://search.google.com/search-console>] :



- Véase [1] para consultar la lista de sitios declarados a Google;
- Consulte [3] para obtener una lista de sus sitios;
- En [4], añada su nuevo sitio;

Sélectionnez le type de propriété



Domaine

- Toutes les URL de tous les sous-domaines (m. www. ...)
- Toutes les URL sur https et http
- Validation DNS obligatoire

Saisissez un domaine ou sous-domaine

CONTINUER

ou



Préfixe de l'URL

- Seules les URL de l'adresse saisie
- Seules les URL du protocole spécifié
- Différentes méthodes de validation acceptées

Saisissez une URL

CONTINUER

- En [5], introduzca el URL de su sitio. Se encuentra en el archivo [config.py]:

```
1. "site_url": "https://stahe.github.io/word-odt-vers-html-janv-2026/",
```

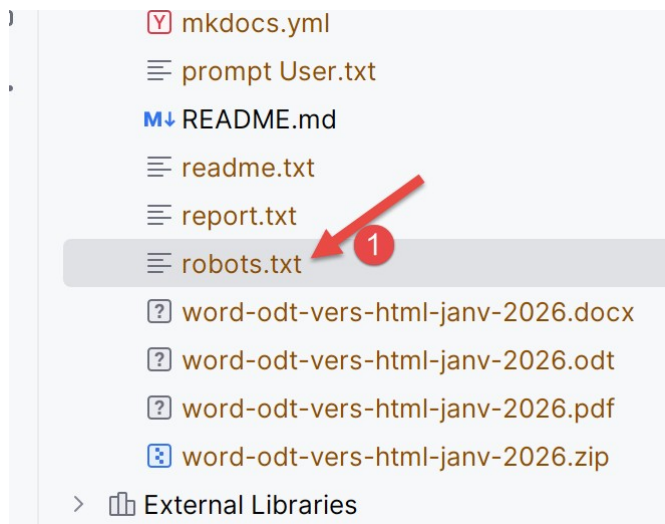
- en [6a continuación, vaya al siguiente paso ;

Normalmente, en esta fase, Google Search Console te ofrece un archivo [googlexxxxx.html] que debes descargar. A mí no me apareció esta pantalla porque ya tenía este archivo en el sitio desplegado. Se trata de uno de los dos archivos declarados en [config.py]

```
1. "files_to_copy": [  
2.     "google5179xxxxx.html",  
3.     "robots.txt"  
4. ]
```

Ponga el archivo [googlexxxxx.html] en la raíz de su carpeta de trabajo e introduzca su nombre en la línea 2 anterior en el archivo [config.py]. El script [convert] se encarga de poner los dos archivos anteriores en la raíz del sitio MkDocs que genera. El script [build] se encargará de ponerlos en la raíz del sitio HTML que genera.

Cuando haya copiado el archivo [googlexxxxx.html] en la raíz de su carpeta de trabajo, comprobar el contenido del fichero [robots.txt] :



El contenido del fichero [robots.txt] es el siguiente:

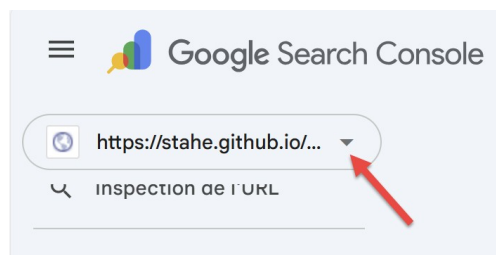
```
1. User-agent: *
2. Allow: /
3. Sitemap: https://stahe.github.io/word-odt-vers-html-janv-2026/sitemap.xml
```

En línea 3, comprobar que el URL es que de su sitio, el mismo que en el archivo [config.py].

Replote su sitio web en GitHub escribiendo los tres comandos siguientes en su terminal Python :

```
1. python .\convert.py .\word-odt-vers-HTML.odt config.py
2. python .\build.py
3. python -m mkdocs gh-deploy
```

Una vez hecho esto, vuelve a [Mar de Googlerch Console] y seleccione la propiedad que creó anteriormente.



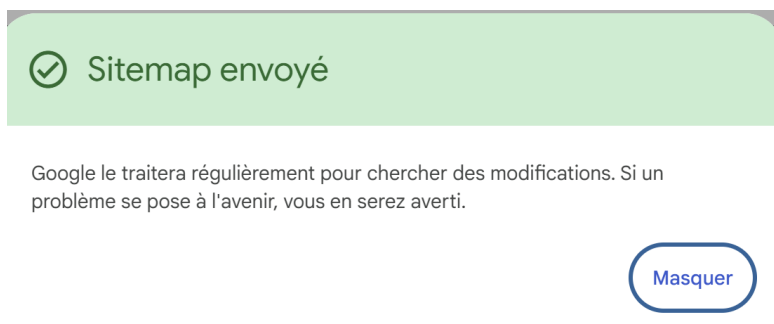
- En [1], seleccione option [Sitemaps] ;
- En [2], tipo [sitemap.xml] ;
- En [3], confirme URL ;

El fichero [sitemap.xml] es un fichero que Mkdocs puso en la raíz del sitio HTML exportado a GitHub. Puede comprobar su presencia escribiendo directamente su URL:

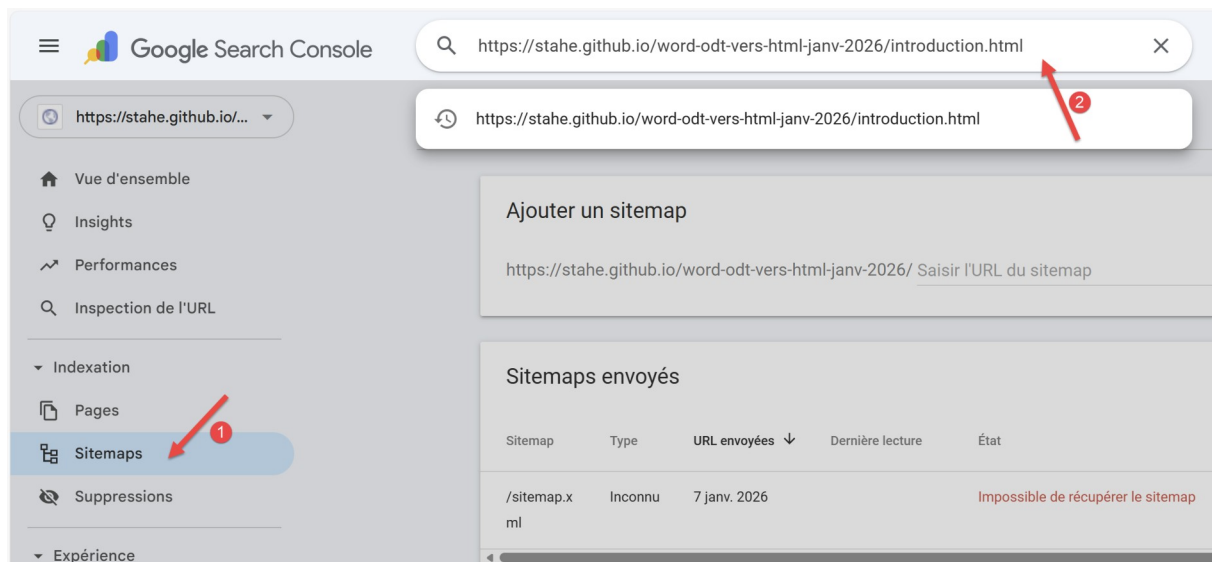


El archivo [sitemap.xml] enumera todas las páginas HTML del sitio.

Si todo va bien, aparecerá la siguiente pantalla:

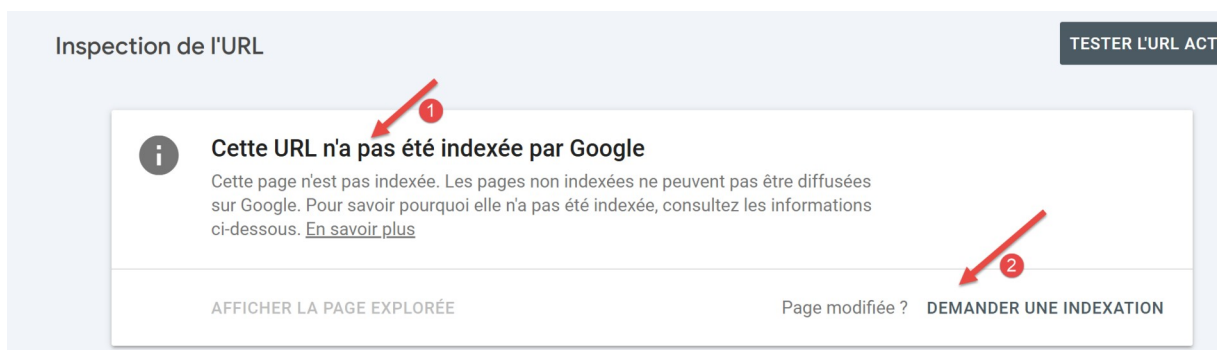


Eso es todo. Tendrá que indexar las páginas usted mismo:



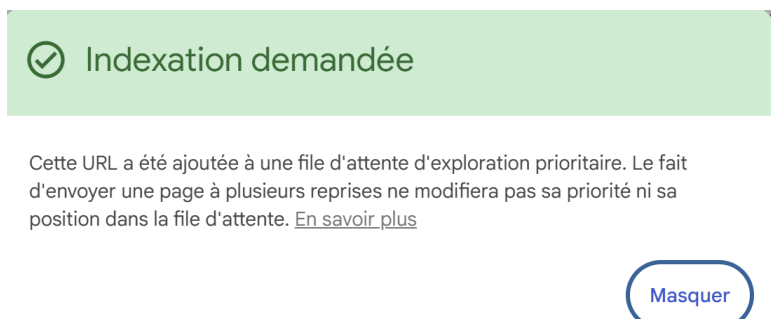
- En [2], introduzca el URL de una de las páginas de su sitio;

Ici, el resultado es el siguiente:



- En [1], Google dice que la página aún no está indexada;
- En [2], puede solicitar su indexación;

Google Search Console comprueba que la página solicitada puede ser indexada. Si es así, obtenemos el siguiente mensaje:



U puede hacer esto para todas las páginas de su sitio, de modo que pueda estar seguro de que su sitio es seguro y que sea indexado por el motor de búsqueda Google.

14. Conclusión

Este documento muestra un conversor de documentos ODT producido por [LibreOffice](#) o documentos DOCX de Word en un sitio estático HTML de muy buena calidad.

En el capítulo [[Los ejemplos de este documento](#)] estructuras gestionadas correctamente por el convertidor Gemini / ChatGPT.

También hemos mostrado cómo exportar a GitHub los sitios HTML creados por los conversores ODT y HTML DOCX (véase el capítulo [Aloje el sitio web HTML en GitHub](#))

Aparte de las estructuras del capítulo de ejemplos, nada está garantizado. Es probable que el conversor muestre anomalías. Entonces tendrá que iniciar una nueva conversación con Gemini o ChatGPT para resolver estas anomalías. La forma más fácil es dar el convertidor actual al IA con su archivo de configuración. Luego preguntar al IA si entiende los códigos que se le han dado. Esto le obliga a analizar el código. Siempre responderá que entiende el código perfectamente. Este es el momento de pedirle los cambios deseados. Tanto para Word como para LibreOffice, puedes ayudar al IA con macros. El IA no "ve" el documento ODT o DOCX que se está utilizando. Por lo tanto, le falta información. Por ejemplo, el conversor DOCX -> HTML no ha podido gestionar correctamente la numeración de los códigos ricos. No podía leer el número de la primera línea de un bloque de código para representarlo en HTML. Siempre empezaba el código en 1 aunque el código en el DOCX empezara en 12. A continuación utilicé una macro. Coloqué el cursor del documento en la línea numerada y pedí a ChatGPT que generara una macro VBA que le diera las características de esta línea, permitiéndole generar la numeración HTML correctamente. ChatGPT lo hizo y yo le entregué el resultado de su macro. La información recuperada le mostró que el número de línea era propagado por su estilo y no por el icono de numeración. Entonces cambió su algoritmo, que funcionó. Así podemos ayudar al IA a avanzar.

Otra forma más sencilla es cambiar tu documento. Cuando el conversor produzca una anomalía, prueba a cambiar tu documento para que sólo utilice las estructuras del capítulo de ejemplos.